



LMBUILD: EVALUATING LLM AGENTS FOR GENERATING BUILDABLE AND FUNCTIONAL STRUCTURES

Jiateng Liu, Rushi Wang, Cheng Qian, Xuejun Zhang, Sun Li, Jiayu Liu,
Yifan Shen, Xu Cao, Jiarui Yao, Bingxuan Li, Ruhi Sarikaya, Heng Ji

University of Illinois Urbana-Champaign

{jiateng5, hengji}@illinois.edu

Project Page: <https://lumos-jiateng.github.io/LMBuild/>

Github: <https://github.com/Lumos-Jiateng/LMBuild>

Dataset: <https://huggingface.co/datasets/Lumos-Jiateng/LMBuild>

ABSTRACT

Large Language Model (LLM)-based agents are increasingly capable of generating complex 3D structures, with the potential to reshape how objects are designed and realized in the physical world. Yet, producing elegant geometry is fundamentally different from producing objects that can actually be built and perform their intended functions. Existing evaluations largely focus on geometric or visual quality while overlooking structural soundness, functional affordances, and physical realizability. We introduce **LMBuild**, a benchmark for evaluating LLM agents on generating buildable and functional structures. LMBuild represents generated objects as assembled structures comprising meaningful part decompositions, joints, materials, and assembly sequences. To support reproducible evaluation, we provide a unified framework consisting of: (1) an interactive environment in which agents can use tools to retrieve, create, and place components to construct an object; (2) a curated benchmark that repurposes established CAD datasets and augments them with real-world object knowledge from Wikipedia; and (3) a four-level evaluation framework covering structural soundness, functional affordance, design quality, and physical realization. Evaluations across 30 domain-specific models, open-source LLMs, and frontier APIs reveal several intriguing findings: (a) Structural soundness and visual alignment are no longer the primary bottlenecks for frontier closed-source models, while functional affordance and physical operability remain substantially more challenging; (b) stronger models more effectively exploit part creation to construct structure-specific components, whereas weaker models tend to rely on retrieval and can struggle with the additional flexibility introduced by creation; and (c) providing explicit functional specifications substantially improves functional-part completeness, kinematic correctness, and overall physical operability. These results show that generating real-world structures requires capabilities beyond geometric synthesis, including deeper reasoning about functional affordances, mechanics, and physical constraints, as well as the ability to design and create novel components when existing ones are insufficient. We expect LMBuild to provide a foundation for measuring progress and incentivizing research toward agents that generate buildable and functional structures, while helping narrow the gap between open-source communities and frontier labs.

1 INTRODUCTION

The recent release of GPT-6 Astra (OpenAI, 2026b) has sparked an explosion of discussion around how much LLM-based agents could transform 3D structure generation. Beyond their established capabilities in language and code, LLM-based agents are increasingly able to directly manipulate complex 3D assets and scenes (Hu et al., 2024; Sun et al., 2025; Tang et al., 2025b; Liu et al., 2026a) and translate language-based specifications into parametric or programmatic 3D designs (Li et al., 2025; Wang et al., 2025b; Alrashedy et al., 2025). These developments lead us to explore a

more ambitious possibility: whether LLM agents could reshape not only how structures are digitally designed, but also how they are constructed and realized in the physical world (Kyaw et al., 2025).

However, producing visually plausible geometry is fundamentally different from producing an object that can actually be built and function in the real world. A structure may appear convincing while remaining invalid, unusable, or impossible to construct. Existing 3D-generation evaluations largely emphasize geometric quality (Poole et al., 2023; Lin et al., 2023), visual consistency (Wang et al., 2023; Maiti et al., 2025; He et al., 2023; Duggal et al., 2025), and semantic alignment (Wu et al., 2024; Zhang et al., 2025), while overlooking critical aspects of physical realization. Yet, we still lack a systematic way to evaluate whether LLM-based agents can leverage their extensive prior knowledge and reasoning capabilities to generate structures that are not only digitally plausible, but also buildable, functional, and aligned with real-world physical constraints.

To this end, we propose **LMBuild**, a first step toward systematically evaluating LLM agents on the generation of buildable and functional structures, while also accommodating domain-specific structure-generation baselines. First, LMBuild provides a unified framework (Figure 1) in which LLM agents are equipped with a standardized agent harness and can interact seamlessly with a 3D environment through a set of construction tools, including retrieving components from a sub-part pool, creating new parts with arbitrary geometry, placing components at desired poses, and refining structures through inspection and revision. Second, as summarized in Table 1, we select 2,549 high-quality reference targets by repurposing existing 3D datasets containing around 368K data, filtering objects by category and model quality, and augmenting them with real-world knowledge extracted from Wikipedia, and further selected 200 objects as the core benchmark dataset for evaluation. This knowledge includes core functional parts, object attributes, joints, and kinematic relationships, which serve as textual anchors for evaluating physically grounded properties. Finally, as shown in Figure 1, we introduce a bottom-up evaluation hierarchy covering four dimensions: (1) structural soundness, assessing whether components form valid and stable 3D assemblies; (2) functional affordance, evaluating whether functional parts, joints, and kinematics support the intended object functions; (3) design quality, assessing decomposition, aesthetics, and overall structural design; and (4) physical realization, evaluating whether the structure can be fabricated with appropriate materials and assembly sequences and ultimately operated or used in the physical world.

We evaluate 30 systems on LMBuild, including 6 closed-source APIs, 13 open-source LLMs, and 11 domain-specific systems, under diverse evaluation settings. Our results show that:

- Structural soundness and visual alignment are no longer primary bottlenecks for frontier closed-source models. Most achieve near-perfect soundness and strong design quality, although open-source models still show room for improvement in aesthetics, and fine-grained visual alignment.
- Functional affordance and physical operability remain the largest capability gaps. All model families struggle with generating a complete list of functional parts, correct joints with kinematics, and artifacts that remain operable under simulation-based evaluation.
- While frontier models can effectively create new components, weaker models rely heavily on retrieval and struggle with the additional reasoning burden introduced by part creation.
- Enriching prompts with desired attributes and functional requirements can force the model to reason, thus improving functional-part completeness, kinematic correctness, and physical operability.

We further show that LMBuild can diagnose failure patterns for open-source LLM agents during structure construction, revealing their gap in part-creation capability as well as persistent challenges in scene-state tracking. Overall, we expect LMBuild to provide a foundation for measuring progress and incentivizing research toward agents capable of generating buildable and functional structures.

2 LMBUILD

In this section, we describe the three core components of **LMBuild**: (1) how we construct an interactive environment for evaluating LLM agents and other systems on generating buildable and functional structures (§ 2.1); (2) how we repurpose existing datasets to support large-scale LMBuild evaluation and illustrate the interactive generation process with a concrete running example (§ 2.2); and (3) how we establish a hierarchical evaluation framework for the generated structures (§ 2.3).

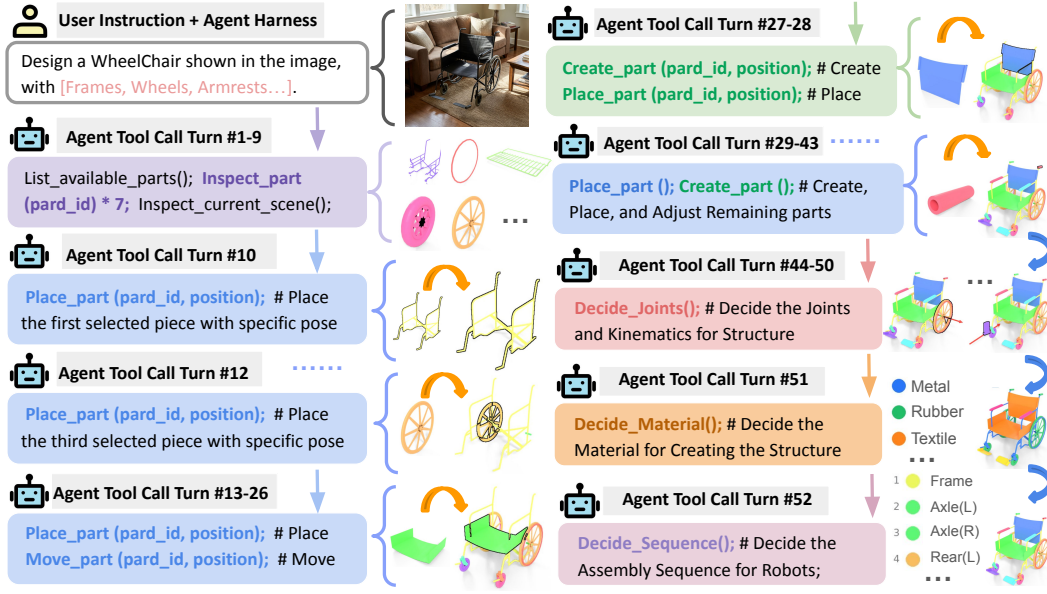


Figure 1: **LMBuild Framework** (Using real example produced by GPT-6 Astra). Given a text prompt and conditional image, the builder LLM agent uses tools to inspect and select existing parts, create new parts, and place and adjust components to construct the wheelchair. It then specifies joints, materials, and an assembly sequence to support kinematics and physical-world realization.

2.1 INTERACTIVE GENERATION OF BUILDABLE AND FUNCTIONAL STRUCTURES

As shown in Figure 1, LMBuild constructs an interactive environment in which agents are equipped with a manually designed harness (detailed in Appendix A.1) and instructed to follow user-provided construction specifications. The agents interact with a set of tools (detailed in Appendix A.2) to generate decomposed and buildable structures, determine appropriate joints and materials for individual components, and specify a feasible assembly sequence for real-world construction.

Baseline Interaction Protocol. LMBuild simulates a real-world setting in which users can “speak structures into existence” (Kyaw et al., 2025). The user provides only a target structure label and a reference image, without geometric specifications, part-level instructions, or intermediate feedback. The agent then autonomously constructs the object in a single interaction round through tool calls (Figure 1), either retrieving components from a candidate pool containing at least three times the required number of parts or creating new parts when needed. It selects or creates components, places and adjusts them, and specifies the final pose. For domain-specific systems, we preserve their native input and output protocols rather than introducing unsupported capabilities. When a system produces geometry without articulation information, we apply Particulate (Li et al., 2026) to infer part-level articulation, joints, and kinematic constraints for evaluation under LMBuild metrics.

Part Retrieval and Creation. The baseline allows agents to **either** retrieve existing components or create new ones when needed. To isolate the contribution of each mechanism, we additionally evaluate two constrained settings: **retrieval-only**, where agents must construct the target structure entirely from the provided component pool, and **creation-only**, where retrieval is disabled and every required component must be generated by the agent. Comparing these settings with the baseline reveals how much agents depend on existing parts versus their ability to design novel components.

Multi-Round Iterative Revision. LMBuild also evaluates whether agents can iteratively improve their constructions using feedback. After each round, the environment returns a six-view rendering and a rule-based diagnostic report covering collisions, disconnected components, stability, joint issues, and missing functional roles, materials, etc. The agent receives this feedback together with

Table 1: Statistics of the source datasets repurposed for **LMBuild**, including original dataset size, objects retained in LMBuild-Core and LMBuild-Full, and average number of Wikipedia-augmented part and affordance annotations obtained. Affordances include functional attributes and kinematic information. Product-level CAD objects are complete open-source hardware assemblies from 27 projects, grouped by domain; We evaluate on the LMBuild-Core split (200 objects) in this work.

Geometry Type	Data Source	Original Size	LMBuild-Core		LMBuild-Full	
			Size	Avg. Parts / Aff.	Size	Avg. Parts / Aff.
LEGO Toy Data	BrickNet (Kulits & Schmid, 2026)	320,808	5	13.80 / 14.60	888	8.79 / 10.82
	BrickComposer (Liu et al., 2026a)	9,977	50	10.16 / 10.34	50	10.16 / 10.34
Static Structures	PartNeXt (Wang et al., 2025a)	23,519	14	5.07 / 8.00	589	6.00 / 7.92
Mechanical Structures	Fusion 360 Gallery (Joinable) (Willis et al., 2022)	8,251	55	6.85 / 8.91	80	6.20 / 8.41
Daily Objects / Furniture	Artiverse (Iliash et al., 2026)	5,402	42	6.50 / 8.57	908	5.90 / 7.43
Product-level CAD (open-source hardware)	Robotics & vehicles (Cominelli, 2025, etc.)	7	7	8.29 / 9.29	7	8.29 / 9.29
	Consumer devices (Ploopy, 2025, etc.)	10	10	6.40 / 7.80	10	6.40 / 7.80
	Lab & medical instruments (Hisakawa, 2024, etc.)	7	7	5.71 / 8.43	7	5.71 / 8.43
	Fabrication machines & tools (VoronDesign, 2025, etc.)	10	10	6.30 / 9.60	10	6.30 / 9.60
Total in LMBuild		367,991	200	7.62 / 9.25	2,549	7.03 / 8.83

its previous interaction trajectory to revise the current structure. We evaluate budgets of $K \in 1, 2, 3$ rounds to measure how effectively agents use this feedback for iterative refinement.

Prompting with Fine-Grained Descriptions. Beyond the baseline category label and reference image, LMBuild evaluates whether richer task specifications improve structure generation. We augment the prompt with fine-grained descriptions of salient attributes and functional components, providing explicit guidance about what parts should be present and how they should behave or be operated. Comparison with the baseline reveals whether failures stem from insufficient specification of user intent or from limitations in translating detailed requirements into buildable structures.

2.2 REPURPOSING EXISTING DATASETS FOR LMBUILD

Evaluating LLM agents for structure generation is substantially more expensive than standard conversational evaluation. Even a simple object with fewer than 20 parts can require over 50 interactions in one construction round, with costs increasing further for complex structures and multi-round revision. Exhaustive evaluation over large 3D datasets is therefore impractical, potentially requiring millions turns of LLM generations. From around 368K candidate 3D objects, we therefore construct a full benchmark of 2,549 objects and a core set of 200 objects through stratified selection from complementary sources, as summarized in Table 1. Rather than matching the frequency distribution of any individual source dataset, our selection is **coverage-oriented**: we prioritize objects that span diverse construction regimes and capabilities, have verifiable real-world references, and remain tractable for comprehensive evaluation. Due to budget constraints, all experiments in this work are conducted on the core set, while the full benchmark will be made publicly available.

Complementary Geometry Sources. No single existing dataset covers the full range of structures and construction regimes targeted by LMBuild. As summarized in Table 1, we therefore combine several complementary sources. **BrickNet** (Kulits & Schmid, 2026) and **BrickComposer** (Liu et al., 2026a) provide LEGO-style structures assembled from reusable part inventories with standardized connections. **PartNeXt** (Wang et al., 2025a) contributes fine-grained part-based objects and static structures, while the Fusion 360 Gallery assembly dataset from **JoinABLE** (Willis et al., 2022) provides engineering CAD assemblies with explicit part and joint structure. **Artiverse** (Iliash et al., 2026) contributes physically grounded articulated everyday objects with functional parts and kinematic annotations. Finally, we also collect product-level CAD from 27 open-source hardware projects, with each project publishes a complete, buildable product as a single assembly file. They span four domains: (1) **Robotics and Vehicles**, including the ASTRA spinal quadruped (Cominelli, 2025), the Hubert robotic arm (Gildea, 2025), the BiDexHand dexterous hand (Weng, 2025), the OpenROV underwater vehicle (OpenROV, 2020), the Ardumower lawn mower (Ardumower, 2022), the OpenAMR delivery robot (openAMRobot, 2026) and the UPSat CubeSat (Libre Space Foundation, 2017); **Consumer Devices**, including Ploopy input and audio devices (Ploopy, 2025), the Framework desktop and keyboard (Framework Computer Inc.,



Figure 2: **Left:** Generation examples produced by the LMBuild framework. Colors are used only to visually distinguish decomposed components. **Right:** The generated structures are evaluated using our comprehensive set of evaluation metrics. We briefly illustrate the reference source and general evaluation procedure for each metric; full evaluation details are provided in Appendix A.4.

2026a;b), a DIY VR headset (dmcke5, 2025), a force-feedback racing pedal (ChrGri, 2026), the OLSK coffee grinder (Ingrassia, 2023), the Shuffle-o-matic card shuffler (DDeGonge, 2022) and a Curta mechanical calculator (Wu, 2026); **Lab and Medical Instruments**, including the OpenRAMAN spectrometer (OpenRAMAN, 2023), a biaxial stretcher for live microscopy (Shiwarski et al., 2019), the ShearView rheometer (somexlab, 2026), the NinjaPCR thermal cycler (Hisakawa, 2024), the PolyVent ventilator (Bechard et al., 2024), a multi-channel peristaltic pump (Buchhorn et al., 2025) and the OpenAstroTracker star tracker (OpenAstroTech, 2025); and **Fabrication Machines and Tools**, including the midTbot pen plotter (Dring, 2020), the Precious Plastic recycling machines (Precious Plastic, 2025), the OpenSlider camera slider (Adamslab, 2021), the OLSK 3D scanner and vinyl cutter (Open Lab Starter Kit, 2025a;b), the Voron 2.4 3D printer (VoronDesign, 2025), an automatic wire stripper (Projects with Red, 2023) and the Open Source Ecology CEB press (Open Source Ecology, 2026). Together, these sources span both assembly from reusable part inventories and settings that require structure-specific part creation, and cover a broad range of real-world structures, from toys and furniture to complete machines.

Data Selection Strategy. Within the combined source pool, we stratify the benchmark data selection along four dimensions. First, we ensure structural and kinematic diversity across six broad families, spanning static load-bearing structures, hinged mechanisms, sliding mechanisms, rotating systems, serial kinematic chains, and multi-joint articulated mechanisms. Second, we prioritize grounded and recognizable objects whose expected components, physical properties, and functional behaviors can be verified from external Wikipedia data; each object category appears at most once in our core set. Third, we maintain difficulty coverage from simple assemblies to complex articulated systems by diversifying the number of parts, and require at least four parts per object. Fourth, we prioritize product-level CAD. These assemblies are the closest proxy for the end goal of LMBuild, building objects that work as real products, but they are orders of magnitude scarcer than part-annotated datasets. We therefore retain every product assembly that passes our screening (a complete single-file assembly, an open license, and a recognizable object category). The concrete data selection process and example data are provided in Appendix A.3.

2.3 HIERARCHICAL EVALUATION FRAMEWORK FOR LMBUILD

Traditional evaluations of generated structures primarily textbfasize geometric quality or semantic alignment. In contrast, LMBuild evaluates whether a generated structure is buildable and functional through a hierarchical framework with four complementary dimensions. **Structural Soundness (S)** assesses whether the generated geometry is physically plausible and structurally stable, avoiding impossible configurations or severe geometric inconsistencies. **Functional Affordance (A)** evaluates whether the object exhibits the appropriate overall shape, contains the functional parts required by the target category, and specifies joints and kinematics consistent with the intended motion and functionality. **Design Quality (D)** measures whether the generated structure is appropriately decomposed into meaningful components and whether its overall form and appearance align with the reference structure and human design preferences. **Physical Realization (R)** evaluates whether the structure can be instantiated in the physical world, including whether it admits a feasible robotic assembly sequence, provides sufficient material information for fabrication, and remains operable with respect to its intended functionality. As shown in Figure 2 (Right), these four dimensions are further decomposed into twelve concrete sub-metrics, together with their general corresponding evaluation strategies. To aid presentation, all scores are normalized to a common 0–100 scale. To avoid overloading the main text with algorithmic details, we provide the complete evaluation criteria and implementation protocols in Appendix A.4, with illustrative examples in Table 4.

3 EXPERIMENTS

3.1 SETUP

Evaluated Systems. We evaluate 30 systems spanning proprietary APIs, open-weight foundation models, and domain-specific generators, covering diverse model families, generations, scales, and architectures. Proprietary systems include GPT-6 Astra (OpenAI, 2026b), GPT-5.6 Sol (OpenAI, 2026a), Claude Opus 5 (Anthropic, 2026b), Claude Fable 5.1 (Anthropic, 2026a), Claude Sonnet 5 (Anthropic, 2026c), and Claude Haiku 4.5 (Anthropic, 2025). Open-weight models include six Qwen variants across Qwen3, Qwen3-VL, and Qwen3.5 (Yang et al., 2025; Bai et al., 2025; Qwen Team, 2026), spanning 4B–35B parameters and both dense and MoE architectures (e.g., 30B-A3B and 35B-A3B), together with GPT-OSS-120B (OpenAI, 2025), Gemma-4-31B-IT (Gemma Team et al., 2026), Ministral-3-8B (Mistral AI, 2025), InternVL3.5-8B/38B (Wang et al., 2025c), MiniCPM-V-4.5 (Yu et al., 2025), and ERNIE-4.5-VL-28B-A3B (ERNIE Team, Baidu, 2025). We further compare with domain-specific systems for brick assembly, part-aware generation, and simulation-ready assets, including BrickGPT (Pun et al., 2025a), LegoACE (Xu et al., 2025), PartCrafter (Lin et al., 2025), PartPacker (Tang et al., 2025a), Cube3D and CubePart (Foundation AI Team, Roblox, 2025; Zhu et al., 2026b), and PhysX-Anything (Cao et al., 2025b). Selected part-based generators are additionally augmented with Particulate for joint specification (Table 3).

3.2 MEASURING MODEL CAPABILITIES FOR FUNCTIONAL STRUCTURE GENERATION

Overall Observations. As shown in Table 2, state-of-the-art closed-source LLMs achieve the strongest overall performance across most metrics, although several open-source and domain-specific systems remain competitive and occasionally surpass them on individual geometric (e.g., Cube3D[†](P) on **D.2**) or realization metrics (e.g., Qwen3.5 models on **R.1**). Notably, frontier closed-source models, including GPT-6 Astra, GPT-5.6 Sol, Claude Opus 5, and Claude Fable 5.1, exhibit broadly similar performance across many evaluation dimensions, suggesting that leading model families are converging toward a comparable capability level for generating 3D structures. More broadly, overall performance is associated with stronger general reasoning capabilities, while the competitive performance of several domain-specific systems suggests that targeted training and architectural specialization can partially compensate for weaker general reasoning on structure generation. Note that although several systems achieve high performance along individual dimensions, strong performance on a single metric does not necessarily translate into a high-quality structure overall. Detailed interpretations of these results are provided in Appendix A.5.

Structural Soundness and Visual Alignment Are No Longer the Primary Bottlenecks Across Structural Soundness, most systems perform reliably in generating interconnected (**S.1**) and

Table 2: Evaluation results across 30 systems under Baseline Interaction Protocol (§ 2.1) on core set, assessed using our hierarchical evaluation metrics (§ 2.3). We use [†] (P) to denote systems integrated with Particulate to obtain joint information (Appendix A.2). Background colors encode performance on a common 0–100 scale, transitioning from blue (lower performance) through gray to orange (higher performance). Systems unable to produce the required output are assigned as N/A.

Models / Metrics	Soundness			Affordance			Design			Realization		
	S.1	S.2	S.3	A.1	A.2	A.3	D.1	D.2	D.3	R.1	R.2	R.3
State-of-the-art Closed-Source APIs												
GPT-6 Astra	83.0	95.9	86.7	63.0	45.3	41.7	76.0	71.2	74.9	60.9	80.4	19.2
GPT-5.6 Sol	85.7	95.6	85.0	48.9	38.8	38.0	77.6	70.7	66.1	65.8	87.5	19.1
Claude Opus 5	86.8	95.7	84.0	52.1	44.9	42.7	76.4	69.3	68.2	58.7	84.4	22.1
Claude Fable 5.1	82.4	97.4	87.5	55.3	44.2	42.5	75.2	69.6	70.5	69.0	76.1	20.8
Claude Sonnet 5	80.5	95.2	82.5	47.1	38.0	39.3	74.6	60.1	64.2	68.5	82.4	17.7
Claude Haiku 4.5	81.5	90.3	70.6	36.7	31.4	23.3	69.3	49.2	54.9	58.9	84.0	11.0
Open-Source (Multi-modal) Large Language Models												
Qwen3.5-27B	86.7	83.4	78.5	39.5	28.8	16.7	66.1	40.9	51.1	74.7	86.4	10.5
Qwen3.5-35B-A3B	74.5	79.3	62.5	31.8	27.2	13.2	60.1	33.5	47.5	47.3	78.6	8.9
GPT-OSS-120B	80.5	88.2	66.8	28.9	27.1	21.3	60.5	45.8	41.7	60.1	85.4	9.6
Gemma-4-31B-IT	57.7	62.2	59.2	30.3	19.9	10.1	47.7	34.9	40.9	36.0	66.1	7.9
Ministral-3-8B	77.0	75.7	44.5	25.1	25.1	6.1	55.9	41.0	43.5	33.0	81.3	5.3
InternVL3.5-38B	45.2	58.2	44.6	19.9	11.4	2.8	39.3	32.5	32.5	8.5	52.5	3.6
Qwen3-VL-30B-A3B	60.0	51.2	34.9	20.1	17.2	4.2	39.3	24.4	32.5	35.4	58.8	4.6
MiniCPM-V4.5	45.7	42.3	32.0	17.2	10.3	0.8	33.8	24.7	22.8	4.4	49.3	2.2
Qwen3-4B-Instruct	81.6	52.3	62.7	23.0	18.5	5.4	37.6	22.9	30.4	36.2	74.4	5.7
Qwen3-VL-32B	52.3	49.8	43.3	19.5	16.1	4.8	36.1	24.6	29.8	33.5	55.7	6.3
ERNIE-4.5-VL-28B-A3B	22.7	20.7	16.6	6.9	4.9	1.3	17.6	12.6	14.8	4.2	22.2	2.5
InternVL3.5-8B	38.9	37.3	41.1	14.4	7.1	0.7	27.5	25.2	25.7	0.5	51.8	2.0
Qwen3-VL-8B	32.0	24.4	27.3	9.4	8.3	2.9	17.3	13.2	14.7	13.6	36.0	3.9
State-of-the-art Domain-Specific Models												
BrickGPT	99.6	100.0	84.5	35.5	33.5	N/A	27.6	13.0	32.1	N/A	N/A	N/A
LegoACE	92.4	97.1	44.4	29.9	29.3	N/A	28.1	18.0	36.9	N/A	N/A	N/A
PartCrafter-15	90.9	78.6	14.3	45.6	18.8	N/A	63.1	44.1	65.3	N/A	N/A	N/A
PartCrafter-8	73.2	84.7	13.4	42.6	13.7	N/A	58.3	38.0	62.6	N/A	N/A	N/A
PartPacker	73.2	87.4	64.9	52.5	20.2	N/A	55.5	61.0	67.3	N/A	N/A	N/A
Cube3D+CubePart	92.7	77.7	37.7	24.7	32.7	N/A	54.0	63.6	51.8	N/A	N/A	13.4
PartCrafter15 [†] (P)	81.7	91.0	10.8	45.4	6.4	2.3	33.4	26.1	52.8	N/A	N/A	N/A
PartCrafter8 [†] (P)	72.1	95.1	13.5	43.2	6.6	2.3	36.4	33.3	59.6	N/A	N/A	N/A
PartPacker [†] (P)	73.4	95.5	64.8	52.3	9.1	3.6	42.8	56.5	65.6	N/A	N/A	N/A
Cube3D [†] (P)	69.1	96.5	73.9	40.3	8.9	3.8	43.1	82.2	57.7	N/A	N/A	N/A
PhysX-Anything	52.3	88.9	49.8	20.0	8.7	5.1	44.9	48.3	40.8	N/A	61.4	1.9

collision-free (S.2) structures, although performance declines somewhat on structural stability (S.3), particularly for domain-specific generators. Similarly, frontier closed-source models already perform strongly in producing reasonable decompositions (D.1), aesthetically appealing designs (D.2), and reference-aligned structures (D.3), suggesting that structural plausibility and visual quality are no longer their primary bottlenecks. Several open-source and domain-specific systems are also competitive on individual design metrics, but a noticeable gap remains overall, leaving room for further improvement in generating structures with better aesthetic quality, stronger structural stability, and more fine-grained visual alignment.

Functional Affordance and Physical Operability Remain the Largest Capability Gaps. Even the strongest systems remain imperfect in generating appropriate functional geometry (A.1) and complete functional parts (A.2), while joints and kinematics (A.3) are particularly challenging: most open-source models score below 25, and even the strongest systems only slightly exceed 50. These limitations carry directly into physical realization. Although models are comparatively more capable of specifying plausible assembly sequences (R.1) and selecting materials (R.2), simulation-based

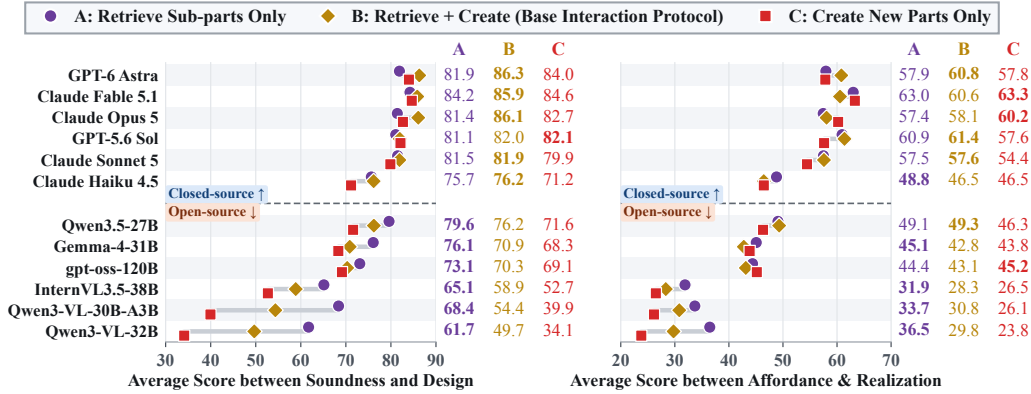


Figure 3: **Model Performance Change** across evaluation metrics under three part-access settings with 25 sampled LMBuild objects. Allowing part creation can hurt weaker models, even when catalog parts remain available, suggesting that the added design freedom increases decision complexity. In contrast, frontier models benefit substantially from creating structure-specific parts, revealing a stronger ability to exploit flexible construction mechanisms rather than relying only on retrieval.

operability (**R.3**) remains low across all systems, with the best scores reaching only the mid-30s. Together, these results suggest that generating a structurally and visually aligned artifact is substantially easier than producing the functional geometry, complete parts, joint configurations, and kinematics required for the artifact to be fabricated and operate correctly in the physical world.

3.3 STRONG MODELS CREATE, WHILE WEAKER MODELS RETRIEVE

We further evaluate LLM agents under three interaction protocols: our baseline setting with both retrieval and part creation, a retrieval-only setting, and a creation-only setting. As shown in Figure 3, model behavior differs substantially across capability levels. Less capable models degrade sharply when required to create all parts themselves. For example, Qwen3-VL-32B achieves an average score of 61.7 across Soundness and Design metrics under retrieval, but drops to 34.1 under creation-only. Even simply introducing part creation alongside retrieval can reduce performance for weaker models, suggesting that additional design freedom also introduces a substantially larger reasoning and decision-making burden. In contrast, frontier models exhibit the opposite trend. They more frequently choose to create their own components when both tools are available, and maintain comparable or even stronger performance when retrieval is removed entirely. For these models, retrieval-only can instead become the more restrictive setting, as the available component pool may constrain their ability to construct geometry tailored to the target structure. This reveals an intriguing capability transition: retrieval helps weaker models by reducing the design space, whereas creation allows stronger models to exploit their generative and structural reasoning capabilities. Part creation therefore appears not merely as an additional tool, but as a capability that increasingly becomes beneficial as the underlying model becomes stronger.

3.4 IMPROVING CAPABILITIES WITH ITERATIVE REVISION AND PROMPTING

As shown in Figure 4, we evaluate two common strategies for improving buildable and functional structure generation: (1) allowing models to inspect their outputs, receive error reports, and iteratively revise the structure; and (2) enriching the target description with explicit functional attributes and requirements to encourage deeper reasoning about the underlying structure.

Iterative Revision Provides Only Modest Gains. Allowing models to inspect and revise their structures over three rounds leads to steady but limited improvement, with overall gains of only about 2%. Because agents can already interact with and inspect the environment under the baseline protocol, additional revision rounds mainly provide more opportunities to retry rather than addressing the underlying reasoning limitations required for functional structure generation.

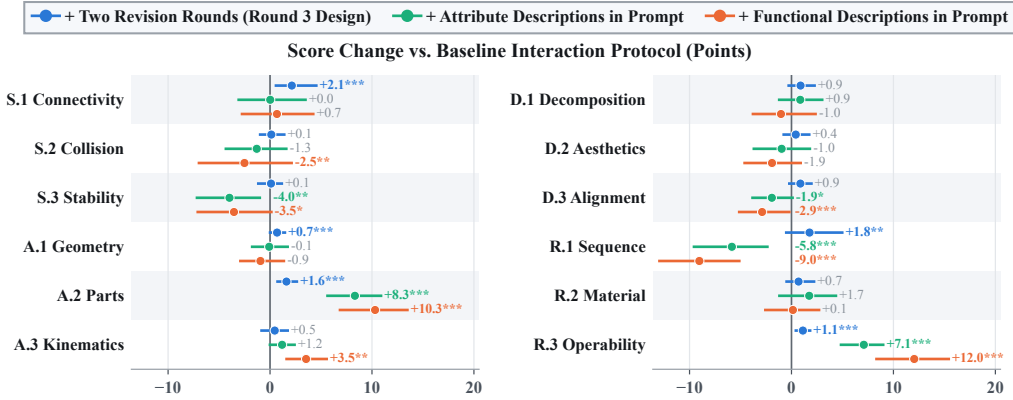


Figure 4: **Average performance change** across models from (1) additional revision rounds, (2) explicit attribute requirements, and (3) explicit functional requirements. Revision yields small but consistent gains, while explicit prompts substantially improve functional-part completeness, kinematics, and especially operability. **Bars:** 95% bootstrap confidence intervals over models; **Stars:** Wilcoxon signed-rank test over episodes (* $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$)

Explicit Functional Specifications Produce Substantial Improvements. In contrast, enriching a minimal prompt with explicit attributes and functional requirements grounded in Wikipedia produces substantially larger gains on the dimensions where current models struggle most. Functional-part completeness improves by 10.3 points and physical operability by 12.0 points ($p < 0.001$), with additional improvements in kinematic correctness. These results indicate that a substantial fraction of baseline failures can be reduced when functional requirements are made explicit. Current agents therefore appear considerably more reliable at translating specified functional constraints into structure than at inferring all such requirements from a minimal object label alone.

Implication. Together, these results suggest that improving task specification can be more effective than increasing the revision rounds. A promising direction is therefore to develop agents that can more reliably infer, retrieve, or elicit the functional requirements of an object before construction, reducing the need for users to manually provide detailed part-level and kinematic specifications.

3.5 LMBUILD IDENTIFIES FAILURE PATTERNS OF EVALUATED LLMs

Open Models Exhibit a Part-Creation Gap. Across Part Creation calls, closed-source models succeed on 95.7% of attempts, compared with only 51.7% for open-source models, with Qwen3.5-27B (89.4%) as the only close exception. Most failures fall into two recurring categories. First, models violate CAD-kernel conventions, such as listing polygon vertices clockwise. Second, models generate invalid program structures by nesting nodes instead of referencing named nodes, using numeric node IDs, or referring to undefined nodes. These errors suggest that much of the creation gap stems from weak adherence to tool-specific conventions and program structure, rather than purely from deficiencies in geometric or spatial reasoning.

Open Models Struggle with Scene-State Tracking. Beyond part creation, open-source models frequently lose track of the evolving scene state. They refer to parts that were never placed at a rate of 7.2 errors per 100 calls, compared with 0.3 for closed-source models, they often attempt to attach, join, or assign properties to parts whose creation had already failed. They also redundantly add existing entities, including repeatedly placing parts instead of moving them and re-adding existing joints, at 3.2 errors per 100 calls versus 0.5 for closed-source models. Additional interface errors include invalid material names, malformed arguments, exhausted inspection calls, and even invoking part names as tools. Together, these failures reveal a broader limitation in maintaining and updating an accurate internal representation of the environment throughout long-horizon construction.

4 RELATED WORK

3D Generation with Language Models. Language models increasingly support natural-language 3D creation by generating procedural scripts and scene blueprints (Hu et al., 2024; Sun et al., 2025; Tang et al., 2025b) or parametric CAD operation sequences (Khan et al., 2024; Li et al., 2025). Visual feedback further enables training and iterative refinement of generated geometry (Huang et al., 2024; Wang et al., 2025b; Alrashedy et al., 2025). These methods connect language understanding with programmatic 3D modeling, but generally focus on producing editable or visually correct geometry rather than structures that can be physically assembled and function as intended.

Part-Aware Generation and Physical Assembly. Recent work represents objects as structured collections of parts (Lin et al., 2025; Tang et al., 2025a), predicts joints and articulation (Willis et al., 2022; Li et al., 2026), and incorporates physical attributes such as scale, materials, and motion (Le et al., 2025; Cao et al., 2025a;b). Complementary approaches target stable brick construction, assembly programs, component placement, and robotic execution (Pun et al., 2025b; Kulits & Schmid, 2026; Liu et al., 2026a; Goldberg et al., 2025). Together, these works motivate evaluating part structure, physical behavior, and whether the completed object can realize its intended function.

Evaluation of 3D Generation. Existing 3D evaluation primarily measures visual quality and text alignment (He et al., 2023), often extended with multimodal judgments of appearance, surface quality, and human preference (Wu et al., 2024; Maiti et al., 2025), or finer-grained geometric and semantic consistency checks (Duggal et al., 2025). For assembled structures, prior work additionally evaluates stability, component selection, placement, and construction success (Pun et al., 2025b; Liu et al., 2026a; Goldberg et al., 2025). LMBuild complements these efforts by jointly evaluating soundness, affordance, design, and physical realization across heterogeneous objects using geometric checks, grounded component assessments, simulation-based motion tests, and VLM judgments.

5 CONCLUSION

In this work, we introduce LMBuild, a benchmark for evaluating LLM-based agents on the generation of structured 3D assemblies across four dimensions: structural soundness, functional affordance, design quality, and physical realization. On the LMBuild-Core benchmark, the evaluated systems perform substantially better on structural and visual criteria than on functional-part completeness, kinematic correctness, and simulation-based operability. These results highlight persistent challenges in translating object specifications into coherent components, joints, and assembly plans, as well as in creating task-specific parts when suitable components are unavailable. We hope LMBuild provides a reproducible foundation for studying these capabilities and for developing agents that generate increasingly functional and physically plausible structures. Bridging these benchmark results to real-world fabrication and operation will require further validation involving manufacturing constraints, physical assembly, and functional testing.

AI USE STATEMENT

We used large language models (LLMs) as writing assistants to improve the clarity, grammar, and presentation of the manuscript. We also used AI-based coding agents to assist with evaluation-environment construction, code refactoring, debugging, code cleaning, and code publishing. All research decisions, experimental designs, implementations, analyses, and reported results were reviewed and verified by the authors. AI tools were not used to determine the scientific conclusions of this work.

ETHICS STATEMENT

This work aims to support more transparent, accessible, and reproducible evaluation of LLM agents for real-world structure generation. By developing an open evaluation framework, we hope to lower the barriers to systematically assessing model capabilities beyond purely textual benchmarks. In particular, our benchmark emphasizes whether generated structures are not only plausible in appearance but also buildable and functionally valid under real-world constraints. We believe that open

evaluation infrastructure can facilitate more rigorous comparison across models, encourage reproducible research, and contribute to the broader open-source AI ecosystem. At the same time, benchmarks can only capture a subset of the requirements involved in deploying AI-generated designs in real-world settings. Performance on LMBuild should therefore not be interpreted as sufficient evidence that a model is ready for safety-critical engineering, construction, or other physical-world applications without appropriate domain-specific validation and human oversight.

REPRODUCIBILITY STATEMENT

We provide representative data samples and evaluation materials in the supplementary materials to facilitate inspection and reproduction of our methodology. We will open-source the complete codebase, data, evaluation environment, evaluation scripts, model trajectories from the reported experiments, and all the necessary artifacts required to reproduce our results. We aim to make the full evaluation pipeline transparent and executable so that future work can independently verify our findings, evaluate additional models, and extend LMBuild for constructing more powerful agents.

REFERENCES

- Adamslab. OpenSlider: A motorised three-axis camera slider. <https://github.com/Adamslab/OpenSlider>, 2021. GPL-3.0. Accessed 2026-09.
- Kamel Alrashedy, Pradyumna Tambwekar, Zulfiqar Haider Zaidi, Megan Langwasser, Wei Xu, and Matthew Gombolay. Generating cad code with vision-language models for 3d designs. In *International Conference on Learning Representations*, 2025.
- Anthropic. Introducing claude haiku 4.5. <https://www.anthropic.com/news/claude-haiku-4-5>, October 2025. Accessed September 2026.
- Anthropic. Claude fable 5.1. <https://platform.claude.com/docs/en/models/fable-5-1/overview>, 2026a. Accessed September 2026.
- Anthropic. Introducing claude opus 5. <https://www.anthropic.com/news/claude-opus-5>, July 2026b. Accessed September 2026.
- Anthropic. Introducing claude sonnet 5. <https://www.anthropic.com/news/claude-sonnet-5>, June 2026c. Accessed September 2026.
- Ardumower. Ardumower: Open-source robotic lawn mower (chassis 0.99). <https://github.com/Ardumower/ardumower>, 2022. GPL-3.0. Accessed 2026-09.
- Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, Mei Li, Kaixin Li, Zicheng Lin, Junyang Lin, Xuejing Liu, Jiawei Liu, Chenglong Liu, Yang Liu, Dayiheng Liu, Shixuan Liu, Dunjie Lu, Ruilin Luo, Chenxu Lv, Rui Men, Lingchen Meng, Xuancheng Ren, Xingzhang Ren, Sibao Song, Yuchong Sun, Jun Tang, Jianhong Tu, Jianqiang Wan, Peng Wang, Pengfei Wang, Qiuyue Wang, Yuxuan Wang, Tianbao Xie, Yiheng Xu, Haiyang Xu, Jin Xu, Zhibo Yang, Mingkun Yang, Jianxin Yang, An Yang, Bowen Yu, Fei Zhang, Hang Zhang, Xi Zhang, Bo Zheng, Humen Zhong, Jingren Zhou, Fan Zhou, Jing Zhou, Yuanzhi Zhu, and Ke Zhu. Qwen3-VL Technical Report. [arXiv preprint arXiv:2511.21631](https://arxiv.org/abs/2511.21631), 2025.
- Nathaniel Bechard, Robert Read, Antal Zuerdwijk, and Victor Sutorin. PolyVent V2.0 project files, 2024. URL <https://zenodo.org/records/14425536>.
- Michael Buchhorn, Gun Deniz Akkoc, and Dominik Dworschak. An open-source peristaltic pump with multiple independent channels for laboratory automation, 2025. URL <https://zenodo.org/records/16118678>.
- Ziang Cao, Zhaoxi Chen, Liang Pan, and Ziwei Liu. Physx-3d: Physical-grounded 3d asset generation. In D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen (eds.), *Advances in Neural Information Processing*

- Systems, volume 38, pp. 93771–93784. Curran Associates, Inc., 2025a. doi: 10.52202/085713-3136. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/86beeac1799c083e4597d4aa17385a25-Paper-Conference.pdf.
- Ziang Cao, Fangzhou Hong, Zhaoxi Chen, Liang Pan, and Ziwei Liu. PhysX-Anything: Simulation-ready physical 3d assets from single image. *arXiv preprint arXiv:2511.13648*, 2025b.
- ChrGri. DIY sim-racing force-feedback pedal: Mechanical design. <https://github.com/ChrGri/DIY-Sim-Racing-FFB-Pedal-Mechanical-Design>, 2026. CC BY-NC-SA 4.0. Accessed 2026-09.
- Federico Cominelli. ASTRA: An open-source spinal quadruped robot. <https://github.com/fedecomio4/opensource-spinal-quadruped>, 2025. MIT License. Accessed 2026-09.
- DDeGonge. Shuffle-o-matic: A motorised playing-card shuffler. <https://github.com/DDeGonge/Shuffle-o-matic>, 2022. MIT License. Accessed 2026-09.
- dmcke5. DIY VR: Open-source virtual reality headset (V2.0). https://github.com/dmcke5/DIY_VR, 2025. CERN-OHL-S-2.0. Accessed 2026-09.
- Bart Dring. midTbot ESP32: An open-source h-bot pen plotter. https://github.com/bdring/midTbot_esp32, 2020. CC BY-SA 4.0. Accessed 2026-09.
- Shivam Duggal, Yushi Hu, Oscar Michel, Aniruddha Kembhavi, William T Freeman, Noah A Smith, Ranjay Krishna, Antonio Torralba, Ali Farhadi, and Wei-Chiu Ma. Eval3d: Interpretable and fine-grained evaluation for 3d generation. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 13326–13336. IEEE, 2025.
- ERNIE Team, Baidu. ERNIE 4.5 Technical Report. https://ernie.baidu.com/blog/publication/ERNIE_Technical_Report.pdf, 2025.
- Foundation AI Team, Roblox. Cube: A roblox view of 3d intelligence. *arXiv preprint arXiv:2503.15475*, 2025.
- Framework Computer Inc. Framework desktop: Open mechanical design files. <https://github.com/FrameworkComputer/Framework-Desktop>, 2026a. CC BY 4.0. Accessed 2026-09.
- Framework Computer Inc. Framework wireless touchpad keyboard: Open design files. <https://github.com/FrameworkComputer/Framework-Wireless-Touchpad-Keybaord>, 2026b. CC BY 4.0. Accessed 2026-09.
- Gemma Team, Sherif El Abd, Vaibhav Aggarwal, Robin Algayres, Alek Andreev, Olivier Bachem, Ian Ballantyne, Cormac Brick, Victor Cărbune, Michelle Casbon, Mayank Chaturvedi, et al. Gemma 4 Technical Report. *arXiv preprint arXiv:2607.02770*, 2026.
- Aidan Gildea. Hubert: An open-source 3-DoF robotic arm. <https://github.com/Aidan-Gildea/robot-arm-3dof>, 2025. MIT License. Accessed 2026-09.
- Andrew Goldberg, Kavish Kondap, Tianshuang Qiu, Zehan Ma, Letian Fu, Justin Kerr, Huang Huang, Kaiyuan Chen, Kuan Fang, and Ken Goldberg. Blox-net: Generative design-for-robot-assembly using vlm supervision, physics simulation, and a robot with reset. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 15493–15500. IEEE, 2025. doi: 10.1109/ICRA55743.2025.11127489.
- Yuze He, Yushi Bai, Matthieu Lin, Wang Zhao, Yubin Hu, Jenny Sheng, Ran Yi, Juanzi Li, and Yong-Jin Liu. τ^3 bench: Benchmarking current progress in text-to-3d generation. *arXiv preprint arXiv:2310.02977*, 2023.
- Shingo Hisakawa. NinjaPCR: Open-source PCR thermal cycler. <https://github.com/hisashin/NinjaPCR>, 2024. GPL-3.0. Accessed 2026-09.

- Ziniu Hu, Ahmet Iscen, Aashi Jain, Thomas Kipf, Yisong Yue, David A Ross, Cordelia Schmid, and Alireza Fathi. Scenecraft: An llm agent for synthesizing 3d scenes as blender code. In ICML, pp. 19252–19282, 2024.
- Ian Huang, Guandao Yang, and Leonidas Guibas. Blenderalchemy: Editing 3d graphics with vision-language models. In European Conference on Computer Vision, pp. 297–314. Springer, 2024.
- Denys Iliash, Jiayi Liu, Egor Fokin, Qirui Wu, Ali Mahdavi-Amiri, Manolis Savva, and Angel X. Chang. Artiverse: A diverse and physically grounded dataset for articulated objects. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 8932–8942, 2026.
- Daniele Ingrassia. OLSK coffee grinder V2. <https://github.com/satshas/Coffee-Grinder>, 2023. Open Lab Starter Kit. CERN-OHL-W-2.0. Accessed 2026-09.
- Mohammad S Khan, Sankalp Sinha, Talha U Sheikh, Didier Stricker, Sk A Ali, and Muhammad Z Afzal. Text2cad: Generating sequential cad designs from beginner-to-expert level text prompts. Advances in Neural Information Processing Systems, 37:7552–7579, 2024.
- Peter Kulits and Cordelia Schmid. Bricknet: Graph-backed generative brick assembly. arXiv preprint arXiv:2604.22984, 2026.
- Alexander Htet Kyaw, Miana Smith, Se Hwan Jeon, and Neil Gershenfeld. Speech to reality: On-demand production using natural language, 3d generative ai, and discrete robotic assembly. In Proceedings of the ACM Symposium on Computational Fabrication (SCF '25), pp. 16:1–16:12. Association for Computing Machinery, 2025. doi: 10.1145/3745778.3766670.
- Long Le, Jason Xie, William Liang, Hung-Ju Wang, Yue Yang, Yecheng Jason Ma, Kyle Vedder, Arjun Krishna, Dinesh Jayaraman, and Eric Eaton. Articulate-anything: Automatic modeling of articulated objects via a vision-language foundation model. In International Conference on Learning Representations, 2025.
- Jiahao Li, Weijian Ma, Xueyang Li, Yunzhong Lou, Guichun Zhou, and Xiangdong Zhou. CAD-Llama: Leveraging large language models for computer-aided design parametric 3d model generation. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 18563–18573, June 2025. doi: 10.1109/CVPR52734.2025.01730.
- Ruining Li, Yuxin Yao, Chuanxia Zheng, Christian Rupprecht, Joan Lasenby, Shangzhe Wu, and Andrea Vedaldi. Particulate: Feed-forward 3d object articulation. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 27708–27718, 2026.
- Libre Space Foundation. UPSat: Open-source CubeSat structural subsystem. <https://gitlab.com/librespacefoundation/upsat/upsat-structural>, 2017. CERN-OHL v1.2. Accessed 2026-09.
- Chen-Hsuan Lin, Jun Gao, Luming Tang, Towaki Takikawa, Xiaohui Zeng, Xun Huang, Karsten Kreis, Sanja Fidler, Ming-Yu Liu, and Tsung-Yi Lin. Magic3D: High-resolution text-to-3d content creation. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 300–309, 2023. doi: 10.1109/CVPR52729.2023.00037.
- Yuchen Lin, Chenguo Lin, Panwang Pan, Honglei Yan, Yiqiang Feng, Yadong Mu, and Kate-rina Fragkiadaki. PartCrafter: Structured 3d mesh generation via compositional latent diffusion transformers. In Advances in Neural Information Processing Systems, volume 38, 2025. doi: 10.52202/085713-1189.
- Jiateng Liu, Bingxuan Li, Zhenhailong Wang, Rushi Wang, Kaiwen Hong, Cheng Qian, Jiayu Liu, Denghui Zhang, Katherine Driggs-Campbell, Manling Li, and Heng Ji. Brick-composer: Using mllms for assembly with diverse bricks. arXiv preprint arXiv:2606.05445, 2026a.

- Jiateng Liu, Zhenhailong Wang, Xiaojiang Huang, Yingjie Li, Xiang Li, Chenlei Guo, Xing Fan, Ruhi Sarikaya, and Heng Ji. Analyzing and internalizing complex policy documents for LLM agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 16814–16851, San Diego, California, United States, 2026b. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.767. URL <https://aclanthology.org/2026.acl-long.767/>.
- Jiateng Liu, Zhenhailong Wang, Rushi Wang, Bingxuan Li, Jeonghwan Kim, Aditi Tiwari, Pengfei Yu, Denghui Zhang, and Heng Ji. OSExpert: Computer-use agents learning professional skills via exploration. *arXiv preprint arXiv:2603.07978*, 2026c. URL <https://arxiv.org/abs/2603.07978>.
- Shalini Maiti, Lourdes Agapito, and Filippos Kokkinos. Gen3deval: Using vllms for automatic evaluation of generated 3d objects. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 18552–18562. IEEE, 2025.
- Mistral AI. Introducing mistral 3. <https://mistral.ai/news/mistral-3/>, December 2025. Includes Ministral 3 model family.
- Open Lab Starter Kit. OLSK 3D scanner V2. <https://github.com/Open-Lab-Starter-Kit/OLSK-3D-Scanner>, 2025a. CERN-OHL-W-2.0. Accessed 2026-09.
- Open Lab Starter Kit. OLSK vinyl cutter V3. <https://github.com/Open-Lab-Starter-Kit/OLSK-Vinyl-Cutter>, 2025b. CERN-OHL-W-2.0. Accessed 2026-09.
- Open Source Ecology. CEB press 6: Open-source compressed earth block press. https://wiki.opensourceecology.org/wiki/CEB_Press/Current_Downloads, 2026. CC BY-SA 4.0. Accessed 2026-09.
- OpenAI. gpt-oss-120b & gpt-oss-20b Model Card, 2025. URL <https://arxiv.org/abs/2508.10925>.
- OpenAI. GPT-5.6: Frontier intelligence that scales with your ambition. <https://openai.com/index/gpt-5-6/>, 2026a. Accessed September 2026.
- OpenAI. GPT-6 Astra: A new generation of intelligence. <https://openai.com/index/gpt-6-astra/>, 2026b. Accessed September 2026.
- openAMRobot. OpenAMR: Open-source autonomous mobile robot. <https://github.com/openAMRobot/openamr>, 2026. MIT License. Accessed 2026-09.
- OpenAstroTech. OpenAstroTracker: Open-source equatorial star-tracking mount. <https://github.com/OpenAstroTech/OpenAstroTracker>, 2025. CC BY-NC 4.0. Accessed 2026-09.
- OpenRAMAN. OpenRAMAN performance edition: Open-source raman spectrometer. <https://github.com/samyk/openraman>, 2023. CERN-OHL-W-2.0. Accessed 2026-09.
- OpenROV. OpenROV v2.8: Open-source underwater remotely operated vehicle hardware. <https://github.com/OpenROV/openrov-hardware>, 2020. CC BY-NC-SA 4.0. Accessed 2026-09.
- Ploopy. Ploopy open-source hardware: Mouse, classic 2 trackball and headphones. <https://github.com/ploopyco>, 2025. CERN-OHL. Accessed 2026-09.
- Ben Poole, Ajay Jain, Jonathan T. Barron, and Ben Mildenhall. DreamFusion: Text-to-3d using 2d diffusion. In *International Conference on Learning Representations (ICLR)*, 2023.
- Precious Plastic. Precious plastic kit: Shredder V2.0, injection V2.0 and sheetpress V1. <https://github.com/ONEARMY/precious-plastic-kit>, 2025. CC BY-SA 4.0. Accessed 2026-09.

- Projects with Red. Automatic wire stripper and cutter. <https://github.com/ProjectsWithRed/auto-wire-stripper-cutter>, 2023. CC BY-NC 4.0. Accessed 2026-09.
- Ava Pun, Kangle Deng, Ruixuan Liu, Deva Ramanan, Changliu Liu, and Jun-Yan Zhu. Generating physically stable and buildable brick structures from text. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp. 14798–14809, October 2025a.
- Ava Pun, Kangle Deng, Ruixuan Liu, Deva Ramanan, Changliu Liu, and Jun-Yan Zhu. Generating physically stable and buildable brick structures from text. In 2025 IEEE/CVF International Conference on Computer Vision (ICCV), pp. 14798–14809. IEEE, 2025b.
- Cheng Qian, Hyeonjeong Ha, Jiayu Liu, Jeonghwan Kim, Emre Can Acikgoz, Bingxuan Li, Kunlun Zhu, Jiateng Liu, Aditi Tiwari, Zhenhailong Wang, Xiusi Chen, Mahdi Namazifar, and Heng Ji. Advancing creative physical intelligence in large multimodal models. arXiv preprint arXiv:2605.26396, 2026a. URL <https://arxiv.org/abs/2605.26396>.
- Cheng Qian, Hyeonjeong Ha, Jiayu Liu, Jeonghwan Kim, Jiateng Liu, Bingxuan Li, Aditi Tiwari, Dwip Dalal, Zhenhailong Wang, Xiusi Chen, Mahdi Namazifar, Yunzhu Li, and Heng Ji. CreativityBench: Evaluating agent creative reasoning via affordance-based tool repurposing. arXiv preprint arXiv:2605.02910, 2026b. URL <https://arxiv.org/abs/2605.02910>.
- Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026. URL <https://qwen.ai/blog?id=qwen3.5>.
- Daniel Shiwerski, Joshua Tashman, and Adam Feinberg. 3D printed biaxial stretcher compatible with live fluorescence microscopy, 2019. URL <https://zenodo.org/records/3483849>.
- somexlab. ShearView: A compact linear shear rheometer for rheo-microscopy. <https://github.com/somexlab/shearview>, 2026. MIT License. Accessed 2026-09.
- Chunyi Sun, Junlin Han, Weijian Deng, Xinlong Wang, Zishan Qin, and Stephen Gould. 3D-GPT: Procedural 3d modeling with large language models. In 2025 International Conference on 3D Vision (3DV), pp. 1253–1263, March 2025. doi: 10.1109/3DV66043.2025.00119.
- Jiaxiang Tang, Ruijie Lu, Zhaoshuo Li, Zekun Hao, Xuan Li, Fangyin Wei, Shuran Song, Gang Zeng, Ming-Yu Liu, and Tsung-Yi Lin. Efficient part-level 3d object generation via dual volume packing. In Advances in Neural Information Processing Systems, volume 38, 2025a. doi: 10.52202/085713-0914.
- Song Tang, Kaiyong Zhao, Lei Wang, Yuliang Li, Xuebo Liu, Junyi Zou, Qiang Wang, and Xiaowen Chu. UnrealLLM: Towards highly controllable and interactable 3d scene generation by llm-powered procedural content generation. In Findings of the Association for Computational Linguistics: ACL 2025, pp. 19417–19435. Association for Computational Linguistics, 2025b. doi: 10.18653/v1/2025.findings-acl.994.
- VoronDesign. Voron 2.4: An open-source CoreXY 3D printer (r2). <https://github.com/VoronDesign/Voron-2>, 2025. GPL-3.0. Accessed 2026-09.
- Penghao Wang, Yiyang He, Xin Lv, Yukai Zhou, Lan Xu, Jingyi Yu, and Jiayuan Gu. Partnext: A next-generation dataset for fine-grained and hierarchical 3d part understanding. In Advances in Neural Information Processing Systems, 2025a. Datasets and Benchmarks Track.
- Ruiyu Wang, Yu Yuan, Shizhao Sun, and Jiang Bian. Text-to-cad generation through infusing visual feedback in large language models. arXiv preprint arXiv:2501.19054, 2025b.
- Weiyun Wang, Zhangwei Gao, Lixin Gu, Hengjun Pu, Long Cui, Xingguang Wei, Zhaoyang Liu, Linglin Jing, Shenglong Ye, Jie Shao, et al. InternVL3.5: Advancing open-source multimodal models in versatility, reasoning, and efficiency. arXiv preprint arXiv:2508.18265, 2025c.

- Xingyao Wang, Zihan Wang, Jiateng Liu, Yangyi Chen, Lifan Yuan, Hao Peng, and Heng Ji. MINT: Evaluating LLMs in multi-turn interaction with tools and language feedback. In International Conference on Learning Representations, 2024. URL <https://openreview.net/forum?id=jp3gWrMuIZ>.
- Zhengyi Wang, Cheng Lu, Yikai Wang, Fan Bao, Chongxuan Li, Hang Su, and Jun Zhu. Prolific-Dreamer: High-fidelity and diverse text-to-3d generation with variational score distillation. In Advances in Neural Information Processing Systems, volume 36, 2023.
- Zhenhailong Wang, Jiateng Liu, Amin Fazel, Ritesh Sarkhel, Xing Fan, Xiang Li, Chenlei Guo, Heng Ji, and Ruhi Sarikaya. Multimodal policy internalization for conversational agents. In International Conference on Learning Representations, 2026. URL <https://arxiv.org/abs/2510.09474>.
- Zhengyang Kris Weng. BiDexHand: Design and evaluation of an open-source 16-DoF biomimetic dexterous hand, 2025. URL <https://arxiv.org/abs/2504.14712>.
- Karl DD Willis, Pradeep Kumar Jayaraman, Hang Chu, Yunsheng Tian, Yifei Li, Daniele Grandi, Aditya Sanghi, Linh Tran, Joseph G Lambourne, Armando Solar-Lezama, et al. Joinable: Learning bottom-up assembly of parametric cad joints. In 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 15828–15839. IEEE, 2022.
- Marcus Wu. Curta type I at 3x scale: Open CAD model of the curta mechanical calculator. <https://github.com/marcuswu/Curta-Type-I-3x>, 2026. CC BY-NC-SA 4.0. Accessed 2026-09.
- Tong Wu, Guandao Yang, Zhibing Li, Kai Zhang, Ziwei Liu, Leonidas Guibas, Dahua Lin, and Gordon Wetzstein. Gpt-4v (ision) is a human-aligned evaluator for text-to-3d generation. In 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 22227–22238. IEEE, 2024.
- Hao Xu, Yuqing Zhang, Yiqian Wu, Xinyang Zheng, Yutao Liu, Xiangjun Tang, Yunhan Yang, Ding Liang, Yingtian Liu, Yuanchen Guo, Yanpei Cao, and Xiaogang Jin. LegoACE: Autoregressive construction engine for expressive lego® assemblies. In Proceedings of the SIGGRAPH Asia 2025 Conference Papers, pp. 40:1–40:11. ACM, 2025. doi: 10.1145/3757377.3763881.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 Technical Report. arXiv preprint arXiv:2505.09388, 2025.
- Ke Yang, Jiateng Liu, John Wu, Chaoqi Yang, Yi R. Fung, Sha Li, Zixuan Huang, Xu Cao, Xingyao Wang, Yiquan Wang, Heng Ji, and Chengxiang Zhai. If LLM is the wizard, then code is the wand: A survey on how code empowers large language models to serve as intelligent agents. arXiv preprint arXiv:2401.00812, 2024. URL <https://arxiv.org/abs/2401.00812>.
- Tianyu Yu, Zefan Wang, Chongyi Wang, Fuwei Huang, Wenshuo Ma, Zhihui He, Tianchi Cai, Weize Chen, Yuxiang Huang, Yuanqian Zhao, Bokai Xu, Junbo Cui, Yingjing Xu, Liqing Ruan, Luoyuan Zhang, Hanyu Liu, Jingkun Tang, Hongyuan Liu, Qining Guo, Wenhao Hu, Bingxiang He, Jie Zhou, Jie Cai, Ji Qi, Zonghao Guo, Chi Chen, Guoyang Zeng, Yuxuan Li, Ganqu Cui, Ning Ding, Xu Han, Yuan Yao, Zhiyuan Liu, and Maosong Sun. MiniCPM-V 4.5: Cooking efficient mllms via architecture, data, and training recipe. arXiv preprint arXiv:2509.18154, 2025.
- Yujie Zhang, Bingyang Cui, Qi Yang, Zhu Li, and Yiling Xu. Benchmarking and learning multi-dimensional quality evaluator for text-to-3d generation. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp. 18563–18574, October 2025. doi: 10.1109/ICCV51701.2025.01725.

Guangxiang Zhao, Qilong Shi, Xusen Xiao, Xiangzheng Zhang, Tong Yang, and Lin Sun. Thinking with reasoning skills: Fewer tokens, more accuracy. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)*, pp. 2295–2308, San Diego, California, USA, 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-industry.154. URL <https://aclanthology.org/2026.acl-industry.154/>.

Sibo Zhu, Shicheng Fan, Xinyue Wang, Wenyi Wu, Kun Zhou, and Biwei Huang. RSIAgent: Autonomous exploration for recursive self-improvement in new environments. *arXiv preprint arXiv:2609.15364*, 2026a. URL <https://arxiv.org/abs/2609.15364>.

Yiheng Zhu, Kangle Deng, Jean-Philippe Fauconnier, Inaki Navarro, Daiqing Li, Ava Pun, Yinan Zhang, Peiye Zhuang, Xiaoxia Sun, Maneesh Agrawala, Kiran Bhat, and Tinghui Zhou. CubePart: An open-vocabulary part-controllable 3d generator. In *ACM SIGGRAPH 2026 Conference Papers*, pp. 7:1–7:12, 2026b.

A APPENDIX

A.1 AGENT HARNESS FOR LMBUILD

Overview. LMBuild exposes a common interactive assembly environment to heterogeneous model families through a lightweight model-specific harness. General-purpose LLM (VLM) agents receive the same environment semantics, task specification, tool definitions, action budgets, and multi-round revision protocol. The harness changes only the mechanism by which observations are delivered and actions are returned: closed-source agents interact with the environment through shell commands, whereas locally served open-source models use an OpenAI-compatible chat interface with text-serialized tool calls. Domain-specific structure-generation systems do not support the common tool interface and are therefore evaluated using their native generation pipelines. This separation allows LMBuild to preserve a common task and environment while accommodating substantial differences in model APIs. In particular, the closed- and open-source agent settings expose equivalent task information, although image transport, tool-call serialization, and conversation-state management necessarily differ across interfaces.

Environment tiers. All general-purpose agents are evaluated in one of three environment specifications. We refer to them as Tier A, Tier B, and Tier C for simplicity. **Tier A** permits retrieval and placement of catalog parts only. **Tier B**, our primary baseline interaction protocol, provides the same catalog while additionally allowing the agent to create a new component when no suitable catalog part is available. **Tier C** removes the catalog entirely and requires the agent to create every component before placing it. Except for the corresponding catalog and part-creation instructions, the environment prompt and tool semantics are unchanged across tiers. The environment enforces a maximum of 160 tool calls per episode, including at most 6 calls to `check`, 4 calls to `render`, and 80 placed parts. Part creation is limited to 30 calls in Tier B and 60 calls in Tier C. Both `check` and `render` are explicitly described to the agent as metered operations so that they are used for verification rather than unrestricted search.

Multi-round interaction. An episode may contain multiple revision rounds. Calling `submit` ends the current round. When another revision round remains, the environment returns a structured diagnostic produced by `check` together with condition, isometric, front, and side-view images of the current assembly. The agent is asked to inspect this feedback, revise the same scene, and submit again. A final `submit` returns `episode_done=true`. The assembly state is persistent across rounds; therefore, revisions modify the existing object rather than restarting generation.

A.1.1 SHARED LMBUILD AGENT PROMPT

The following system prompt is shared by all general-purpose LLM/VLM agents. The placeholder `{PARTS.SOURCE}` is instantiated according to the tier, after which LMBuild appends the tier-specific part-creation instructions and the generated tool documentation.

Shared environment system prompt

You assemble a physical object out of 3D parts by calling tools. The result is judged as a

- ↪ real, buildable, working product: it must hold together, stand, look like what was
- ↪ asked for, move where the real object moves, be made of sensible materials, and be
- ↪ buildable in the order you give.

WORLD FRAME

- Units are metres. +Z is up. The ground is the plane $z = 0$. The FRONT of the object faces
- ↪ -Y (a person sitting on a chair faces -Y). Standing in front of the object (at -Y) and
- ↪ looking at it, +X is to your right.
- Real-world size matters. Build the object at its real size.

PARTS

```
{PARTS_SOURCE}
{TOOLS_DOCUMENTATION}
```

- Every part's local origin is the centre of its bounding box. `place_part` puts that centre
- ↪ at `'position'` and applies `'rotation'` = [rx, ry, rz] degrees about the world X, then Y,
- ↪ then Z axes. So an unrotated part placed at $z = \text{size}_z / 2$ rests exactly on the
- ↪ ground.
- `attach` moves a part so that one face centre of its bounding box (top, bottom, left,
- ↪ right, front, back, center) lands on a face centre of another part's box, plus an
- ↪ optional offset. It is the easiest way to stack parts exactly: `attach(child, "bottom",`
- ↪ `parent, "top")`.
- Parts count as connected when their real surfaces touch (gap under about 4 mm), or when
- ↪ a joint joins them at a place where both parts are: the joint's origin within 2 cm of
- ↪ each part's surface. A joint whose origin is far from either part connects nothing.
- ↪ Parts that interpenetrate a lot collide. Anything not connected to the rest is
- ↪ floating.

DECLARATIONS (all are scored)

- `role`: say what each part is, in plain words ("seat", "backrest", "caster wheel", ...).
- `material`: choose one per part from `list_materials`.
- `joints`: declare every place the real object moves. `add_joint(joint_id, type, parent,`
- ↪ `child, axis, origin, limits)`. `type` is one of fixed, revolute, continuous, prismatic,
- ↪ cylindrical, ball. `axis` is a direction in world coordinates; `origin` is a world point
- ↪ on the axis where parent and child meet;
- `limits` = [low, high] in radians (revolute) or metres (prismatic, cylindrical = the sliding
- ↪ range); omit limits for continuous. The child moves relative to the parent, carrying
- ↪ whatever is attached to the child.
- `assembly sequence`: `set_assembly_sequence` with every part once, in the order a person
- ↪ would build the object, each step optionally with the direction the part travels as it
- ↪ is put in place (e.g. [0, 0, -1] for lowering it from above).

ROUNDS AND BUDGETS

- `submit` ends a round. If review rounds remain you are shown a check of the design and
- ↪ renders, and you may revise it and submit again. After the final round the episode
- ↪ ends.
- check and render are metered; use them to verify, not to search. Tool calls are
- ↪ budgeted.

Keep reasoning brief. Prefer making tool calls over explaining.

Task prompt. The user message specifies the target category, optional reference image, optional fine-grained attributes and functional constraints, and the current environment tier. For example, the main Tier-B multimodal condition for an office chair is:

Example user prompt: Tier B with reference image

TASK: According to image W2, build an office chair. The condition image W2 is attached.
 Tier B: Use catalog parts, and create new parts where the catalog has none that fit. A
 ↪ picture of every catalog part, labelled with id, name and size, is attached.
 ↪ `view_catalog_sheet` shows it again. Start with `list_parts` and `list_materials`.

For name-only conditions, the first line is simply “Build an office chair.” For image-conditioned tasks, the corresponding image identifier is prepended. Attribute-conditioned tasks append descriptions of required components, materials, and other object properties. Functional conditions additionally provide subsystem and kinematic constraints, such as specifying that an office chair seat rotates relative to its base about a vertical axis. Tier A instructs the model to use catalog parts only, while Tier C states that no catalog exists, requires every placed component to be created, and begins with `list_materials` followed by `create_part`. For models without visual capability, image-conditioned conditions are preserved rather than silently converted to text: the message

explicitly states that the task refers to an image that the model cannot see. This allows the same benchmark condition to be retained while exposing the model’s actual input limitation.

A.1.2 CLOSED-SOURCE AGENT HARNESS

Claude models. Claude Haiku 4.5, Sonnet 5, Opus 5, and Fable 5.1 are evaluated through the Claude command-line interface. Each episode is placed in an isolated working directory and the model is given only the session interface and the image files returned by that session. The agent first requests the complete LMBuild prompt and subsequently invokes environment tools through shell commands of the form

```
.venv_eval/bin/python -m ppbench.v2.session call --state <STATE> <TOOL_NAME>
↪ '<JSON_ARGUMENTS>'
```

Each invocation returns one JSON object. Tools that produce images return PNG paths, which Claude may inspect using its image-reading capability. The outer wrapper is:

Claude shell wrapper

You are the model being evaluated in a 3D assembly benchmark. Work only through the
 ↪ session command below; do not edit, create or delete any files, and do not run any
 ↪ other programs except reading image files.

Your session state file: {state}

1. Run: {cmd} prompt --state {state}
 Read the whole output: it is your task, your tools and how to call them. Open every
 ↪ image file it names with your file-reading tool.
2. Build the object with the tools. Tool calls look like:
 {cmd} call --state {state} <tool_name> '<JSON arguments>'
 Each call prints one JSON result; image results are PNG paths you open to look at.
3. The episode may have several rounds: each submit ends a round and returns a review.
 ↪ Keep working until a submit result says "episode_done": true. If a call returns an
 ↪ error, read it and correct the call.

Reply at the end with one line: the number of parts and joints in your final design.

GPT models. GPT-5.6 Sol and GPT-6 Astra use the same session environment but are driven through codex. Their outer wrapper is slightly more restrictive with respect to filesystem access:

GPT shell wrapper

You are the model being evaluated in a 3D assembly benchmark. Work only through the
 ↪ session command below. Do not edit, create, or delete files directly. Do not inspect
 ↪ the reference assembly, evaluator data, other results, or repository code. You may
 ↪ read only the condition image, catalog sheets, and renders named by your session.

Your session state file is {state}.

Run `{cmd} prompt --state {state}` first and read its full output. Open the image files it
 ↪ names. Build the object using only session tool calls such as `{cmd} call --state
 ↪ {state} <tool_name> '<JSON arguments>'`. Each successful submit saves a round and
 ↪ returns feedback. Continue revising the same design until a submit returns
 ↪ `episode\_done: true`. If the saved session is already partly complete, call get_scene
 ↪ and continue from that state; never start over. Inspect your renders and correct
 ↪ errors before submitting. At the end, report the final part and joint counts.

Thus, Claude and GPT receive the same LMBuild environment prompt and tool interface; their outer wrappers differ only in provider-specific execution and isolation instructions. GPT episodes permit up to three execution attempts.

A.1.3 OPEN-SOURCE LLM/VLM AGENT HARNESS

Open-source LLMs and VLMs are served using vLLM through an OpenAI-compatible chat endpoint. This group includes Qwen3-VL, Qwen3.5, Gemma-4, InternVL3.5, MiniCPM-V, GPT-OSS,

Minstral-3, ERNIE-4.5-VL, Llama-3.1-8B, etc. Unlike the shell-based agents, these models interact with LMBuild through an in-process agent loop.

The system message is constructed as

```
system_prompt(tier) + tool_docs(tier) + tool-call protocol,
```

while the user message contains the task text, the condition image when the model supports vision, and the catalog sheet for Tier A/B. Condition images are resized to a maximum side length of 1024 pixels, while catalog sheets are resized to a maximum side length of 1600 pixels. Because the shared vLLM deployment does not require model-specific native tool parsers, LMBuild uses the following text protocol for all locally served agents:

Open-source tool-call protocol

Write each tool call as a block on its own: `<tool_call>{"name": "place_part", "arguments": {"part_id": "P001", "instance_id": "wheel_1", "position": [0, 0, 0.03]}}``</tool_call>`

You may write several blocks in one reply; they run in order and you then see every result. Arguments are strict JSON (double quotes, no comments, no trailing commas).

Tool outputs are returned as user messages using the form

```
<tool_result name="TOOL_NAME">{...JSON result...}</tool_result>
```

The parser first searches for explicit `<tool_call>` blocks and then supports a small number of model-specific fallback formats, including ERNIE-style `<name>/<arguments>` outputs, standalone or fenced JSON, and function-like expressions. Automatic repair is deliberately limited to evaluating simple arithmetic expressions in numeric arguments and closing truncated JSON structures. When parsing fails, the model receives the concrete parsing error and is instructed to resend a strict-JSON tool call. If a model produces no environment action, it is reminded that the interaction contains no human operator and that it should continue through tools or call `submit` once the object is complete. An episode is terminated after three consecutive responses containing no executable tool call. All locally served models use temperature 0.6 and random seed 0. Model-specific serving modifications are used only when required by the underlying model implementation.

A.1.4 DOMAIN-SPECIFIC GENERATION SYSTEMS

Domain-specific structure generators are not forced into the LMBuild agent interface when their native architectures do not support iterative tool use. Instead, each system receives the subset of the benchmark condition supported by its original inference pipeline and performs a single generation. We retain each method’s native representation and prompting procedure rather than adding capabilities unavailable to the original system. For these methods, Tier B is operationally identical to Tier A because their native generation pipelines do not expose LMBuild’s optional `create_part` action.

A.2 ENVIRONMENT DETAILS FOR LMBUILD

Implementation details of the LMBuild environment. LMBuild is a stateful, pure-Python tool environment. Entire episode state coming from LMBuild (placed instances with poses, created-part programs, joints, assembly sequence, counters and a full call trace) is treated as one JSON-serialisable dictionary, so the same episode can be driven in-process or from a shell, and it can be checkpointed after every round and replayed exactly.

World frame and parts All geometry of LMBuild lives in one metric world frame: $+Z$ up, ground plane at $z = 0$, object back facing Y . Catalog parts are triangle meshes taken from segmented reference assets and stored as glTF. On loading, trimesh converts each mesh with the defined convention and re-centers it on its bounding box. Therefore reduce to exact rigid transforms: a translation plus XYZ Euler rotations, or face-centered alignment of bounding boxes.

Part creation The part creation tool accepts a declarative CSG program. The program is a DAG of at most 96 nodes over box, cylinder, sphere, torus, extrude and revolve primitives. These can

Table 3: Comparison of domain-specific structure-generation systems and LLM agents in terms of the information received and produced throughout the construction pipeline. While LLM agents interact seamlessly with LMBuild, domain-specific systems cover only a subset.

#	System	Pipeline	Input	Geometry	Parts	Roles	Joints	Materials	Sequence
1	BrickGPT	Generates LEGO Brick Structures	Text	✓	✓	✗	✗	✗	✗
2	LegoACE	Generates LEGO Brick Structures	Text	✓	✓	✗	✗	✗	✗
3	PartCrafter-15	Generates a mesh of 15 parts.	Image	✓	✓	✗	✗	✗	✗
4	PartCrafter-8	Generates a mesh of 8 parts.	Image	✓	✓	✗	✗	✗	✗
5	PartPacker	Generates part meshes.	Image	✓	✓	✗	✗	✗	✗
6	Cube3D+CubePart	Generate geometry and decompose	Text	✓	✓	✓*	✗	✗	✗
7	Particulate +(3)	Re-segment PartCrafter-15 Mesh	Image	✓	✓	✗	✓	✗	✗
8	Particulate +(4)	Re-segment PartCrafter-8 Mesh	Image	✓	✓	✗	✓	✗	✗
9	Particulate +(5)	Re-segment PartPacker Mesh	Image	✓	✓	✗	✓	✗	✗
10	Particulate +(6)	Re-segment Cube-3D Mesh	Text	✓	✓	✗	✓	✗	✗
11	PhysX-Anything	Generation aligned with Physics	Image	✓	✓	✓	✓	✓	✗
–	LLM Agents (19)	Interact with LMBuild	Multimodal	✓	✓	✓	✓	✓	✓

be combined with union, difference, intersection and hull, and moved with transform and mirror. The program is compiled with manifold3d (v3.5), which guarantees watertight 2-manifold output. Every created part is therefore a valid solid with an exact volume. Parameters are range-checked (0.5 mm–3 m), curved primitives are tessellated with 48 segments, and parts that fall apart into disconnected pieces are flagged.

Physical feedback All physical feedback comes from one geometry kernel, built on NumPy and SciPy, that is shared with the evaluator. Each placed part is voxelised on a 4-mm grid: area-proportional surface sampling, then scipy gives a solid occupancy grid.

Material Definitions Declared materials are restricted to 23 submaterials in 9 classes. Their densities are medians over 5,510 Artiverse annotations.

Rendering Renders are produced out-of-process by a headless Blender (bpy 5.2 LTS) worker. The posed scene is exported as a GLB, with each part in a distinct hue so that parts can be told apart. Each image is rendered with Cycles on the CPU (448 px, 16 samples with denoising, AgX view transform) under three sun lights and above a ground plane. The camera has a 50 mm lens and auto-framing from the design’s bounding sphere. Six fixed (azimuth, elevation) viewpoints are available, including one matching the condition image’s camera. The rendered views are tiled with PIL. The catalog contact sheet is pre-rendered once in the same way.

Running restrictions The environment never repairs a model’s action. Invalid calls return explicit error messages and are counted. Budgets limitation include: 160 tool calls per round to restrict unlimited interactions. 6 check calls and 4 render calls per episode; the review returned by submit is free. An episode consists of up to three cumulative rounds (R1-R3). Each submit returns a free review (a check plus four views), and the design is checkpointed.

Comparability Across System Families LMBuild includes both general-purpose LLM agents and domain-specific structure-generation systems to provide a broad view of current capabilities for structured 3D generation. Comparisons should nevertheless be interpreted with respect to the outputs natively supported by each system. General-purpose LLM agents receive the same LMBuild task specification, environment semantics, tool definitions, and interaction budgets, and are evaluated end-to-end across geometry, parts, joints, materials, and assembly sequence. In contrast, domain-specific systems are evaluated using their native inference pipelines and may produce only a subset of these outputs, as summarized in Table 3. We therefore report a metric as N/A when the information required by that metric lies outside a system’s native output space, rather than treating the absence of an unsupported output as a generation failure. Comparisons between LLM agents and domain-specific systems should consequently be interpreted primarily on their shared applicable metrics, while the complete LMBuild evaluation for LLM agents reflects their ability to solve the full end-to-end construction task. We include domain-specific systems not to imply identical interfaces or capabilities, but to contextualize how specialized generation methods compare with general-purpose agents on the portions of the structure-generation pipeline they support.

Information exposed to agents. The environment communicates generic, system-independent construction rules and diagnostic feedback needed to use its tools, while target-specific evaluation sheets and final metric outcomes are not exposed to agents. We view these generic rules as part of the benchmark interface rather than hidden test labels. The functional-prompt experiments intentionally expose additional task requirements and should be interpreted as measuring an agent’s ability to translate explicit specifications into structures, whereas the baseline also tests whether those requirements can be inferred from a category label and reference image. Future work may separately study settings with weaker feedback, hidden interface tolerances, or interactive requirement elicitation.

We provide the complete list of 19 tools implemented in the LMBuild environment that models can invoke when constructing structures, together with description of the functionality of each tool.

Detailed Description for Tool Set Available in LMBuild

Tool: list-parts(query);

Function: Lists the parts currently available to the agent, including catalog parts and any parts created by the model, together with their IDs, names, and dimensions.

Tool: inspect-part(part-id);

Function: Inspects a specified part and returns its information, including dimensions, watertightness, surface area, and thumbnail path.

Tool: view-catalog-sheet();

Function: Displays a labeled visual sheet containing all parts available in the catalog. This tool is unavailable when there is no sub-part pool.

Tool: list-materials();

Function: Lists the available construction materials, including their material classes and physical densities.

Tool: place-part(part-id, instance-id, position, rotation, role-description);

Function: Places a selected part into the 3D scene at a specified position and optional rotation, and specify its role if possible. It also returns feedback about its bounding box, contacts, collisions, and ground penetration.

Tool: set-material(part-id, instance-id);

Function: Specify the material of a selected part.

Tool: move-part(part-id, position, rotation);

Function: Moves or rotates an existing part using either an absolute target position or a relative translation, while returning updated contact, collision, and ground-penetration information.

Tool: attach(instance-id, anchor, target, target-anchor, offset);

Function: Aligns an anchor of one part with an anchor of another part or with the ground, enabling structured placement without manually specifying exact coordinates.

Tool: measure(part a, part b);

Function: Measures the spatial relationship between two parts, including their minimum surface gap, closest points, touching status, and overlap volume.

Tool: `remove-part(instance-id);`

Function: Removes a placed part from the current structure, together with its associated joints and its entry in the assembly sequence.

Tool: `set-part-info(instance-id, role, material);`

Function: Updates the information associated with a placed part, such as its functional role and material assignment.

Tool: `remove-joint(joint-id);`

Function: Removes an existing joint declaration between two parts.

Tool: `add-joint(joint-id, type, parent, child, axis, origin);`

Function: Defines a joint relationship between two parts. Supported joint types include fixed, revolute, continuous, prismatic, cylindrical, and ball joints.

Tool: `set-assembly-sequence(steps);`

Function: Specifies the intended assembly order of the generated structure, with an optional insertion direction for each assembly step.

Tool: `create-part(name, program);`

Function: Creates a new custom part from a constructive solid geometry program. This tool is available when create new parts is enabled.

Tool: `get-scene();`

Function: Returns the current state of the construction environment, including all placed parts, their poses, bounding boxes, roles, materials, joints, assembly sequence, and remaining interaction budgets.

Tool: `check();`

Function: Use rule-based implementations to check the current structure for potential issues, including collisions, disconnected components, ground contact, stability problems, joint issues, and missing declarations. This tool is metered.

Tool: `render(view);`

Function: Generates visual renderings of the current structure from one or more specified viewpoints, allowing the agent to inspect and refine its design. This tool is metered.

Tool: `submit();`

Function: Ends the current interaction round. If additional rounds remain, it returns structural feedback and multiple rendered views before advancing to the next round; after the final round, the task is completed.

A.3 DATA SELECTION AND DATA EXAMPLES IN LMBUILD

Data Selection Strategy. LMBuild is constructed from a heterogeneous pool of approximately 367K raw 3D assets. Because these datasets differ substantially in geometry quality, semantic annotation, part decomposition, and kinematic information, we do not directly sample benchmark objects from the combined pool. Instead, we apply a five-stage selection and grounding pipeline designed to ensure that retained objects are structurally valid, sufficiently articulated, semantically identifiable, diverse in their underlying construction, and grounded in independently retrievable real-world knowledge.

Candidate Admission and Structural Validity. We first remove assets that are unsuitable for evaluating part-based structure generation. An object is admitted into the candidate catalog only if it contains at least four separately identifiable components, its geometry can be successfully loaded and rendered, and any available joint annotations can be resolved to parts belonging to the same object. These requirements exclude corrupted meshes, effectively monolithic objects, incomplete assemblies, and assets whose kinematic annotations refer to missing or external components. After this initial filtering step, approximately 74.7K objects remain as structurally valid candidates.

Source-aware and Diversity-aware Sampling. We next construct a manageable candidate subset while preserving structural diversity within each source and semantic category. Because the source datasets differ substantially in their characteristic level of decomposition, we use source-specific part-count ranges rather than imposing a single global threshold. We retain Artiverse objects with 4–60 parts, PartNeXt objects with 5–40 parts, LEGO assemblies containing 25–150 bricks, and Fusion 360 assemblies with 6–60 components and at least one movable joint. These ranges remove both trivial instances and extremely large assemblies whose complexity is dominated by dataset-specific decomposition rather than meaningful object structure. Within each source-category pair, we then perform diversity-aware sampling instead of random selection. Each candidate object is represented by a structural descriptor containing counts of semantic part roles together with counts of moving-part and joint-type combinations. We first select the instance closest to the component-wise median of these descriptors, yielding a representative medoid-like example. Subsequent instances are chosen using farthest-point sampling under ℓ_1 distance, repeatedly selecting the object that is most dissimilar from the examples already retained. This procedure favors coverage of distinct structural configurations rather than repeatedly selecting near-duplicate instances of the same design. Dataset-specific metadata are additionally used when necessary: LEGO categories are recovered from captions and assemblies are prioritized according to movable connectors such as hinges, axles, and ball joints, whereas Fusion 360 categories are assigned from rendered views using a VLM.

Semantic Concept Grounding and Visual Verification. Structural validity alone does not guarantee that a CAD instance corresponds to a well-defined real-world object category. We therefore ground every retained category to both Wikidata and WordNet. For each category name, we retrieve the top Wikidata search candidates and the corresponding WordNet noun synsets. A language model is permitted only to select among these retrieved candidates, rather than freely generating an entity identifier. The selected Wikidata item and WordNet synset must denote the general physical object type represented by the category; entities corresponding to brands, named products, artworks, locations, or other non-category concepts are rejected. Categories for which no suitable Wikidata entity can be identified are removed. We additionally verify the category assignment at the individual-instance level. A VLM observes two rendered views of each candidate object and determines whether the asset both matches the intended category and represents a reasonably complete instance of that object. This step removes fragments, isolated subassemblies, severely incomplete models, and semantically mislabelled examples that may have passed the metadata-based filters.

Kinematic Coverage and Core-set Construction. From the grounded candidate pool, we construct the LMBuild-Core benchmark by explicitly covering qualitatively different forms of mechanical organization. Rather than selecting the 50 core tasks solely according to object category frequency, we organize them into six broad kinematic families: (A) static load-bearing structures, (B) hinged mechanisms, (C) sliding mechanisms, (D) rolling or rotating systems, (E) serial kinematic chains, and (F) structures containing many joints attached to a common body. These families are intended to expose models to substantially different requirements for connectivity, motion, joint

specification, and physical realization. The final 50 core objects span a wide range of complexity, from small assemblies containing only four parts to highly decomposed structures containing as many as 417 parts. The remaining selected data form LMBuild-Full, containing 2,349 additional objects from 152 semantic categories. LMBuild-Full may share object categories with LMBuild-Core, but it contains no overlapping object instances, allowing the extended set to preserve broader intra-category structural variation without duplicating the tasks used in our primary evaluation.

Retrieval-based Real-world Knowledge Grounding. LMBuild further augments each selected object with external knowledge describing its expected components, materials, functional attributes, and mechanisms. Importantly, these annotations are not manually authored from the benchmark designers’ prior knowledge and are not inferred directly from the CAD geometry. Instead, we construct a reproducible evidence bundle from Wikidata, Wikipedia, WordNet, and, only as a corroborating source, ConceptNet. For each category’s Wikidata entity, we retrieve structured statements including has part, material, use, uses, powered by, and inverse part of relations. To recover information stated at a more general taxonomic level, we additionally traverse up to three levels of subclass of ancestors, retaining at most three ancestors per level and terminating at overly generic concepts such as physical object, product, or equipment. Part and material information may be inherited from these ancestors, but inherited parts are retained only when they have an English Wikipedia article and their own Wikidata entry independently confirms an appropriate part of relation. This conservative inheritance rule reduces the risk of propagating overly broad taxonomic properties to specific object categories. We also pin specific Wikipedia revisions for the target object, identified components, and mechanism-level concepts relevant to the object’s kinematic family. For example, hinged structures may retrieve evidence from articles such as *Hinge*, rotating systems from *Wheel* or *Caster*, and articulated mechanisms from *Kinematic pair*. For identified moving parts, we additionally retrieve a relevant Wikipedia article when available. WordNet part and substance meronyms are incorporated as supplementary structured evidence and may be inherited over up to two hypernym levels. ConceptNet is used only to corroborate evidence already supported by stronger sources and is never treated as the sole basis for benchmark annotations.

Evidence-constrained Claim Extraction and Verification. The retrieved evidence is converted into benchmark annotations through a two-model extraction and verification procedure. Qwen3.5-27B first proposes candidate claims describing object parts, attributes, subsystems, and kinematic relationships. Every proposed claim must include a supporting quotation from the retrieved evidence bundle, and claims whose quoted evidence cannot be matched verbatim to the source text are automatically discarded. This prevents the extraction model from introducing unsupported world knowledge not present in the retrieved sources. A second model, Gemma-4-31B, independently evaluates each surviving claim along two dimensions: whether it is supported by the cited evidence and whether it is typical of the general object class rather than an unusual property of a particular instance. Core properties and more detailed attributes are assessed separately. When a claim fails because its supporting evidence is incomplete or poorly localized, we permit a single re-grounding step that retrieves at most three additional sentences from the same evidence bundle; no unrestricted external generation is allowed during this stage. The final annotation therefore retains only claims that can be traced to explicit external evidence and judged to describe characteristic properties of the object category. This pipeline produces the 50-task LMBuild-Core benchmark together with the 2,349-object, 152-category extended set. Each retained knowledge claim remains traceable to its source through either a Wikidata entity and statement identifier or a pinned Wikipedia page revision and supporting quotation. This provenance is important for LMBuild because the benchmark evaluates not only whether a generated structure visually resembles an object, but also whether its parts, materials, affordances, and kinematics are consistent with independently grounded knowledge about how that object is constructed and functions.

Concrete Data Examples in LMBuild (1)



Specification the evaluator checks

Verified claims from the pinned Wikipedia and Wikidata sources; 6 further extracted claims were dropped by the verifier. The small tag says how each claim is checked on a design; hover a source for its quote. Full sources and verdicts are in the panel below.

Parts 2

- P1 **work surface** — provides a flat area for reading, writing, or using equipment
- P2 **drawers** — stores items such as office supplies and papers
- Wikidata “has part” (P527): [builders hardware](#) via Q14745, [drawer](#), [fiberboard](#) via Q14745, [furniture leg](#) via Q14748, [particle board](#) via Q14745, [plank](#) via Q14745

Attributes 6

- A1 **material** Desks are usually made of wood or metal, though glass is sometimes used.
- A2 **material** Modern mass-produced desks may be constructed from engineered wood products like fiberboard or particle board.
- A3 **property** A desk must be sturdy to support users who lean on it while working.
- A4 **use** Desks are used for academic, professional, or domestic activities such as reading, writing, or using computers.
- A5 **operating environment** Desks are found in schools, offices, homes, and libraries.
- A6 **user interaction** Users typically sit at a desk, either on a separate chair or a built-in chair.

Functional subsystems 1

- F1 **Storage subsystem**: drawers, compartments, pigeonholes
Stores office supplies, papers, and documents to keep the work surface clear.

Kinematics 1

- K1 **drawer** slides relative to desk frame *prismatic* range: from fully enclosed to fully extended

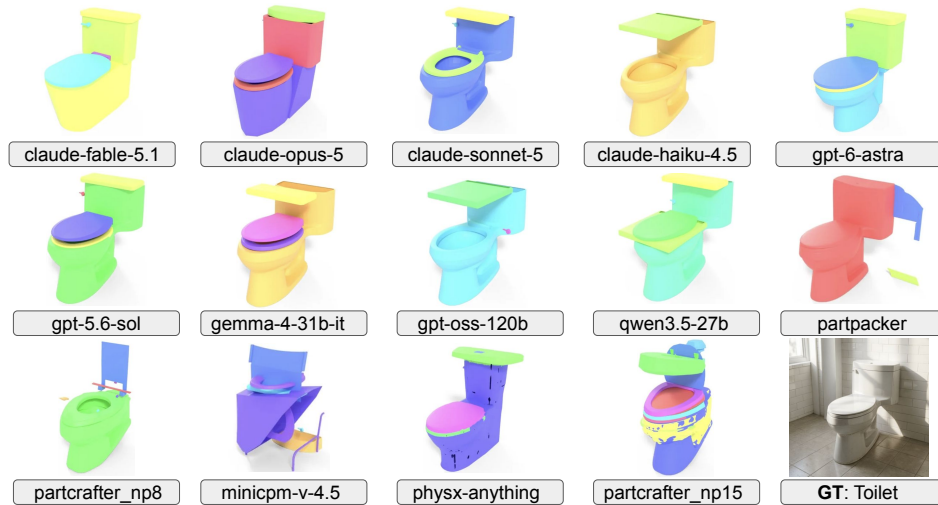
Attributes prompt

Build a desk. Include a work surface to provide a flat area for reading, writing, or using equipment and drawers to store items such as office supplies and papers. Use wood, metal, or glass, or engineered wood products like fiberboard or particle board. Ensure the desk is sturdy to support users who lean on it while working. Design it for academic, professional, or domestic activities such as reading, writing, or using computers. Ensure it is suitable for schools, offices, homes, and libraries. Design it so users typically sit at the desk, either on a separate chair or a built-in chair.

Functional prompt

Build a desk. Include a work surface to provide a flat area for reading, writing, or using equipment and drawers to store items such as office supplies and papers. Use wood, metal, or glass, or engineered wood products like fiberboard or particle board. Ensure the desk is sturdy to support users who lean on it while working. Design it for academic, professional, or domestic activities such as reading, writing, or using computers. Ensure it is suitable for schools, offices, homes, and libraries. Design it so users typically sit at the desk, either on a separate chair or a built-in chair. Incorporate a storage subsystem consisting of drawers, compartments, and pigeonholes to store office supplies, papers, and documents to keep the work surface clear. Install drawer slides relative to the desk frame that range from fully enclosed to fully extended.

Concrete Data Examples in LMBuild (2)



Specification the evaluator checks

Verified claims from the pinned Wikipedia and Wikidata sources; 5 further extracted claims were dropped by the verifier. The small tag says how each claim is checked on a design; hover a source for its quote. Full sources and verdicts are in the panel below.

Parts 5

- P1 **bowl** — collects human waste and holds water to seal against sewer gases
- P2 **trap** — holds water in the bowl to act as a seal against noxious sewer gases
- P3 **drain pipe** — removes effluent from the bowl to a sewer or septic tank
- P4 **flush mechanism** — releases a large volume of water rapidly to sweep waste into the drain
- P5 **cistern** — stores water for flushing in tank-style toilets
- Wikidata “has part” (P527): [dry toilet](#), [flush toilet](#), [toilet](#), [toilet bowl](#), [toilet building](#), [toilet seat](#), [toilets in japan](#)

Attributes 7

- A1 **material** Toilets are commonly made of ceramic (porcelain), concrete, plastic, or wood.
- A2 **use** The primary function is to collect human waste (urine and feces) and sometimes toilet paper for disposal.
- A3 **operating_environment** Flush toilets are usually connected to a sewer system in urban areas or a septic tank in isolated areas.
- A4 **power_source** Flush toilets use water as the primary medium to move waste, while dry toilets use no water.

- A5 **user interaction** Users operate the flush by pressing a button, pushing a handle, pulling a lever, or pulling a chain.
- A6 **property** The water in the bowl acts as a barrier to prevent sewer gas from entering the building.
- A7 **use** Toilets can be designed for sitting or squatting postures depending on regional preferences.

Functional subsystems 3

- F1 **Waste Collection and Disposal**: bowl, trap, drain pipe
Collects waste and uses a water seal to prevent gas entry while channeling waste to the sewer.
- F2 **Water Storage and Delivery**: cistern, fill valve, flush valve
Stores water and releases it rapidly into the bowl to flush waste.
- F3 **User Interface**: flush handle, flush button, seat
Allows the user to initiate the flush and provides a seating surface.

Kinematics 2

- K1 **flush handle** swings relative to cistern *revolute*
- K2 **flush valve (flapper)** swings relative to cistern outlet *revolute* range: lifts clear of the seat to allow flow

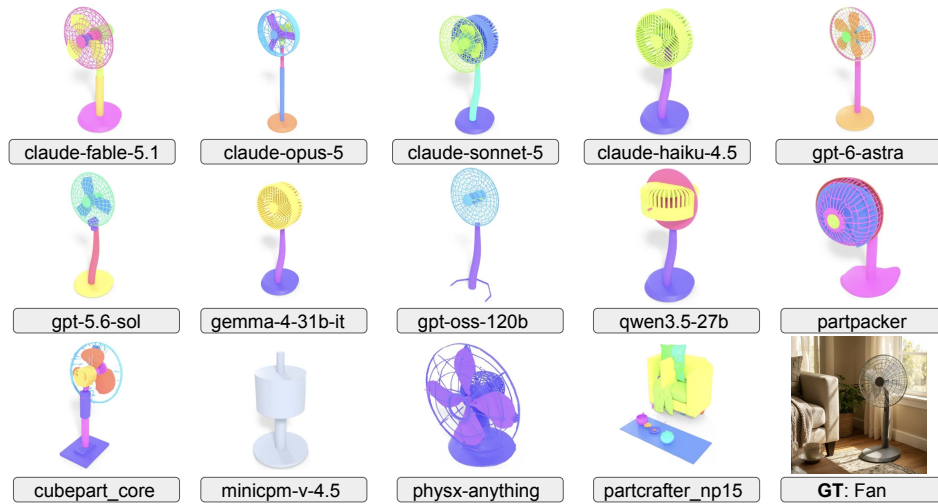
Attributes prompt

Build a toilet. Use a bowl to collect human waste and hold water to seal against sewer gases. Include a trap to hold water in the bowl as a seal against noxious sewer gases. Attach a drain pipe to remove effluent from the bowl to a sewer or septic tank. Incorporate a flush mechanism to release a large volume of water rapidly to sweep waste into the drain. Add a cistern to store water for flushing in tank-style toilets. Construct it from ceramic, concrete, plastic, or wood. Design it to collect human waste and sometimes toilet paper for disposal. Ensure it can connect to a sewer system in urban areas or a septic tank in isolated areas. Use water as the primary medium to move waste. Allow users to operate the flush by pressing a button, pushing a handle, pulling a lever, or pulling a chain. Ensure the water in the bowl acts as a barrier to prevent sewer gas from entering the building. Design it for sitting or squatting postures.

Functional prompt

Build a toilet. Use a bowl to collect human waste and hold water to seal against sewer gases. Include a trap to hold water in the bowl as a seal against noxious sewer gases. Attach a drain pipe to remove effluent from the bowl to a sewer or septic tank. Incorporate a flush mechanism to release a large volume of water rapidly to sweep waste into the drain. Add a cistern to store water for flushing in tank-style toilets. Construct it from ceramic, concrete, plastic, or wood. Design it to collect human waste and sometimes toilet paper for disposal. Ensure it can connect to a sewer system in urban areas or a septic tank in isolated areas. Use water as the primary medium to move waste. Allow users to operate the flush by pressing a button, pushing a handle, pulling a lever, or pulling a chain. Ensure the water in the bowl acts as a barrier to prevent sewer gas from entering the building. Design it for sitting or squatting postures. Create a Waste Collection and Disposal subsystem where the bowl connects to the trap and the trap connects to the drain pipe to channel waste and prevent gas entry. Create a Water Storage and Delivery subsystem where a fill valve carries fluid to the cistern, the cistern carries fluid to the flush valve, and the flush valve carries fluid to the bowl. Create a User Interface subsystem where a flush handle or flush button carries motion to the flush valve. Ensure the flush handle swings relative to the cistern. Ensure the flush valve swings relative to the cistern outlet and lifts clear of the seat to allow flow.

Concrete Data Examples in LMBuild (3)



Specification the evaluator checks

Verified claims from the pinned Wikipedia and Wikidata sources; 3 further extracted claims were dropped by the verifier.

Parts 7

- P1 **blades** — to act on the air and create airflow
- P2 **electric motor** — to convert electrical energy into mechanical rotation
- P3 **hub** — to connect the blades to the rotating shaft
- P4 **housing** — to direct airflow and prevent contact with blades
- P5 **blade guard** — to increase safety by preventing objects from contacting the fan blades
- P6 **oscillator gearbox** — to motion the fan from side to side
- P7 **base** — to support the fan and provide stability
- Wikidata “has part” (P527): [rotor](#), [impeller](#), [electric motor](#), [electrical load](#) via Q2425052

Attributes 4

- A1 **material** Blades are generally made of wood, plastic, or metal.
- A2 **power source** Most fans are powered by electric motors using AC or DC electricity.
- A3 **operating environment** Fans are effective for cooling people but do not lower air temperature directly.
- A4 **property** Fans produce air flows with high volume and low pressure.

Functional subsystems 3

- F1 **Airflow generation**: blades, hub, electric motor
Rotates blades to move air parallel to the shaft in axial fans or radially in centrifugal fans.
- F2 **Oscillation mechanism**: oscillator gearbox, oscillator shaft, base
Moves the fan head from side to side to distribute airflow.
- F3 **Safety enclosure**: housing, blade guard
Prevents objects from contacting the fan blades while allowing air to pass through.

Kinematics 1

- K1 **fan head** swings relative to base *revolute* range: side to side

Attributes prompt

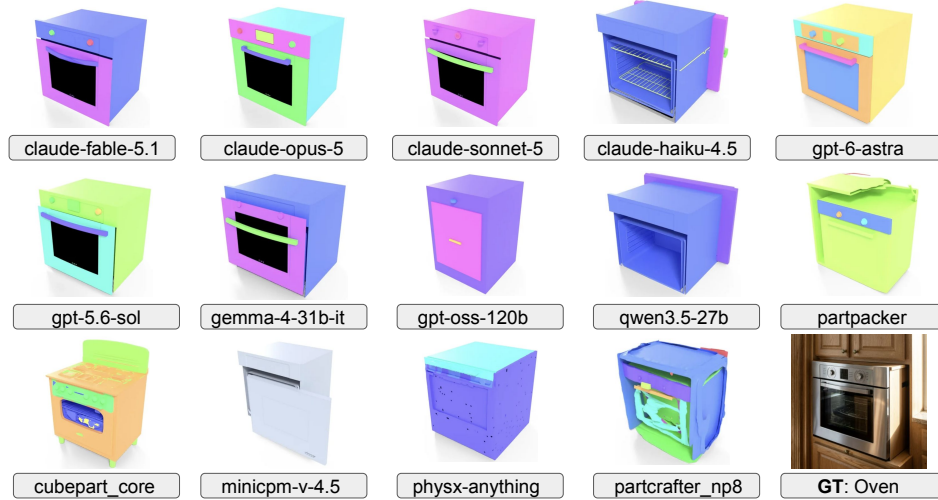
Build an electric fan. Use blades to act on the air and create airflow, an electric motor to convert electrical energy into mechanical rotation, and a hub to connect the blades to the rotating shaft. Include a housing to direct airflow and prevent contact with blades, a blade guard to increase safety by preventing objects from contacting the fan blades, an oscillator gearbox to motion the fan from side to side, and a base to support the fan and provide stability. The blades are generally

made of wood, plastic, or metal. Most fans are powered by electric motors using AC or DC electricity. Fans are effective for cooling people but do not lower air temperature directly. Fans produce air flows with high volume and low pressure.

Functional prompt

Build an electric fan. Use blades to act on the air and create airflow, an electric motor to convert electrical energy into mechanical rotation, and a hub to connect the blades to the rotating shaft. Include a housing to direct airflow and prevent contact with blades, a blade guard to increase safety by preventing objects from contacting the fan blades, an oscillator gearbox to motion the fan from side to side, and a base to support the fan and provide stability. The blades are generally made of wood, plastic, or metal. Most fans are powered by electric motors using AC or DC electricity. Fans are effective for cooling people but do not lower air temperature directly. Fans produce air flows with high volume and low pressure. For airflow generation, rotate blades to move air using the electric motor and hub, where the hub to blades carries motion. For the oscillation mechanism, use the oscillator gearbox and base to move the fan head from side to side to distribute airflow, where the oscillator gearbox to oscillator shaft carries motion and the oscillator shaft to base carries motion. For the safety enclosure, use the housing and blade guard to prevent objects from contacting the fan blades while allowing air to pass through, where the blade guard to motor case carries load. Ensure the fan head swings relative to base with a side to side range.

Concrete Data Examples in LMBuild (4)



Specification the evaluator checks

Verified claims from the pinned Wikipedia and Wikidata sources; 3 further extracted claims were dropped by the verifier. The small tag says how each claim is checked on a design; hover a source for its quote. Full sources and verdicts are in the panel below.

Parts 5

- P1 **heating chamber** — encloses the space where materials are exposed to heat
- P2 **heating element** — generates heat to raise the temperature inside the chamber
- P3 **rack** — supports food items inside the chamber for even heating
- P4 **thermostat** — regulates the temperature by turning the heating source on or off
- P5 **control interface** — allows the user to set temperature and operating modes
- Wikidata “has part” (P527): [oven](#), [grate](#), [zug](#)

Attributes 6

- A1 **use** Ovens are used to expose materials to a hot environment for cooking, baking, or industrial processing.
- A2 **power_source** Ovens are typically powered by electricity, natural gas, or combustion of solid fuels like wood or coal.
- A3 **property** The interior of an oven reaches high temperatures to cook or process materials.
- A4 **operating_environment** Ovens are commonly found in kitchens, restaurants, and industrial facilities.
- A5 **user_interaction** Users interact with ovens via controls to set temperature, time, and cooking modes.
- A6 **material** Oven chambers are constructed from fireproof materials such as brick, stone, clay, or metal.

Functional subsystems 4

- F1 **Heating System**: heating element, fuel source, combustion chamber
Generates and distributes heat within the chamber to cook or process materials.
- F2 **Temperature Control System**: thermostat, control interface, sensors
Maintains the chamber temperature at a user-defined setpoint by regulating the heating source.
- F3 **Access System**: door, hinge, handle
Provides a sealable opening for loading and unloading materials while retaining heat.
- F4 **Support System**: rack, shelf
Holds food or materials at specific heights within the chamber for optimal heat exposure.

Kinematics 2

- K1 **door** swings relative to oven frame *revolute*
- K2 **rotisserie spit** rotates relative to oven interior *revolute*

Attributes prompt

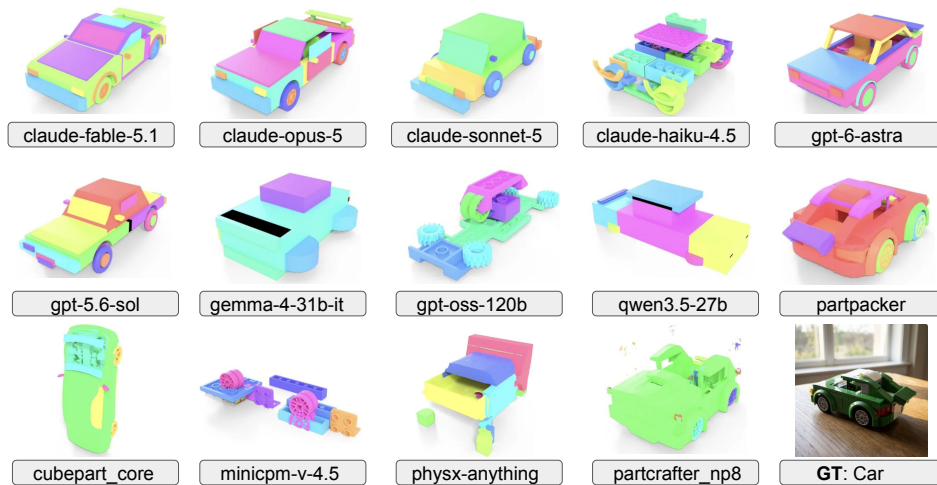
Build an oven. Use it to expose materials to a hot environment for cooking, baking, or industrial processing. Power it with electricity, natural gas, or combustion of solid fuels like wood or coal. Ensure the interior reaches high temperatures to cook or process materials. Place it in kitchens, restaurants, or industrial facilities. Allow users to interact via controls to set temperature, time, and cooking modes. Construct the heating chamber from fireproof materials such as brick, stone, clay, or metal. Include a heating chamber to enclose the space where materials are exposed to heat. Include a heating element to generate heat to raise the temperature inside the chamber. Include a rack to support food items inside the chamber for even heating. Include a thermostat to regulate the temperature by turning the heating source on or off. Include a control interface to allow the user to set temperature and operating modes.

Functional prompt

Build an oven. Use it to expose materials to a hot environment for cooking, baking, or industrial processing. Power it with electricity, natural gas, or combustion of solid fuels like wood or coal. Ensure the interior reaches high temperatures to cook or process materials. Place it in kitchens, restaurants, or industrial facilities. Allow users to interact via controls to set temperature, time, and cooking modes. Construct the heating chamber from fireproof materials such as brick, stone, clay, or metal. Include a heating chamber to enclose the space where materials are exposed to heat. Include a heating element to generate heat to raise the temperature inside the chamber. Include a rack to support food items inside the chamber for even heating. Include a thermostat to regulate the temperature by turning the heating source on or off. Include a control interface to allow the user to set temperature and operating modes. Implement a Heating System using a heating element, fuel source, and combustion chamber to generate and distribute heat within the chamber. Implement a Temperature Control System using a thermostat, control interface, and sensors to maintain the chamber temperature at a user-defined setpoint. Implement an Access System using a door, hinge, and handle to provide a sealable opening, where the door to the heating chamber carries load. Implement a Support System using a rack and shelf to hold materials, where the rack to the

heating chamber carries load. Ensure the door swings relative to the oven frame.
Ensure the rotisserie spit rotates relative to the oven interior.

Concrete Data Examples in LMBuild (5)



Specification the evaluator checks

Verified claims from the pinned Wikipedia and Wikidata sources; 11 further extracted claims were dropped by the verifier. The small tag says how each claim is checked on a design; hover a source for its quote. Full sources and verdicts are in the panel below.

Parts 9

- P1 **Engine** — converts fuel energy into mechanical motion to propel the vehicle
- P2 **Wheels and Tires** — support the vehicle load and provide traction on the road surface
- P3 **Steering System** — allows the driver to control the direction of the vehicle
- P4 **Brakes** — inhibit motion to slow or stop the vehicle
- P5 **Transmission** — transfers engine power to the wheels and adjusts torque and speed
- P6 **Fuel Tank** — stores the flammable fluid required for the engine to operate
- P7 **Seats** — provide a place for occupants to sit while the vehicle is in motion
- P8 **Doors** — allow ingress and egress for passengers and secure the vehicle interior
- P9 **Suspension** — absorbs shock impulses from the road to maintain tire contact and ride comfort

Attributes 6

- A1 **power_source** Most cars are powered by internal combustion engines burning fossil fuels, though electric motors are increasingly common.
- A2 **material** Car bodies are typically constructed from steel, aluminum, or increasingly from composite materials like carbon fiber.
- A3 **use** Cars are primarily used to transport people rather than cargo on roads.
- A4 **operating_environment** Cars are designed to operate primarily on paved roads.
- A5 **property** Modern cars typically weigh between 1 and 3 tonnes.
- A6 **user_interaction** Drivers control the vehicle using a steering wheel, pedals for braking and acceleration, and a gear shift.

Functional subsystems 2

- F1 **Propulsion System**: Engine, Fuel Tank, Transmission, Clutch, Drive Shaft, Differential, Axle
Generates power from fuel and transmits it to the wheels to move the vehicle.

- **F2 Steering System:** Steering Wheel, Steering Column, Rack and Pinion, Tie Rods, Steering Knuckle, Ball Joint
Translates driver input into the turning of the front wheels to change direction.

Kinematics 5

- **K1 Steering Wheel** rotates relative to Steering Column *revolute*
- **K2 Front Wheels** swings relative to Vehicle Chassis *revolute*
- **K3 Car Door** swings relative to Vehicle Body *revolute*
- **K4 Side Windows** slides relative to Door Frame *prismatic* axis: vertical axis
- **K5 Brake Pedal** swings relative to Dashboard/Floor *revolute*

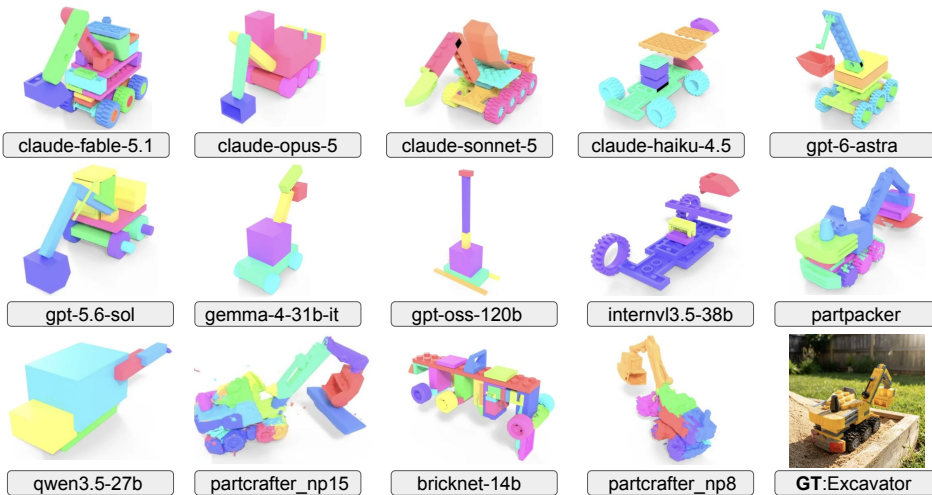
Attributes prompt

Build a car. Use an engine, wheels and tires, a steering system, brakes, a transmission, a fuel tank, seats, doors, and suspension. The vehicle is powered by internal combustion engines or electric motors and constructed from steel, aluminum, or composite materials. It is primarily used to transport people on roads, specifically paved roads. The car typically weighs between 1 and 3 tonnes. Drivers interact with it using a steering wheel, braking and acceleration pedals, and a gear shift.

Functional prompt

Build a car. Use an engine, wheels and tires, a steering system, brakes, a transmission, a fuel tank, seats, doors, and suspension. The vehicle is powered by internal combustion engines or electric motors and constructed from steel, aluminum, or composite materials. It is primarily used to transport people on roads, specifically paved roads. The car typically weighs between 1 and 3 tonnes. Drivers interact with it using a steering wheel, braking and acceleration pedals, and a gear shift. Integrate a propulsion system where the engine to transmission carries power and the differential to axle carries motion. Include a steering system where the steering wheel to steering column carries motion, the steering column to rack and pinion carries motion, the rack and pinion to tie rods carries motion, the tie rods to steering knuckle carries force, and the steering knuckle to ball joint carries force. Ensure the steering wheel rotates relative to the steering column, front wheels swing relative to the vehicle chassis, the car door swings relative to the vehicle body, side windows slide vertically relative to the door frame, and the brake pedal swings relative to the dashboard or floor.

Concrete Data Examples in LMBuild (6)



Specification the evaluator checks

Verified claims from the pinned Wikipedia and Wikidata sources; 5 further extracted claims were dropped by the verifier. The small tag says how each claim is checked on a design; hover a source for its quote. Full sources and verdicts are in the panel below.

Parts 9

- P1 **Undercarriage** — Provides the mobile base with tracks or wheels for locomotion
- P2 **House** — Contains the engine, operator cab, and hydraulic systems on a rotating platform
- P3 **Boom** — Provides the primary up-and-down movement for the digging arm
- P4 **Stick (Dipper)** — Provides the digging movement to pull the bucket through the ground
- P5 **Bucket** — Serves as the container for scooping and holding excavated material
- P6 **Engine** — Converts fuel energy into mechanical power to drive hydraulic pumps
- P7 **Hydraulic Pumps** — Convert mechanical engine power into high-pressure fluid flow for actuators
- P8 **Hydraulic Cylinders** — Convert fluid pressure into linear force to move the boom, stick, and bucket
- P9 **Swing Motor** — Rotates the house 360 degrees relative to the undercarriage

Attributes 5

- A1 **power_source** Excavators typically use an internal combustion engine to drive hydraulic pumps.
- A2 **use** Excavators are used for digging trenches, holes, foundations, and material handling.
- A3 **operating_environment** Excavators operate in construction sites, mining operations, and confined spaces.
- A4 **property** Hydraulic systems in excavators operate at high pressures up to 5000 psi for main functions.
- A5 **user_interaction** Operators control the machine using joysticks that distribute digging functions across two x-y controls.

Functional subsystems 3

- F1 **Hydraulic Power System:** Engine, Hydraulic Pumps, Hydraulic Oil Tanks, Hydraulic Cylinders, Hydraulic Motors
Converts engine mechanical energy into hydraulic fluid pressure to actuate all moving parts.
- F2 **Arm Assembly:** Boom, Stick, Bucket, Hydraulic Cylinders
Performs the digging, lifting, and dumping motions required for excavation.
- F3 **Travel System:** Undercarriage, Tracks, Track Motors, Final Drives
Propels the machine forward, backward, and turns via independent track drive.

Kinematics 4

- K1 **House** rotates relative to Undercarriage *revolute* axis: vertical center pin range: 360 degrees
- K2 **Stick** swings relative to Boom *revolute* range: closer and further
- K3 **Bucket** swings relative to Stick *revolute*
- K4 **Piston Rod** slides relative to Cylinder Barrel *prismatic* range: extension and retraction

Attributes prompt

Build an excavator. Use an undercarriage for the mobile base. Build a house containing the engine, operator cab, and hydraulic systems on a rotating platform. Attach a boom for primary up-and-down movement, a stick for digging movement, and a bucket for scooping material. Include an engine to convert fuel energy into mechanical power, hydraulic pumps to convert that power into high-pressure fluid flow, and hydraulic cylinders to convert fluid pressure into linear force. Add a swing motor to rotate the house 360 degrees relative to the undercarriage. The excavator uses an internal combustion engine to drive hydraulic pumps and is used for digging trenches, holes, foundations, and material handling. It operates in construction sites, mining operations, and confined spaces. The hydraulic systems operate at high pressures up to 5000 psi. Operators control the machine using joysticks.

Functional prompt

Build an excavator. Use an undercarriage for the mobile base. Build a house containing the engine, operator cab, and hydraulic systems on a rotating platform. Attach a boom for primary up-and-down movement, a stick for digging movement, and a bucket for scooping material. Include an engine to convert fuel energy into mechanical power, hydraulic pumps to convert that power into high-pressure fluid flow, and hydraulic cylinders to convert fluid pressure into linear force. Add a swing motor to rotate the house 360 degrees relative to the undercarriage. The excavator uses an internal combustion engine to drive hydraulic pumps and is used for digging trenches, holes, foundations, and material handling. It operates in construction sites, mining operations, and confined spaces. The hydraulic systems operate at high pressures up to 5000 psi. Operators control the machine using joysticks. Create a Hydraulic Power System using the engine, hydraulic pumps, hydraulic oil tanks, hydraulic cylinders, and hydraulic motors to convert mechanical energy into fluid pressure, where the engine connects to the hydraulic pumps. Assemble an Arm Assembly using the boom, stick, bucket, and hydraulic cylinders, where the boom carries load to the stick and the stick carries load to the bucket. Build a Travel System using the undercarriage, tracks, track motors, and final drives, where track motors and final drives carry motion to the tracks. Ensure the house rotates 360 degrees relative to the undercarriage on a vertical center pin. The stick must swing relative to the boom, and the bucket must swing relative to the stick. The piston rod must slide relative to the cylinder barrel.

Examples of Declared Joint Motion Figures 5–12 visualize joints declared in generated structures. Each row shows three poses of a generated structure under a fixed camera, illustrating the motion associated with one declared joint. Outlines identify the parts that move with the joint, and the orange rod indicates its declared axis. The moving set is determined by traversing the declared joint graph from the child part without passing through the parent part.

A.4 DETAILED EVALUATION METRICS FOR LMBUILD

Unless otherwise specified, all metrics are reported on a normalized scale of $[0, 100]$. We use $\text{clip}(x) = \min(1, \max(0, x))$ throughout the definitions below. Our evaluation framework comprises four hierarchical dimensions that assess the **Structural Soundness**, **Functional Affordance**, **Design Quality**, and **Physical Realization** of the generated structure. A concise overview of the evaluation question addressed by each metric is provided in Table 4.

Evaluation Framework Positioning. Most prior evaluation protocols for generated structures focus primarily on semantic alignment or geometric consistency. In contrast, our framework evaluates buildable structure generation across complementary dimensions of structural soundness, functional affordance, design quality, and physical realization. Importantly, our benchmark spans heterogeneous geometry domains, including LEGO assemblies, mechanical structures, everyday objects, and freely generated meshes, whose representations, part semantics, and physical constraints differ substantially. This cross-domain setting makes many existing domain- or representation-specific metrics difficult to apply uniformly, motivating evaluation criteria that can generalize across structure types. Several metrics necessarily introduce heuristic thresholds, tolerances, discretization choices, or weighting coefficients. These hyperparameters were selected based on geometric and physical reasoning, empirical inspection of representative structures, and practical considerations of the evaluation setting. AI tools were additionally consulted during metric development to help examine candidate formulations, identify potential edge cases, and improve the clarity of the evaluation rules; they were not used to determine benchmark outcomes or tune parameters to individual models or results. The resulting hyperparameters should therefore be interpreted as standardized evaluation parameters rather than theoretically optimal values or exact models of real-world physical behavior. All hyperparameters are fixed before benchmark-wide evaluation and applied identically across systems and outputs. We therefore view the proposed metrics as practical, interpretable, and reproducible comparative proxies rather than definitive solutions. Our goal is to provide a consistent basis for evaluating heterogeneous generated structures while making the assumptions and limitations of each metric explicit. Several dimensions, particularly design quality, human-use geometry, assembly feasibility, and operability, remain difficult to quantify in a fully unbiased and model-independent

Example joint · office desk · gpt-6-astra



Example joint · office desk · particulate-cube3d



Figure 5: Declared joint motion in office desk examples. Each row shows a generated structure in three poses under a fixed camera. Outlined parts belong to the moving set, and the orange rod marks the declared joint axis.

Example joint · toilet · claude-opus-5



Example joint · toilet · gpt-6-astra



Figure 6: Declared joint motion in toilet examples. Each row shows a generated structure in three poses under a fixed camera. Outlined parts belong to the moving set, and the orange rod marks the declared joint axis.

Example joint · oscillating fan · claude-opus-5



Example joint · oscillating fan · claude-fable-5.1



Figure 7: Declared joint motion in oscillating fan examples. Each row shows a generated structure in three poses under a fixed camera. Outlined parts belong to the moving set, and the orange rod marks the declared joint axis.

Example joint · wheelchair · claude-opus-5



Example joint · wheelchair · gpt-6-astra



Figure 8: Declared joint motion in wheelchair examples. Each row shows a generated structure in three poses under a fixed camera. Outlined parts belong to the moving set, and the orange rod marks the declared joint axis.

Example joint · laptop · claude-fable-5.1



Example joint · laptop · physx-anything



Figure 9: Declared joint motion in laptop examples. Each row shows a generated structure in three poses under a fixed camera. Outlined parts belong to the moving set, and the orange rod marks the declared joint axis.

Example joint · passenger car · claude-opus-5



Example joint · passenger car · claude-sonnet-5

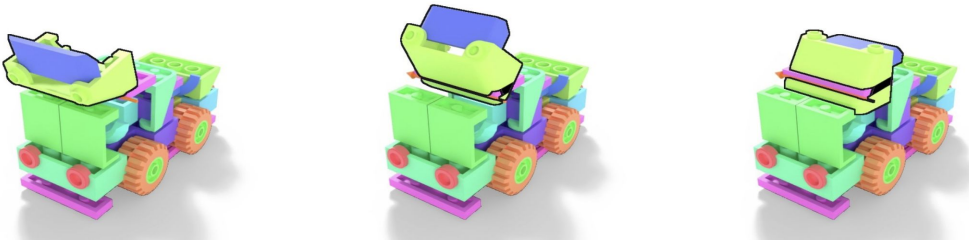


Figure 10: Declared joint motion in passenger car examples. Each row shows a generated structure in three poses under a fixed camera. Outlined parts belong to the moving set, and the orange rod marks the declared joint axis.

Example joint · farm tractor · claude-opus-5



Example joint · farm tractor · particulate-partpacker



Figure 11: Declared joint motion in farm tractor examples. Each row shows a generated structure in three poses under a fixed camera. Outlined parts belong to the moving set, and the orange rod marks the declared joint axis.

Example joint · wheeled excavator · claude-fable-5.1



Example joint · wheeled excavator · gpt-6-astra



Figure 12: Declared joint motion in wheeled excavator examples. Each row shows a generated structure in three poses under a fixed camera. Outlined parts belong to the moving set, and the orange rod marks the declared joint axis.

Table 4: Illustrative questions used to interpret the LMBuild evaluation metrics, using the construction of an off-road vehicle as a running example.

Metric	Question to Ask for Evaluation
S.1 Connectivity	Are the wheels, axles, chassis, and other connected components properly attached rather than floating, detached, or disconnected?
S.2 Collision	Do the wheels and surrounding components avoid unintended interpenetration with suspension, axles, or neighboring parts?
S.3 Stability	Can the vehicle remain stably supported on the ground without tipping, collapsing, or losing appropriate contact through its wheels?
A.1 Geometry	Are the wheels appropriately shaped, sized, oriented, and positioned so that they can contact the ground and support effective driving?
A.2 Parts	Does the vehicle contain the functional components required for driving, such as wheels, axles, steering elements, and appropriate supports?
A.3 Kinematics	Can the wheels rotate around their axles and, where required, steer or articulate along the intended motion axes?
D.1 Decomposition	Is the vehicle divided into meaningful granular components rather than overly monolithic or unnecessarily fragmented parts?
D.2 Aesthetics	Does the vehicle exhibit aesthetically elegant proportions, coherent overall form, and sufficiently refined geometry as a standalone design?
D.3 Alignment	Are the wheels, cabin, and other major components arranged consistently with the intended off-road vehicle design or reference?
R.1 Sequence	Can the components be assembled in a feasible order without becoming unstable during assembly or blocking subsequent assembly steps?
R.2 Material	Are the materials appropriate for each component’s physical role, such as compliant tires and sufficiently rigid and durable structural parts?
R.3 Operability	Once assembled, can the vehicle actually roll and steer through its intended mechanisms, with the required components usable?

manner. We hope this framework serves as a useful baseline and motivates further work toward more robust, generalizable, and standardized evaluation protocols for buildable structure generation.

Shared Evaluation Conventions. Each generated design consists of a collection of triangle-mesh parts represented in a coordinate system, with the vertical axis aligned to z . A design is required to additionally provide semantic role for each part, materials, declared joints with their type, axis, origin, and an assembly sequence. Evaluation uses a task-specific reference object together with curated factual claims describing required parts, functional subsystems, kinematics, and attributes extracted from **Wikipedia** knowledge. Unless otherwise specified, geometric tests use a common voxel representation with pitch $r = 4$ mm. For a part i , let G_i denote the set of voxels occupied by the part. We define $G_i^\ominus$ as its eroded core, obtained by removing the outermost one-voxel-thick boundary of G_i ; equivalently, a voxel remains in $G_i^\ominus$ only when its required neighboring voxels also belong to G_i . We define $G_i^\oplus$ as its one-cell halo, obtained by expanding G_i outward by one voxel, such that $G_i^\oplus$ contains G_i together with its immediately neighboring voxels. Using this representation, two parts form an interfacing pair when

$$\mathcal{I} = \{(i, k) : G_i^\oplus \cap G_k \neq \emptyset\}.$$

A declared joint between two incident parts is considered **origin-valid** when its declared origin lies within 2 cm of the surface of each part. This tolerance accommodates small discrepancies introduced by voxelization, mesh discretization, and coordinate quantization while excluding joint declarations that are spatially inconsistent with the corresponding part interface. We measure proximity to the part surfaces rather than their centroids because physical joints are expected to occur at or near

the interface between connected components. We distinguish physical connectivity from mere geometric contact. Two parts are considered **physically connected** if at least one of three conditions holds: (i) their geometry forms a valid interlocking relation that mechanically constrains separation; (ii) they are connected by a geometrically valid declared fastening relation, such as a hinge, pin, screw, or other supported joint type; or (iii) for brick-based structures, they form a valid stud–receptacle connection. This hybrid definition allows the evaluation to accommodate heterogeneous construction paradigms without treating simple surface contact or proximity as sufficient evidence of attachment. These definitions serve as shared geometric primitives and are reused consistently across the structural, decomposition, assembly, and operability evaluations described below.

All per-design metric scores lies in $[0, 1]$ and will be re-scaled to $[0, 100]$. Main-table values are reported as the mean score over benchmark tasks. Models which provide empty or failed generations receive a score of zero.

Structural Soundness – S.1 Connectivity

Although the target structures in our benchmark are decomposable into multiple constituent parts, the resulting generation should ultimately represent an integrated assembled artifact rather than a collection of independently plausible but disconnected components. Connectivity therefore measures the extent to which the generated parts are physically integrated into a coherent assembly. We first construct an assembly graph in which each generated part corresponds to a node and an edge is introduced only when the relationship between two parts is supported by explicit physical evidence. Such evidence may include a valid declared joint located at a physically plausible contact region, a geometric interlock, a brick–stud connection, or another direct part-to-part attachment supported by the generated geometry. This graph-based representation allows us to distinguish nominally related parts from components that are actually supported by a plausible physical connection. Let n denote the number of generated parts, m the number of connected components in the resulting assembly graph, C_1 the largest connected component, $V(C_1)$ the total volume of the parts contained in C_1 , V the total volume of the generated object, and n_{fast} the number of physically supported connections identified in the assembly. We compute

$$S_{1.1} = \frac{1}{3} \left(\frac{V(C_1)}{V} + \left(1 - \frac{m-1}{n-1} \right) + \min \left(1, \frac{n_{\text{fast}}}{n-1} \right) \right).$$

The three terms capture complementary aspects of assembly connectivity. The first term, $V(C_1)/V$, measures the fraction of the total object volume contained in the largest connected component. This volume-weighted formulation captures whether the dominant physical mass of the generated object belongs to a common assembly, while preventing a small detached decorative or auxiliary component from disproportionately reducing the score. The second term, $1 - (m-1)/(n-1)$, measures the degree of fragmentation at the part level. A fully connected assembly has $m = 1$ and therefore receives the maximum value, while the score decreases as the generated object is divided into an increasing number of disconnected components. In contrast to the volume-based term, this component remains sensitive to the number of disconnected groups even when those groups contain relatively small parts. The final term, $\min(1, n_{\text{fast}}/(n-1))$, measures whether the generated structure contains a sufficient number of connections that are explicitly supported by physical evidence. A connected graph of n nodes requires at least $n-1$ edges; we therefore use $n-1$ as a natural normalization factor. The value is capped at one because additional redundant connections may improve structural redundancy but should not increase a metric whose primary purpose is to assess whether the parts are sufficiently connected to constitute a single assembly. These three signals intentionally address different failure modes. A structure may contain one large connected body while leaving several smaller parts detached, producing a high largest-component score but a lower fragmentation score. Conversely, a structure may appear graph-connected through numerous declared relationships while those relationships are not supported by plausible physical contacts, which is reflected by the physically supported connection term. Their combination therefore evaluates not only whether most of the generated object belongs to a common component, but also whether the assembly is globally integrated and whether its connectivity is grounded in explicit physical evidence. For a valid single-part structure ($n = 1$), no inter-part connection is required; we therefore define the connectivity score as one.

Structural Soundness – S.2 Collision

A physically valid assembly should allow its constituent parts to occupy compatible regions of space without substantial unintended interpenetration. Collision therefore measures whether the generated components can coexist geometrically in a plausible assembled configuration. We do not require meshes to be strictly intersection-free, since small overlaps may arise from mesh discretization, floating-point imprecision, approximate geometric representations, or intentional connection geometry. Instead, we identify only overlaps that are sufficiently large to indicate a likely structural conflict.

For each geometrically relevant pair of parts $(i, k) \in \mathcal{I}$, let O_{ik} denote their overlap volume and V_i and V_k their respective part volumes. We classify the pair as an invalid collision only when both an absolute and a relative overlap criterion are satisfied:

$$O_{ik} > 1 \text{ cm}^3 \quad \text{and} \quad \frac{O_{ik}}{\min(V_i, V_k)} > \tau_{ik},$$

where

$$\tau_{ik} = \begin{cases} 0.25, & \text{if a valid joint is declared between parts } i \text{ and } k, \\ 0.10, & \text{otherwise.} \end{cases}$$

The collision score is then defined as

$$S_{1.2} = 1 - \frac{\left| \left\{ (i, k) \in \mathcal{I} : O_{ik} > 1 \text{ cm}^3 \wedge \frac{O_{ik}}{\min(V_i, V_k)} > \tau_{ik} \right\} \right|}{|\mathcal{I}|}.$$

The two-stage criterion avoids over-penalizing harmless geometric artifacts. The absolute threshold filters negligible intersections caused by numerical tolerance or mesh approximation, while the relative threshold identifies overlaps that occupy a meaningful fraction of either participating part. A more permissive threshold is used for explicitly jointed parts, since valid connection geometries may naturally introduce localized overlap. This formulation distinguishes plausible attachment geometry from substantial unintended interpenetration, such as components occupying the same physical region or overlapping far beyond what is necessary for a connection. The metric therefore measures the proportion of geometrically plausible part interactions rather than enforcing a strict zero-intersection requirement. When no geometrically relevant part pair exists, we define the collision score as one.

Structural Soundness – S.3 Reliability

A connected and collision-free structure may still be physically implausible if it cannot maintain a stable configuration under small external perturbations. Reliability therefore evaluates whether the generated artifact can remain upright under quasi-static loading. In this work, we focus specifically on **tipping stability**, which can be estimated from the relationship between the object’s center of mass and its support region without requiring a full rigid-body dynamic simulation. Let d denote the horizontal stability margin between the projection of the object’s center of mass and the nearest boundary of its support footprint, h_c the height of the center of mass above the supporting surface, and h_{top} the total height of the generated structure. A larger support margin relative to the object’s height generally indicates greater resistance to tipping. We derive two complementary stability proxies. The first is the critical tilt angle,

$$\theta = \arctan \left(\frac{d}{h_c} \right),$$

which approximates the angular perturbation required for the projected center of mass to cross the boundary of the support region. A larger θ therefore indicates that the object can tolerate greater angular displacement before becoming unstable. The second proxy is

$$f = \frac{d}{h_{\text{top}}},$$

which corresponds to the horizontal force applied at the top of the structure, expressed as a fraction of the object’s weight, required to initiate tipping under quasi-static torque balance.

This term captures stability with respect to lateral loading and complements the angular interpretation of θ . Because appropriate stability requirements vary substantially across object categories, we normalize both quantities relative to the corresponding real product rather than imposing one universal target. We define

$$\theta^* = \text{clip}_{[5^\circ, 30^\circ]}(\theta_{\text{ref}})$$

and

$$f^* = \text{clip}_{[0.05, 0.35]}(f_{\text{ref}}),$$

where θ_{ref} and f_{ref} are computed from the reference structure. The final reliability score is

$$S_{1.3} = \frac{1}{2} \left(\text{clip} \left(\frac{\theta}{\theta^*} \right) + \text{clip} \left(\frac{f}{f^*} \right) \right).$$

Reference-relative normalization is important because different artifact categories naturally exhibit different stability profiles. For example, a tall articulated structure and a wide low-profile object should not be expected to share the same tipping margin. Using the corresponding real product as a reference therefore provides a category-sensitive target. At the same time, clipping θ^* and f^* prevents unusually stable or unusually unstable reference objects from defining excessively demanding or permissive evaluation criteria. The two terms also capture complementary failure modes. A structure with a narrow support footprint and a high center of mass may tip under only a small angular perturbation, while a tall structure may also be vulnerable to relatively small lateral forces applied near its top. Averaging the two normalized proxies reduces dependence on any single geometric characteristic and provides a more balanced measure of quasi-static stability. This metric is intentionally limited to tipping reliability. It does not attempt to model sliding, elastic deformation, dynamic impact, material fracture, joint strength, or fatigue, which require more detailed material models and dynamic simulation. These aspects are therefore outside the scope of the current geometric stability evaluation.

Evaluation Basis for Functional Affordance. Functional Affordance evaluates whether a generated structure contains the geometry, constituent components, and articulation needed to support the intended object category. The evaluation is fully deterministic and does not rely on perceptual judging. Instead, its evaluation targets are derived from structured functional knowledge extracted from Wikipedia together with class-level statistics obtained from real object instances. For each task, we construct a component sheet from required-part (P), functional-subsystem (F), and kinematic (K) claims. Canonical component phrases are normalized through tokenization, removal of positional modifiers, singularization, spelling normalization, and merging of equivalent alternatives. Roles in the generated structure are then grounded to these canonical components using deterministic phrase matching, predefined lexical mappings, and component-specific fallback rules. To avoid treating a single reference decomposition as the only valid realization of an object category, we estimate component relevance and expected multiplicity from a population consisting of the task reference and up to eleven additional real instances from the same object class. A component is retained as evaluation-relevant if it occurs in the reference or appears in at least 40% of the available population instances. For plural components, we define the minimum expected multiplicity as

$$m_c = \min(8, \max(1, \lfloor Q_{25}(\text{count}_c) \rfloor)),$$

where Q_{25} denotes the lower quartile of the observed component counts. Using the lower quartile provides a conservative class-level expectation that accommodates legitimate variation across realizations, while the upper cap of eight prevents highly repetitive components from disproportionately dominating the evaluation. Singular components use $m_c = 1$.

Functional Affordance – A.1 Geometry

Functional objects must not only contain recognizable components, but should first, exhibit a global geometry consistent with the intended object category. Geometry therefore evaluates correspondence between the generated structure and the task reference at both the whole-object and, when semantic roles are available, component levels. Importantly, the metric is designed to

measure shape rather than absolute physical scale. We uniformly sample 20,000 surface points from both the generated design and the reference object. Each point cloud is independently centered using its horizontal bounding-box center and minimum height and then divided by its own bounding-box diagonal. Let p_τ denote the fraction of generated samples lying within distance τ of the reference surface and r_τ the corresponding fraction of reference samples lying within τ of the generated surface. We compute the bidirectional geometric F-score

$$F_\tau = \frac{2p_\tau r_\tau}{p_\tau + r_\tau}.$$

To account for ambiguity in the canonical front direction, we evaluate the design under four 90° yaw orientations and retain the best whole-object match:

$$F_{\text{shape}} = \max_{\psi \in \{0^\circ, 90^\circ, 180^\circ, 270^\circ\}} F_{0.025}(R_z(\psi)D, R).$$

When the design provides sufficiently informative part roles, we additionally evaluate component-level correspondence. A design is considered role-aware when at least 50% of its parts have non-generic semantic names. For each visible component c represented by at least 20 reference samples, we collect the generated samples belonging to parts whose roles ground to c and compute

$$F_c = F_{0.05}(D_c, R_c).$$

If the generated design contains no part grounded to component c , we set $F_c = 0$. Component comparison remains in the common normalized coordinate frame, such that the correct component geometry at an incorrect spatial location is appropriately penalized.

The final score is

$$A_{2.1} = \frac{0.6F_{\text{shape}} + 0.4\overline{F_c}}{0.6 + 0.4 \mathbf{1}[\text{component term defined}]}.$$

Thus, role-less designs are evaluated solely on overall shape, while designs that explicitly expose semantic part structure are additionally evaluated on whether those components occupy geometrically appropriate regions. Created parts and library-retrieved parts are treated identically. The metric therefore does not reward a particular construction mechanism; it evaluates the geometry ultimately produced by that mechanism.

Functional Affordance – A.2 Parts

Global geometric similarity alone does not establish whether a generated object contains the distinct functional components expected of the target category. The Parts metric therefore evaluates both component coverage and multiplicity, while explicitly allowing valid generated decompositions to differ from the segmentation of any single reference instance. Generated parts are first assigned to canonical components. When an informative semantic role is available and can be grounded to a canonical component, this assignment is made directly from the declared role. Otherwise, we fall back to spatial correspondence: a generated part is assigned to the reference component that covers at least 50% of its surface samples within normalized distance 0.05, provided that at least five samples are available. This fallback allows systems without explicit semantic labels to remain evaluable while requiring substantial geometric agreement before a component identity is inferred. Because a single functional component may itself be represented by multiple generated parts, parts assigned to the same canonical component are grouped into a common instance when they are connected by fixed joints or when their axis-aligned bounding boxes overlap by more than 20% of the smaller box. This prevents decomposition granularity alone from affecting component counts: for example, a wheel represented by separate rim, tire, and hub geometries may still correspond to a single functional wheel instance. Reference segmentations do not necessarily expose all legitimate internal functional structure. To avoid systematically penalizing such alternative realizations, we additionally identify anonymous interior modules. The generated object is voxelized using

$$r_{\text{int}} = \text{clip}\left(\frac{\text{diag}(D)}{150}, 0.5 \text{ mm}, 4 \text{ mm}\right),$$

and exterior air is recovered through a 6-connected flood fill. An unassigned, non-brick part is treated as interior when at most 5% of its boundary cells are adjacent to exterior air. Parts satisfying this criterion are grouped into anonymous internal modules. This mechanism does not assume that such modules correspond to a particular semantic component; rather, it provides conservative evidence that the generated design contains nontrivial internal structure that may implement a required but unobserved function. For each canonical component c , an identified instance receives score 1 when it participates in the connected assembly and score $1/2$ when all of its constituent parts are floating. This distinction ensures that mere geometric presence is not treated as equivalent to functional integration into the artifact. Given the task-specific minimum expected count m_c , the component score is

$$s_c = \frac{\sum_{j=1}^{m_c} s_c^{(j)}}{m_c},$$

where $s_c^{(j)}$ are the m_c highest instance scores. Thus, a design receives full credit only when the required number of component instances is present and integrated, while additional redundant instances do not inflate the score. For a non-visible component for which no explicit instance can be identified, one anonymous interior module may be consumed to provide partial credit of $1/2$. This deliberately conservative treatment acknowledges plausible hidden functional structure without equating an unlabeled internal module with an explicitly identified component. The final Parts score is

$$A_{2.2} = \frac{2}{3} \overline{s_c}^{\text{visible}} + \frac{1}{3} \overline{s_c}^{\text{internal}},$$

with the weights renormalized when either group is empty. The larger weight on visible components reflects the stronger evidence available for externally identifiable structure, while the internal term preserves sensitivity to hidden functional organization. Overall, the metric is designed to evaluate whether the expected functional parts are present in sufficient number and meaningfully integrated, without requiring the generated artifact to reproduce a particular reference decomposition.

Functional Affordance – A.3 Kinematics

Many buildable objects are defined not only by the presence and arrangement of their parts, but also by the relative motions that enable their intended function. Kinematics therefore evaluates whether generated moving joints provide the expected motion type, orientation, placement, range, moving-part extent, and collision-free articulation required by the target object. The metric is deliberately stricter than simply checking whether a non-fixed joint has been declared: a useful articulation must be both semantically appropriate and geometrically realizable. Kinematic targets are constructed from two complementary sources. First, each non-fixed reference articulation whose moving child can be grounded to a canonical component defines a reference-based motion target. Revolute, continuous, cylindrical, and screw joints are grouped as rotational motion; prismatic joints define translational motion; and ball joints define spherical motion. Second, each curated K claim whose moving phrase grounds to a canonical component defines a claim-based target. When a claim describes the same articulation as a reference joint, the two are merged into a single target rather than counted independently. Claims associated with non-visible components are treated as **declared-only**: they may justify a generated joint when computing precision, but they do not introduce an additional recall requirement. This distinction prevents incomplete observability from creating unsupported mandatory motions while still allowing semantically plausible articulations to be recognized. Where sufficient real-instance evidence is available, expected kinematic properties are derived from the population rather than imposed as universal assumptions. These properties include the permissible joint type, whether the motion axis should align with a symmetry or principal axis, whether its direction should be vertical or horizontal, whether the joint should lie near the component center or edge, its expected angular span or translation ratio, and the maximum plausible fraction of the artifact that should move with the joint. When population evidence is insufficient, the corresponding generic expectation is used. This combines category-specific evidence with

deterministic fallback rules, allowing the metric to accommodate heterogeneous object classes without requiring a single kinematic template for all structures. For a generated joint j , we define joint quality as

$$q_j = q_j^{\text{type}} q_j^{\text{axis}} q_j^{\text{offset}} q_j^{\text{range}} q_j^{\text{moveset}} q_j^{\text{free}}.$$

The multiplicative form is intentional: these terms represent complementary conditions that are jointly required for a useful articulation, such that a severe failure in any one dimension should substantially reduce the validity of the overall motion. The type term is 1 for a compatible joint, 0.75 when a cylindrical joint is used for an otherwise compatible rotational or translational target, and 0 for an incompatible type. Axis consistency is evaluated using

$$g(e) = \text{clip} \left(\frac{30^\circ - e}{20^\circ}, 0, 1 \right),$$

where e denotes the relevant angular deviation from the expected symmetry, principal, vertical, or horizontal axis. The continuous falloff avoids introducing a brittle binary decision near a single angular threshold while still assigning no credit to substantially misaligned motion axes. Joint placement is evaluated relative to the geometry of the corresponding component. Let o denote the distance from the component-instance center to the joint axis, normalized by the component extent in the relevant direction. A center-oriented joint receives

$$q_j^{\text{offset}} = \text{clip} \left(\frac{0.7 - o}{0.4}, 0, 1 \right),$$

whereas an edge-oriented joint receives

$$q_j^{\text{offset}} = \text{clip} \left(\frac{o - 0.2}{0.3}, 0, 1 \right).$$

Offsets larger than 1.5 receive zero credit. These two cases capture common functional distinctions such as rotation about a wheel center versus articulation about the edge of a door-like component. The range term evaluates whether the declared motion is sufficiently large to support the intended articulation. For rotational targets,

$$q_j^{\text{range}} = \min \left(1, \frac{\text{span}_j}{\text{span}^*} \right),$$

while for translational targets,

$$q_j^{\text{range}} = \min \left(1, \frac{|\Delta \text{limit}_j|}{\rho^* L_{\text{move}}} \right),$$

where ρ^* is the expected travel ratio and L_{move} is the moving set extent along the joint axis. Ball joints receive full range credit. These ratios measure whether a declared joint merely permits motion in principle or provides a motion extent consistent with the intended function.

The moving set contains the joint child together with geometry reachable through subsequent joints and, for otherwise joint-less parts, physical contact, without traversing through the parent. A motion receives no moving set credit if this set contains the target’s relative component or degenerates into motion of the entire object. Otherwise, the score decreases linearly when the moving fraction exceeds the target-specific expected maximum. This term is important because a formally valid joint is not functionally meaningful if it causes both sides of the intended articulation—or nearly the entire artifact to move together. Finally, q_j^{free} evaluates whether the declared articulation is geometrically realizable rather than merely specified in metadata. We voxel-sweep the moving set through multiple poses over the declared motion range and measure the fraction of poses that do not introduce substantial deep overlap with static geometry. Full rotations use 11 poses at 30° increments; other motions use eight poses over their declared interval. A moving set that is already deeply embedded in static geometry at rest receives zero free-motion credit. This discrete sweep is intended as a deterministic geometric feasibility test rather than a full dynamic simulation. We aggregate joint-level measurements through both recall and precision. Each required target takes the best joint quality obtained by each of its component instances, and the target receives the average of its best m_c instances. Averaging

over all targets gives recall R , which measures whether the expected articulations are successfully realized. For precision, every generated moving joint receives the best quality it achieves on any required target or declared-only claim. A joint unmatched to any such target may still receive half of its quality under the generic expectation for its own joint type, allowing mechanically plausible but unclaimed articulations to receive limited credit without being treated as equivalent to expected motions. Letting P denote the mean score over generated moving joints, the final metric is

$$A_{2.3} = \frac{2RP}{R + P}.$$

If no moving joint is generated, $P = 0$. The harmonic aggregation prevents systems from obtaining a high score either by omitting required articulations or by introducing large numbers of unsupported moving joints. The metric is not applicable to tasks for which no kinematic target exists; in the current benchmark, this only includes: bed-frame, dining-chair, dining-table, and step-ladder.

Design Quality: Perceptual Evaluation and Human Calibration.

Design Quality captures aspects of generated structures for which no single deterministic geometric criterion provides an adequate evaluation. We therefore use a **Calibration-then-Evaluation** protocol: human judgments on a fixed, pre-declared calibration subset are used only to calibrate the scale and relative contribution of automated evaluation signals; the resulting calibration parameters are then frozen and applied uniformly to the remaining benchmark outputs.

Our automatic perceptual signal is obtained from two independent VLM judges, Qwen2.5-VL-32B-Instruct and Gemma-3-27B-IT. Each judge is provided with explicitly defined, metric-specific judging criteria and assigns a raw score $s \in 1, \dots, 10$ to each criterion. We normalize each raw judgment as $(s - 1)/9$. Let $J \in [0, 1]$ denote the mean of these normalized scores across the defined criteria and the two VLM judges. Where an objective geometric or programmatic measurement is available, we denote the corresponding computed signal by C . Thus, for a given Design Quality metric, the available automated evaluation signals are drawn from $h \in J, C$.

Because the raw perceptual and computed signals need not naturally share the same scale as human judgments, each available signal h is independently calibrated. Let h' denote its calibrated value, and let ℓ_h and u_h denote the raw values of h that map to calibrated scores 0 and 1, respectively. We define

$$h' = \text{clip}\left(\frac{h - \ell_h}{u_h - \ell_h}, 0, 1\right).$$

Human calibration targets are collected on a 1–5 rating scale. Let $r \in 1, \dots, 5$ denote a raw human rating, and let

$$H = \frac{r - 1}{4}$$

denote its normalized value in $[0, 1]$. To determine the calibration range for each automated signal h , we fit an affine mapping from h to the human score distribution. Let b denote the fitted scale factor and a the fitted offset. These parameters are obtained by matching the mean and standard deviation of the transformed automated scores to those of the normalized human ratings:

$$b = \frac{\text{sd}(H)}{\text{sd}(h)}, \quad a = \overline{H} - b\overline{h}.$$

The corresponding raw-score endpoints that map to calibrated values 0 and 1 are then

$$\ell_h = -\frac{a}{b}, \quad u_h = \frac{1 - a}{b}.$$

Accordingly, ℓ_h and u_h define the metric-specific calibration interval for signal h .

Calibration is fit on 48 pre-declared items spanning eight objects. When both perceptual and computed signals are available, we combine their calibrated values using nonnegative weights α and β , where α controls the contribution of the perceptual signal J' and β controls the contribution of the computed signal C' . The ratio $\alpha : \beta$ is selected on the calibration set using leave-one-object-out mean absolute error (MAE), such that all items associated with the held-out object are excluded when evaluating each candidate blending ratio. After this fitting stage, the calibration ranges and blending weights are fixed and reused unchanged for benchmark evaluation.

The resulting metric-specific calibration parameters are

Metric	$\ell_J \rightarrow u_J$	$\ell_C \rightarrow u_C$	$\alpha : \beta$
$D.1$	$0.3611 \rightarrow 0.7644$	$0.1276 \rightarrow 1.0247$	$1 : 1$
$D.2$	$0.3741 \rightarrow 0.7251$	–	$1 : 0$
$D.3$	$0.4500 \rightarrow 0.7766$	$0.1536 \rightarrow 0.6057$	$3 : 1$

where J' and C' denote the calibrated perceptual and computed signals obtained from the transformation above. When both signals are available, the final Design Quality metric is

$$D = \frac{\alpha J' + \beta C'}{\alpha + \beta}.$$

When only one signal is defined, its calibrated value is used directly.

Human Annotation Details for Design Evaluation. We recruited four university lab volunteers to annotate 90 objects generated by six systems. Annotators provided 1–5 ratings for decomposition quality, aesthetics, and reference alignment. We used human ratings from a subset of eight objects to calibrate our Design evaluation metrics, and evaluated the resulting metrics on a disjoint held-out set, as reported in Table 5. The calibrated metrics show strong agreement with human judgments at the system level, with Spearman correlations of 0.77, 0.83, and 0.94 for decomposition, aesthetics, and alignment, respectively. Item-level correlations are more moderate but remain positive across all three dimensions. Overall, these results suggest that our Design metrics capture human-perceived differences between systems reasonably well, while still leaving room for disagreement at the individual-object level.

Design Quality – D.1 Decomposition

A buildable structure should be decomposed into components that correspond to meaningful and independently realizable parts, rather than represented as either a single monolithic mesh or an arbitrary collection of fragments. At the same time, many real objects admit multiple valid decompositions, and the segmentation of a single reference instance should not be treated as the unique ground truth. Decomposition therefore combines a perceptual assessment of whether the proposed part organization is meaningful with conservative programmatic signals that capture specific decomposition failure modes. The perceptual and computed signals are calibrated independently using the common Calibration-then-Evaluation protocol described above and are then combined using the pre-determined D.1 blending ratio.

Perceptual decomposition quality. To expose the generated part organization to the perceptual judges, each part is rendered using a distinct color so that part boundaries remain visually identifiable. For designs containing at most 40 parts, the decomposition sheet contains two assembled views and two exploded views, with each part displaced outward from the object center by 0.25 times the object diagonal. For designs containing more than 40 parts, non-exploded front and top views are used instead to avoid excessive visual clutter. The judges additionally receive the task condition image for semantic context and are explicitly informed of the generated part count.

Each judge evaluates three complementary criteria: **boundary quality**, **granularity**, and **separability**. Boundary quality measures whether part divisions occur at structurally or semantically meaningful locations. Granularity measures whether the decomposition uses an appropriate level of detail rather than being excessively fragmented or overly coarse. Separability

bility measures whether the resulting components correspond to plausible independently realizable parts. Let $x_{q,c} \in 1, \dots, 10$ denote the raw score assigned by judge q to criterion c . Let $\mathcal{C} = \text{boundary, granularity, separability}$ denote the three perceptual criteria, and let $x_{q,c} \in 1, \dots, 10$ be the score assigned by judge q to criterion c . The perceptual decomposition signal is

$$J_{3.1} = \frac{1}{6} \sum_{q \in \{\text{Qwen, Gemma}\}} \sum_{c \in \mathcal{C}} \frac{x_{q,c} - 1}{9}.$$

Programmatic decomposition quality. We complement the perceptual assessment with a computed signal $C_{3.1}$ consisting of the programmatic terms that are defined for the generated design. These terms are not intended to prescribe a unique decomposition; instead, they provide conservative evidence for three concrete failure modes: substantial disagreement in decomposition granularity, unnecessary fragmentation of otherwise continuous components, and geometrically unsupported joint placement.

The first term measures reference-relative decomposition granularity. Let n denote the number of generated parts and n_{ref} the number of parts in the segmented reference. We define

$$C_{\text{gran}} = \frac{\min(n, n_{\text{ref}})}{\max(n, n_{\text{ref}})} = 2^{-|\log_2(n/n_{\text{ref}})|}.$$

This formulation symmetrically penalizes proportional over- and under-decomposition: for example, generating twice as many parts as the reference receives the same score as generating half as many. Importantly, the reference part count serves only as supporting evidence of decomposition granularity rather than as a required target, since alternative physically meaningful realizations may legitimately contain different numbers of components. The second term detects unnecessary fragmentation of a component that otherwise appears continuous. This signal is defined when material declarations are available. Let V_{split} denote the number of interfacing pairs $(i, k) \in \mathcal{I}$ that simultaneously satisfy three conditions: the two parts ground to the same canonical semantic role, use the same material, and have no declared joint between them. Such a configuration provides evidence that geometry corresponding to a common functional component may have been unnecessarily divided. We define

$$C_{\text{consol}} = \text{clip} \left(1 - \frac{V_{\text{split}}}{\max(n-1, 1)}, 0, 1 \right).$$

The normalization by $\max(n-1, 1)$ scales the penalty relative to the size of the generated decomposition, while clipping ensures that the term remains within the common $[0, 1]$ score range even when the number of flagged interfacing pairs exceeds $n-1$. This can occur because a part may interface with multiple neighboring parts. The criterion is intentionally conservative: geometric contact alone is insufficient to identify unnecessary fragmentation. Parts must additionally share the same canonical semantic role and material, and the absence of a declared articulation prevents intentionally movable interfaces from being penalized. The third term evaluates whether declared joints are supported by the corresponding geometric interfaces. Among interfacing part pairs carrying a declared joint, let N_{decl} denote the number of such declarations and N_{rej} the number that fail the shared fastening criterion defined in the common evaluation conventions. When at least one such declaration is available, we compute

$$C_{\text{joint}} = 1 - \frac{N_{\text{rej}}}{N_{\text{decl}}}.$$

This term prevents an implausible decomposition from receiving support merely through arbitrary joint declarations: a declared connection contributes positively only when its placement and relationship to the incident parts are geometrically supported. When no applicable joint declaration exists, this term is treated as unavailable rather than as evidence for or against the decomposition. The raw computed decomposition signal is the mean of the programmatic terms available for the design:

$$C_{3.1} = \text{mean} (C_{\text{gran}}, C_{\text{consol}}, C_{\text{joint}})_{\text{available}}.$$

Calibration and aggregation. The perceptual signal $J_{3.1}$ and computed signal $C_{3.1}$ are independently calibrated to the human rating scale using the common calibration procedure described above, producing $J'_{3.1}$ and $C'_{3.1}$. The calibration parameters and blending ratio are determined during the pre-declared calibration stage and remain fixed during benchmark evaluation. As established by this procedure, D.1 assigns equal weight to the perceptual and computed signals:

$$D_{3.1} = \frac{J'_{3.1} + C'_{3.1}}{2}.$$

When only one signal is available, its calibrated value is used directly. This hybrid formulation deliberately avoids treating the segmentation of a single reference object as the unique correct decomposition. Instead, the perceptual component evaluates whether the proposed organization forms meaningful and independently realizable parts, while the programmatic component provides complementary evidence for specific structural failure modes, including severe granularity mismatch, unnecessary fragmentation, and unsupported joint placement.

Design Quality – D.2 Aesthetics

Aesthetics evaluates the intrinsic visual quality of the generated structure independently of its exact correspondence to a particular reference instance. This distinction is important because buildable objects may admit substantial variation in valid design: a generated artifact may differ from the reference while still exhibiting well-resolved proportions, coherent form, and high geometric refinement. Conversely, approximate reference similarity does not by itself imply a visually well-resolved design. To isolate geometric form from superficial appearance, all generated components are rendered using the same neutral gray material. Colors, textures, semantic labels, and material assignments therefore do not contribute to this metric, and individual part boundaries are intentionally deemphasized. The judges receive only the generated clay renderings and are not shown the reference object or task condition image. Aesthetics is therefore evaluated as a reference-free property of the generated design. Each judge evaluates three criteria: **proportion**, **coherence**, and **refinement**. Proportion measures whether the relative dimensions and scales of the visible forms produce a plausible and balanced whole. Coherence measures whether the geometry reads as a unified design rather than as an arbitrary collection of independently generated shapes. Refinement measures whether the major forms, transitions, and characteristic geometric details are sufficiently resolved rather than remaining coarse or unfinished. Let

$$\mathcal{Q} = \{\text{Qwen, Gemma}\}, \quad \mathcal{C}_{\text{aest}} = \{\text{proportion, coherence, refinement}\},$$

and let $x_{q,c} \in 1, \dots, 10$ denote the score assigned by judge $q \in \mathcal{Q}$ to criterion $c \in \mathcal{C}_{\text{aest}}$. The raw perceptual signal is

$$J_{3.2} = \frac{1}{6} \sum_{q \in \mathcal{Q}} \sum_{c \in \mathcal{C}_{\text{aest}}} \frac{x_{q,c} - 1}{9}.$$

Unlike D.1 and D.3, Aesthetics deliberately contains no separate programmatic component. Simple geometric proxies such as reference overlap, symmetry, or part count would impose assumptions about what a visually successful design should look like and would partially conflate intrinsic design quality with reference fidelity. D.2 therefore follows the same Calibration-then-Evaluation protocol using only its perceptual signal: $J_{3.2}$ is calibrated to the human rating scale using the pre-declared calibration set, and the resulting calibration parameters are subsequently frozen for benchmark evaluation. The final score is

$$D_{3.2} = J'_{3.2}.$$

This metric is consequently reference-free by design. It rewards generated artifacts whose geometry is proportionally plausible, visually integrated, and sufficiently refined, without requiring reproduction of the particular design choices present in the benchmark reference.

Design Quality – D.3 Structural Alignment

Structural Alignment evaluates whether the generated artifact preserves the characteristic organization and design-defining structure expressed by the task condition. Unlike D.2 Aesthetics, which evaluates the generated object independently of any reference, D.3 is explicitly reference-conditioned. At the same time, it does not require exact geometric reconstruction: multiple valid realizations may preserve the same characteristic design while differing in detailed geometry, construction strategy, or decomposition. We therefore combine perceptual reference-based assessment with conservative programmatic signals of structural correspondence. D.3 is also intentionally distinct from A.1 Geometry. A.1 primarily measures general geometric correspondence at the whole-object and component levels, asking whether the generated geometry broadly matches the intended object and occupies appropriate spatial regions. In contrast, D.3 focuses on finer-grained, design-specific correspondence that is difficult to capture with geometry-only rules, including whether characteristic components are represented, whether their relative arrangement and scale preserve the target design, and whether distinctive structural details remain recognizable. These properties often depend on semantic and perceptual interpretation rather than distance alone. Accordingly, D.3 uses VLM judgment as its primary signal, while programmatic measurements provide complementary geometric evidence rather than replacing perceptual assessment.

Perceptual structural alignment. For the perceptual component, the judges receive both the task condition image and the neutral gray clay rendering of the generated structure. Each judge evaluates four criteria: **inventory**, **placement**, **relative scale**, and **detail fidelity**. Inventory measures whether the major characteristic components of the target are represented. Placement measures whether these components occupy appropriate relative locations. Relative scale measures whether their sizes and proportions preserve the target organization. Detail fidelity measures whether distinctive geometric features and structural details remain recognizable in the generated design. Let

$$\mathcal{Q} = \{\text{Qwen, Gemma}\}, \quad \mathcal{C}_{\text{align}} = \{\text{inventory, placement, scale, detail}\},$$

and let $x_{q,c} \in 1, \dots, 10$ denote the score assigned by judge $q \in \mathcal{Q}$ to criterion $c \in \mathcal{C}_{\text{align}}$. The raw perceptual signal is

$$J_{3.3} = \frac{1}{8} \sum_{q \in \mathcal{Q}} \sum_{c \in \mathcal{C}_{\text{align}}} \frac{x_{q,c} - 1}{9}.$$

Programmatic structural alignment. We complement the perceptual assessment with up to two programmatic signals. These signals are intended as supporting evidence for structural correspondence and do not impose exact reconstruction of the reference.

The first signal measures part-level volumetric correspondence. The generated design and reference object are independently normalized for scale using their respective bounding-box diagonals and placed in the common normalized comparison frame. Each generated and reference part is then voxelized from surface samples followed by hole filling, using a voxel pitch equal to 1% of the corresponding object’s diagonal. For each reference part r , we identify the generated part p with maximum volumetric intersection-over-union. The resulting score is

$$C_{\text{part}} = \frac{\sum_r |r| \max_p \text{IoU}(r, p)}{\sum_r |r|},$$

where $|r|$ denotes the occupied volume of reference part r .

The matching is intentionally many-to-one rather than one-to-one. This allows alternative valid decompositions in which multiple reference segments are represented by a single generated component. Weighting by reference-part volume gives greater influence to major structural components while preventing very small decorative pieces from dominating the score. No additional rotation search is performed for this term, so correspondence is evaluated in the shared task-aligned coordinate frame.

The second programmatic signal evaluates consistency in global mirror symmetry when the corresponding reference symmetry measurement is available. For each object, occupied geometry is represented on a 1 cm voxel grid. We evaluate 36 vertical mirror planes passing through

System	Decomposition		Aesthetics		Alignment	
	Human	Ours	Human	Ours	Human	Ours
Claude Sonnet 5	92.9 (1)	84.9 (1)	76.8 (2)	79.6 (1)	80.4 (1)	74.4 (2)
GPT-6 Astra	83.9 (2)	80.0 (2)	85.7 (1)	72.0 (3)	78.6 (2)	78.9 (1)
PartPacker	69.6 (3)	59.4 (5)	60.7 (3)	78.1 (2)	60.7 (3)	71.4 (3)
Qwen3.5-27B	50.0 (5)	76.6 (3)	44.6 (4)	66.0 (4)	46.4 (4)	65.8 (4)
Gemma-4-31B	60.7 (4)	60.2 (4)	32.1 (5)	54.9 (5)	37.5 (5)	62.0 (5)
BrickGPT	3.6 (6)	13.0 (6)	23.2 (6)	12.2 (6)	8.9 (6)	35.8 (6)
System Spearman $\rho \uparrow$	0.77		0.83		0.94	
System MAE $\downarrow$	9.8		14.8		14.6	
Item Spearman $\rho \uparrow$	0.52		0.60		0.64	
Human-human ρ (ref.)	0.69		0.52		0.73	

Table 5: Alignment between calibrated Design (D) scores and human ratings on 7 held-out objects (6 systems, 42 designs per dimension; 4 annotators). Cells report system mean scores with ranks in parentheses. We report system-level Spearman ρ and MAE, item-level Spearman ρ , and human-human ρ as a reference for annotator agreement.

the object’s centroid, separated by 5° increments, and define the object’s symmetry value as the maximum IoU between its occupied cells and their reflected counterparts. Let $\sigma \in [0, 1]$ and $\sigma_{\text{ref}} \in [0, 1]$ denote the resulting symmetry values for the generated and reference structures, respectively. We define

$$C_{\text{sym}} = 1 - \frac{|\sigma - \sigma_{\text{ref}}|}{\max(\sigma_{\text{ref}}, 1 - \sigma_{\text{ref}})}.$$

Importantly, C_{sym} does not reward symmetry itself. It measures agreement with the symmetry characteristics of the target. A strongly asymmetric reference therefore does not require the generated artifact to become symmetric, whereas a highly symmetric reference provides evidence that comparable global organization is characteristic of the target design.

The raw computed signal is the mean of whichever programmatic terms are available:

$$C_{3.3} = \text{mean}(C_{\text{part}}, C_{\text{sym}})_{\text{available}}.$$

For tasks where reference geometry is available but no valid symmetry measurement is provided, C_{part} remains defined and constitutes the computed component on its own.

Calibration and aggregation. The perceptual signal $J_{3.3}$ and computed signal $C_{3.3}$ are independently calibrated to the human rating scale using the common Calibration-then-Evaluation procedure, producing $J' * 3.3$ and $C' * 3.3$. The calibration parameters and blending ratio are determined during the pre-declared calibration stage and remain fixed during benchmark evaluation. As established by this procedure, D.3 assigns a 3:1 perceptual-to-computed weighting:

$$D_{3.3} = \frac{3J'_{3.3} + C'_{3.3}}{4}.$$

This weighting reflects the role of D.3 as a fine-grained structural and design-alignment measure: perceptual judgment captures semantic organization and distinctive design features that are difficult to express through deterministic geometric rules alone, while the computed component provides complementary evidence of part-level correspondence and, where available, symmetry consistency. In this way, D.3 complements rather than duplicates A.1: A.1 measures broad geometric correspondence, whereas D.3 evaluates whether the characteristic structural identity of the specific target design is preserved.

Physical Realization: Evaluation Basis. The preceding metric levels evaluate whether a generated structure is geometrically sound, contains the functional components expected of the target object, and exhibits coherent design quality. Physical Realization asks a complementary question: whether the generated specification provides a plausible path from a digital assembly to an artifact that could be constructed and used. We evaluate three complementary aspects. **Assembly Sequence** examines

whether the parts can be inserted in the declared order while maintaining physically supportable intermediate states. **Material** evaluates whether declared material choices are compatible with the functional role of each component. **Operability** evaluates whether the completed structure supports the required functional subsystems, articulated motions, and human-use conditions.

These metrics therefore evaluate information contained in the generated assembly specification rather than only the final rendered geometry. Missing assembly instructions, missing materials, absent semantic roles, or unsupported articulation may reduce realization performance even when the final shape appears visually plausible.

Physical Realization – R.1 Assembly Sequence

A physically plausible final geometry is not necessarily assemblable. A component may be trapped by previously placed geometry, an intermediate subassembly may remain unsupported or become unstable before later components are added, or the declared construction sequence may omit parts that appear in the final design. Assembly Sequence therefore evaluates both whether the declared ordering admits physically feasible insertion operations and whether the resulting intermediate configurations remain supportable throughout construction. Let the declared sequence contain m placement operations,

$$\{(p_k, d_k)\}_{k=1}^m,$$

where p_k denotes the component placed at step k and d_k its declared insertion direction. Let n denote the total number of parts in the final generated design, including any parts omitted from the declared sequence. A design for which no usable assembly sequence is provided receives a score of zero. After each placement, the currently assembled structure is partitioned into connected subassemblies according to the shared **physical-connectivity** relation defined above, which recognizes geometrically interlocked parts, valid fastening relations, and valid brick–stud connections. Mere geometric contact alone is therefore insufficient to merge two parts into the same subassembly. Each step is then evaluated along two dimensions: **insertability**, which tests whether the newly introduced component can reach its final pose without passing through previously assembled geometry, and **intermediate support**, which tests whether the resulting configuration remains physically grounded and, where applicable, stable against tipping.

Insertability. For component p_k , the declared direction d_k is evaluated first as the preferred insertion hypothesis. We additionally test the fixed canonical directions

$$-z, \pm x, \pm y, +z$$

as deterministic fallbacks. Thus, the metric evaluates whether the declared assembly order admits a physically plausible insertion path, rather than requiring exact agreement with a single insertion-direction annotation. For each candidate direction, the component is swept from outside the current object extent toward its final pose in discrete odd-cell increments over a search distance extending to the object bounding-box diagonal plus 5 cm. Let O_t denote its voxel overlap with previously placed geometry at an intermediate sweep position and O_{final} the corresponding overlap at the final pose. The insertion path is accepted only when

$$O_t \leq O_{\text{final}} + 2$$

throughout the sweep. This relative criterion tolerates small discretization or interface overlap already present in the intended final configuration while rejecting paths that require the component to pass substantially through existing geometry. Because the supporting floor is not represented as collision geometry, upward insertion directions with vertical component $d_z > 0.3$ are excluded when the component’s final pose lies above the ground-contact band. This prevents artificial assembly paths in which a component would effectively enter the structure from below the supporting surface. The first component in the sequence is considered insertable by definition.

Intermediate support and stability. After a successful insertion test, the connected subassembly containing the newly placed component must reach the common ground-contact band,

$$z_{\min} \leq z_0 + \beta,$$

where z_0 and β follow the same ground-contact convention used in S.3. For free-standing tasks, each intermediate subassembly must additionally satisfy positive quasi-static tipping stability. Let X_k denote the set of connected subassemblies present after step k , let θ_B denote the critical tilt angle of subassembly $B \in X_k$, and let θ^* denote the task-specific stability target defined in S.3. We define the intermediate-state margin as

$$s_k = \min_{B \in X_k} \min \left(1, \frac{\theta_B}{\theta^*} \right).$$

The minimum over subassemblies reflects the fact that a single unsupported or unstable intermediate component is sufficient to compromise the current assembly state. If step k is not executable, or if any resulting subassembly is floating or self-tipping, we set

$$s_k = 0.$$

For tasks whose references are not free-standing, the stability requirement reduces to binary grounding, and therefore $s_k \in \{0, 1\}$. A component associated with a failed step remains in the simulated assembly, and later declared steps are still evaluated. This is intentional: the executable-prefix term described below records the first break in sequential feasibility, whereas the intermediate-state term retains information about whether subsequent portions of the declared construction trajectory remain locally plausible.

Aggregation. Let k^* denote the number of consecutive assembly steps successfully completed before the first failure. The executable-prefix score is

$$E = \frac{k^*}{n}.$$

The intermediate-state score is

$$S = \frac{1}{n} \sum_{k=1}^m s_k.$$

Both terms are normalized by the total number of parts n in the generated design rather than by the number of declared steps m . Consequently, omitting difficult components from the assembly sequence cannot improve the score.

The final Assembly Sequence metric is

$$R_{4.1} = \frac{1}{2}E + \frac{1}{2}S.$$

The two terms capture complementary aspects of constructability: E measures how much of the intended artifact can be assembled sequentially before the first infeasible operation, while S measures the quality and supportability of the intermediate configurations encountered across the declared construction process.

Physical Realization – R.2 Material

Physical realization additionally requires material assignments that are compatible with the structural and functional role of each component. Because real products frequently admit multiple valid material choices, Material does not evaluate exact agreement with the material of a single reference model. Instead, for each component we construct a set of acceptable material classes from multiple independent evidence sources and evaluate whether the generated assignment is compatible with that set. This formulation is intentionally permissive with respect to legitimate material variation while remaining restrictive toward assignments that are unsupported by either documented usage, observed population statistics, reference correspondence, or functional plausibility. Our material library contains 23 materials organized into nine classes. For compatibility reasoning, related classes are additionally grouped into broader families, including metal, polymer, soft material, wood, glass, and ceramic. For a generated part i with canonical semantic role r_i , the accepted material set is

$$\mathcal{A}_i = W(r_i) \cup P(r_i) \cup R(i),$$

where W , P , and R denote textual evidence, population statistics, and reference-overlap evidence, respectively. We use the union because the three sources are complementary and individually incomplete: a material may be well supported by one source while being absent from another simply because that source does not exhaust all valid realizations. Requiring agreement across sources would therefore be unnecessarily restrictive. If this union is empty, we instead use the functional material judgment

$$\mathcal{A}_i = J(r_i).$$

The first source, $W(r_i)$, is derived from verified material claims in the cited Wikipedia sources. We retain only claims that explicitly name the corresponding component. Per-part material extraction is performed from the cited text, and an extracted answer is accepted only when accompanied by a verbatim supporting quotation containing each relevant material expression. This requirement makes the textual evidence auditable and prevents unsupported material inference. Material claims associated with clearly historical usage, such as descriptions explicitly marked by historical years or terms such as ‘originally’ or ‘traditionally,’ are excluded so that the metric better reflects plausible contemporary construction rather than historically possible but currently atypical realizations. The second source, $P(r_i)$, is a population prior estimated from Artiverse instances in the corresponding object category. A material class is included when it accounts for at least 10% of the observed class mentions for the role, provided that the role has at least five observations. The 10% threshold is intended to retain recurring alternatives while excluding isolated or exceptionally rare assignments, and the minimum observation count avoids drawing population-level conclusions from insufficient evidence. This permits multiple commonly observed materials to be treated as valid rather than forcing the design toward a single modal material. The third source, $R(i)$, transfers material evidence from geometrically corresponding reference components. The generated and reference objects are independently normalized by their diagonals and voxelized at a pitch equal to 2% of the corresponding diagonal. For every reference component that covers at least 25% of the generated part’s occupied cells, we add the reference component’s interior and exterior material classes together with the population prior associated with its semantic role. The overlap threshold requires substantial geometric correspondence before material evidence is transferred, reducing the chance that incidental contact or weak spatial proximity incorrectly determines the acceptable material set. When none of these three sources provides an applicable class, we use $J(r_i)$, a fixed task-and-part functional material table. This fallback represents material classes that are functionally plausible for the corresponding component role. It is used only when evidence from text, population statistics, and reference correspondence is unavailable, thereby providing coverage without allowing the fallback to override stronger task-specific evidence. Let c_i denote the library material class assigned to generated part i . Its per-part compatibility score is

$$a_i = \begin{cases} 1, & c_i \in \mathcal{A}_i, \\ 0.5, & c_i \text{ belongs to the same material family as a class in } \mathcal{A}_i, \\ 0, & \text{otherwise.} \end{cases}$$

A part with no declared library material receives zero whenever an evaluation basis exists. Exact class agreement therefore receives full credit, while a materially related choice receives partial credit. The family-level score of 0.5 should be interpreted as coarse compatibility rather than equivalence: it acknowledges that materials within the same broad family may share relevant physical characteristics while still distinguishing them from explicitly supported classes. For example, two metal subclasses may be treated as more similar than a metal and a textile even when the exact preferred class differs. The final Material score is

$$R_{4.2} = \frac{1}{|\{i : \mathcal{A}_i \neq \emptyset\}|} \sum_{i: \mathcal{A}_i \neq \emptyset} a_i.$$

Parts for which no evaluation basis can be established are excluded rather than automatically treated as incorrect. This conservative treatment avoids penalizing the generator when the

benchmark itself lacks sufficient evidence to determine material suitability. If no part has an available material basis, the metric is considered not applicable. Conversely, a generated design in which no part declares a material from the supported library receives a score of zero. Overall, this evidence-based formulation is designed to balance two competing requirements: it accommodates genuine material diversity across real products while still rejecting choices that are incompatible with the structural or functional role of the corresponding component. The numerical thresholds used for evidence inclusion and partial compatibility are fixed benchmark-wide heuristic choices, selected to provide stable and interpretable behavior across heterogeneous object categories rather than to reproduce any single reference construction exactly.

Physical Realization – R.3 Operability

A completed structure may be geometrically plausible and physically assemblable while still failing to support its intended use. Operability therefore evaluates whether the generated artifact can realize the task-level capabilities associated with the corresponding real object. Rather than treating functionality as a single visual or geometric property, each capability is represented as an explicit combination of required components, structural relationships, articulated behavior, and, where applicable, human-use constraints. R.3 consequently evaluates whether these individually necessary conditions are integrated into a structure that can plausibly operate as intended.

Capability specification. For each task, we construct a fixed capability sheet from the curated functional and kinematic claims together with a pre-declared human-use layer. The capability specification is task-level and system-independent: it is defined independently of the generated outputs and applied uniformly to all evaluated systems. Capabilities draw from four sources. Functional capabilities, denoted F , correspond to required functional subsystems of the object. Motion capabilities, denoted M , represent articulated behaviors required by the task. Human-use capabilities, denoted H , capture geometric requirements for interaction, including reachability, clearance, graspability, and usable support surfaces. Finally, P contains required components not already represented by a functional subsystem. A functional subsystem may consist of multiple required links. Each named component must be present, and explicitly specified component relationships must be supported by a contact or joint path of at most four hops. When the subsystem includes a control component, the control must additionally be reachable from the estimated user position and connected through a path of at most six hops, containing an appropriate moving joint, to as many as three associated output components. The slightly longer control path accommodates mechanisms in which user input is transmitted through intermediate linkage before reaching the functional output.

Component and structural requirements. Component presence receives score 1 when a generated role matches the required component, including accepted lexical relations and components rigidly fixed to the corresponding matched part, and 0 otherwise. For a required relationship between two components, a structural path receives full credit when an appropriate path exists through a moving joint whose moving set contains the required endpoint component. When the required components are structurally connected but the expected articulation is absent, the relationship receives partial credit of $1/2$. If no valid structural path exists, the score is 0. This distinction separates structural presence from functional articulation: connectivity alone provides evidence of partial realization, but full credit requires the motion mechanism needed by the capability.

Human-use geometry. Human-use terms are treated as standardized geometric proxies for practical interaction rather than as universal ergonomic standards. Their purpose is to provide consistent benchmark-wide tests of whether controls and usable regions occupy plausible interaction spaces. For reachability, let d denote the Euclidean distance between the relevant control and the estimated eye position. For seated objects, the eye point is placed 0.60 m above the seat top; otherwise, a standing eye point is placed 1.40 m above ground. We define

$$s_{\text{reach}} = \max\left(0, 1 - \frac{\max(0, d - 0.70)}{1.4}\right).$$

Controls within 0.70 m therefore receive full credit, after which the score decreases continuously with distance rather than introducing a brittle binary reachability boundary. Clearance is evaluated using task-specific median human occupancy boxes sampled on a 10^3 point grid. Let f_{free} denote the fraction of the corresponding occupancy volume that remains unobstructed. We define

$$s_{\text{clear}} = \min \left(1, \frac{f_{\text{free}}}{0.9} \right).$$

Thus, clearance is evaluated volumetrically rather than through isolated collision tests. A component expected to be graspable receives full grip credit when its two smallest principal extents lie between 15 mm and 70 mm. A required usable surface receives full credit when its area is at least

$$0.05 \text{ m}^2$$

and, where a task-specific expected height interval is available, the surface lies within the corresponding band. For components expected to be spatially co-located, let g denote their geometric gap. We define

$$s_{\text{coloc}} = \text{clip} \left(1 - \frac{g - 0.10}{0.20}, 0, 1 \right).$$

Because these human-use rules are intentionally general geometric proxies, we avoid imposing them when they are inconsistent with the task itself. A human-geometry link is disabled whenever the corresponding benchmark reference receives a score below 0.5 on that criterion. For tasks without reference-based human calibration, such links are disabled except for ground contact. This reference gating prevents a generic anthropometric heuristic from penalizing generated designs for failing a condition that is not satisfied by the corresponding real reference.

Dynamic motion feasibility. Motion capabilities are evaluated through rigid-body simulation in MuJoCo. The simulation is intended as a standardized test of geometric and articulated feasibility rather than a high-fidelity reconstruction of material-specific real-world dynamics. All systems are evaluated under the same simulation assumptions.

Simulation uses a 1 ms timestep and the implicit-fast integrator. Parts connected by physical contact or fixed joints are grouped into rigid bodies and represented using convex-hull collision geometry. A grounded wheel attached through an approximately horizontal and centered hinge is instead represented using cylindrical collision geometry to better approximate rolling behavior. Rigid-body masses are constructed from the S.3 geometric volumes using the common density

$$1000 \text{ kg/m}^3,$$

and friction coefficients are fixed to

$$(1, 0.02, 0.002).$$

The uniform density is used as a standardized inertial proxy rather than an estimate of the true material density of each generated component. This isolates motion feasibility from errors in material prediction and provides comparable simulation conditions across generated artifacts. Before commanded actuation, the completed artifact is allowed to settle for 0.6 s. If the object tilts by at least 20° or translates by at least 0.3 times its object diagonal during this period, all motion capabilities receive zero. Such motion indicates that the completed artifact is insufficiently stable for subsequent actuation to provide a meaningful test of intended functionality. For articulated motion claims, candidate joints are driven by a proportional-derivative servo toward their declared limits. We use the standardized mass- and scale-dependent proportional gain

$$k_p = 100MgL,$$

where M and L denote the moving mass and characteristic length, respectively. This scaling provides comparable actuation strength across structures of different sizes; it is used as a simulation control heuristic rather than as a model of a specific physical actuator. When a generated

joint does not provide an explicit motion limit, the target is defined as $\pm 90^\circ$ for a revolute joint, $\pm 360^\circ$ for a continuous joint, or $\pm$ half of the corresponding moving-part extent for a prismatic joint. Actuation uses a 1.5 s ramp followed by a 0.4 s holding period.

Let Δq denote the achieved generalized displacement and q_{target} the commanded displacement. We define

$$s_{\text{motion}} = \min \left(1, \frac{|\Delta q|}{|q_{\text{target}}|} \right) \mathbf{1}[\text{tilt} < 15^\circ].$$

The displacement ratio measures how much of the commanded motion is physically realized. The additional tilt condition prevents a mechanism from receiving motion credit when the apparent displacement is achieved by destabilizing the entire artifact rather than by successful articulation. The actuation stability threshold is intentionally stricter than the initial settling threshold because successful operation requires the artifact to remain stable while the mechanism is being used. The better of the two actuation directions is retained. For each claimed motion, we consider up to the first four candidate joints on the structural path between the moving component and the corresponding relative component.

Rolling capabilities. Rolling capabilities associated with wheels, casters, or tracks are tested separately because the existence of a rotational joint alone does not establish that the completed object can translate through rolling. The corresponding wheel joints are left free and a horizontal force equal to 10% of the object’s weight is applied for 1.5 s. If d_{travel} denotes the resulting translation, rolling receives

$$s_{\text{roll}} = \min \left(1, \frac{d_{\text{travel}}}{0.276 \text{ m}} \right) \mathbf{1}[\text{tilt} < 15^\circ].$$

The fixed forcing and normalization conditions provide a common benchmark-wide test of whether the assembled wheel or track mechanism can produce meaningful translation without destabilizing the object.

Capability aggregation. Each task-level capability c consists of a set of active requirement links $\mathcal{L}_c$. Because these links represent jointly necessary conditions for the capability to be usable, capability performance is defined by its weakest active requirement:

$$w_c = \min_{\ell \in \mathcal{L}_c} s_\ell.$$

The weakest-link formulation is intentional. Averaging the requirements could assign substantial credit to a capability whose components are present but disconnected, whose mechanism cannot move, or whose control cannot be reached. In each of these cases, the capability as a whole is not operational despite success on its remaining conditions. Let $\mathcal{C}$ denote the set of applicable task-level capabilities. The final Operability score is

$$R_{4.3} = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} w_c.$$

R.3 therefore evaluates functionality compositionally rather than through isolated part or joint checks. It asks whether the generated artifact integrates the required components, structural relationships, articulated mechanisms, and human-use geometry into capabilities that can plausibly function as intended. The geometric and simulation thresholds used in these tests are fixed benchmark-wide evaluation parameters and are applied identically to all systems; they are intended as consistent comparative proxies for operability rather than claims of exact real-world physical behavior.

A.5 INTERPRETATION OF EVALUATION METRICS AND RESULTS

Stable Metrics Across Object Sub-samples. Because several of our evaluation metrics involve design choices and hyperparameters, we further examine whether they capture underlying system capabilities rather than being overly sensitive to the particular objects included in the benchmark. To validate this, we draw two disjoint stratified 50-object samples from the 200-object core set and

Table 6: Level scores, computed as the mean of the three metrics within each level with N/A metrics excluded, are reported on the full 200-object core set and on two disjoint stratified 50-object samples. All three columns are derived from the same evaluation episodes. Their close agreement across subsets demonstrates strong consistency across object types, suggesting that LMBuild primarily captures intrinsic differences in model capability rather than artifacts of a particular object sample.

Models	Full core (200)				Sample A (50)				Sample B (50)			
	S	A	D	R	S	A	D	R	S	A	D	R
State-of-the-art Closed-Source APIs												
GPT-6 Astra	88.5	50.0	74.0	53.5	86.4	50.6	72.5	55.7	91.5	48.2	73.1	52.0
GPT-5.6 Sol	88.8	41.9	71.5	57.4	88.0	43.8	70.4	56.8	89.5	41.1	72.9	57.1
Claude Opus 5	88.8	46.6	71.3	55.1	88.0	48.9	71.5	57.7	87.4	43.3	72.6	50.9
Claude Fable 5.1	89.1	47.3	71.8	55.3	86.1	49.7	70.9	56.0	85.7	42.7	72.5	53.7
Claude Sonnet 5	86.1	41.5	66.3	56.2	86.0	43.3	66.1	58.7	86.8	37.5	66.9	53.7
Claude Haiku 4.5	80.8	30.5	57.8	51.3	82.0	29.2	57.9	50.3	81.1	29.8	58.6	51.1
Open-Source (Multi-modal) Large Language Models												
Qwen3.5-27B	82.9	28.3	52.7	57.2	84.6	28.2	53.1	60.1	84.2	27.3	53.2	55.2
Qwen3.5-35B-A3B	72.1	24.0	47.0	44.9	69.2	23.6	45.8	44.1	78.7	25.4	48.9	47.4
GPT-OSS-120B	78.5	25.8	49.3	51.7	79.6	25.4	48.8	51.1	76.4	24.0	49.8	50.7
Gemma-4-31B-IT	59.7	20.1	41.2	36.7	61.6	21.1	43.2	34.3	63.5	22.2	43.1	39.7
Minstral-3-8B	65.7	18.8	46.8	39.9	67.5	18.9	46.7	40.1	67.3	19.5	49.6	37.0
InternVL3.5-38B	49.3	11.4	34.8	21.5	50.2	9.6	33.8	17.2	45.7	12.9	36.7	22.6
Qwen3-VL-30B-A3B	48.7	13.8	32.1	33.0	51.9	15.0	36.9	37.1	50.2	13.0	35.0	30.2
MiniCPM-V-4.5	40.0	9.5	27.1	18.6	45.2	10.4	27.8	19.6	38.7	8.9	25.1	19.5
Qwen3-4B-Instruct	65.5	15.6	30.3	38.8	67.8	13.8	30.3	36.6	66.1	18.5	34.0	40.7
Qwen3-VL-32B	48.5	13.5	30.2	31.8	43.0	11.6	27.3	27.9	49.6	14.3	32.8	31.1
ERNIE-4.5-VL-28B-A3B	20.0	4.4	15.0	9.6	22.7	4.9	15.0	8.1	20.6	6.3	17.2	10.4
InternVL3.5-8B	39.1	7.4	26.2	18.1	36.1	6.5	22.3	17.2	45.2	9.0	29.0	18.2
Qwen3-VL-8B	27.9	6.9	15.1	17.8	31.8	8.9	18.0	22.8	19.5	4.0	10.6	11.3
State-of-the-art Domain-Specific Models												
BrickGPT	94.7	34.5	24.2	N/A	94.3	34.5	24.2	N/A	94.9	40.0	23.4	N/A
LegoACE	78.0	29.6	27.7	N/A	79.1	28.8	28.4	N/A	81.6	31.5	25.9	N/A
PartCrafter-15	61.3	32.2	57.5	N/A	62.6	33.1	57.1	N/A	65.6	36.5	60.4	N/A
PartCrafter-8	57.1	28.2	53.0	N/A	58.1	27.7	52.8	N/A	60.5	32.8	56.4	N/A
PartPacker	75.1	36.4	61.3	N/A	75.7	34.9	61.3	N/A	79.7	40.8	63.3	N/A
Cube3D+CubePart	69.4	28.7	56.5	13.4	70.4	29.9	55.2	16.6	70.8	30.4	58.2	13.5
PartCrafter15 [†] (P)	61.2	18.0	37.4	N/A	61.8	18.4	37.7	N/A	65.8	19.9	40.1	N/A
PartCrafter8 [†] (P)	60.2	17.3	43.1	N/A	61.8	17.5	42.9	N/A	63.5	18.4	44.3	N/A
PartPacker [†] (P)	77.9	21.7	55.0	N/A	77.0	20.7	55.3	N/A	81.1	24.7	54.3	N/A
Cube3D [†] (P)	79.8	17.6	61.0	N/A	82.7	19.6	61.5	N/A	81.7	19.1	62.6	N/A
PhysX-Anything	63.7	11.3	44.7	31.6	67.1	11.7	43.8	34.3	62.8	10.6	46.1	30.5

compare the resulting scores with each other and with those obtained on the full set. Table 6 reports the level scores for the full core set and the two sub-samples. Aggregate performance remains highly stable: closed-source APIs score 63.4 on the full set, 63.6 on sample A, and 62.5 on sample B, while open-source models score 33.9, 34.1, and 34.4, respectively. At the individual-system level, the mean absolute deviation from the full-set score is only 1.0 point for sample A and 1.7 points for sample B. Table 7 further shows that this consistency holds at the metric level. Across the 30 systems, the two disjoint samples achieve Pearson correlations of $r = 0.937\text{--}0.988$ and Spearman correlations of $\rho = 0.86\text{--}0.99$. Across 1,000 repeated stratified resamplings, the mean correlation between disjoint sample pairs remains between 0.953 and 0.980, with all 95% intervals at or above 0.91. These results indicate that the evaluation metrics produce highly consistent relative system performance across substantially different object subsets, suggesting that LMBuild primarily measures persistent differences in model capability rather than artifacts of a particular object sample. The full 200-object evaluation therefore provides the most stable estimate of overall system performance. The full results in Table 2 also reveal clear differences across system families. Closed-source APIs achieve the strongest overall performance across the four levels: GPT-6 Astra reaches 88.5 / 50.0 / 74.0 / 53.5 on Soundness / Affordance / Design / Realization, with Claude Opus 5 and Claude

Table 7: Agreement of system-level scores across task sets: Pearson r (Spearman ρ in parentheses) over the 30 systems’ means. A vs. B compares two disjoint 50-object samples; the last two columns repeat the stratified draw 1,000 times (mean, 95% interval in brackets). Sample-vs-full correlations are an upper bound because each sample is part of the full set.

Metric	A vs. 200	B vs. 200	A vs. B	1,000 resamples	
				50 vs. 200	disjoint pairs
S.1	0.982 (0.97)	0.986 (0.97)	0.960 (0.95)	0.987 [0.98, 0.99]	0.967 [0.94, 0.98]
S.2	0.990 (0.96)	0.995 (0.98)	0.981 (0.96)	0.989 [0.98, 1.00]	0.972 [0.95, 0.99]
S.3	0.991 (0.99)	0.978 (0.97)	0.960 (0.96)	0.983 [0.97, 0.99]	0.957 [0.93, 0.98]
A.1	0.994 (0.99)	0.981 (0.98)	0.967 (0.96)	0.990 [0.98, 1.00]	0.975 [0.95, 0.99]
A.2	0.991 (0.98)	0.986 (0.99)	0.966 (0.95)	0.991 [0.98, 1.00]	0.977 [0.96, 0.99]
A.3	0.992 (0.94)	0.995 (0.98)	0.987 (0.92)	0.992 [0.99, 1.00]	0.980 [0.96, 0.99]
D.1	0.992 (0.99)	0.995 (0.99)	0.988 (0.99)	0.991 [0.98, 1.00]	0.977 [0.96, 0.99]
D.2	0.993 (0.98)	0.989 (0.98)	0.975 (0.97)	0.988 [0.98, 0.99]	0.970 [0.95, 0.98]
D.3	0.994 (1.00)	0.990 (0.98)	0.979 (0.97)	0.990 [0.98, 1.00]	0.975 [0.96, 0.99]
R.1	0.984 (0.96)	0.988 (0.98)	0.959 (0.94)	0.989 [0.98, 1.00]	0.970 [0.94, 0.99]
R.2	0.982 (0.95)	0.979 (0.95)	0.937 (0.86)	0.982 [0.96, 0.99]	0.953 [0.91, 0.98]
R.3	0.987 (0.95)	0.994 (0.98)	0.980 (0.93)	0.987 [0.98, 0.99]	0.967 [0.94, 0.98]

Table 8: Failure modes of the six closed-source APIs on the 200-object core set (% of tasks; thresholds in parentheses). All models submit a design on every task.

Level	Failure mode	GPT-6	GPT-5.6	Opus 5	Fable 5.1	Sonnet 5	Haiku 4.5
Soundness	Disconnected assembly (S.1)	21.0	13.0	6.5	6.5	19.5	38.0
	Interpenetrating parts (S.2)	13.0	16.5	4.0	3.0	15.5	38.5
	Unstable under gravity (S.3 < 0.5)	8.5	10.5	10.0	6.5	12.5	24.0
Affordance	Required parts missing or misplaced (A.2 < 0.5)	54.5	68.5	59.0	56.0	68.5	79.0
	A required moving part is never built (A.3)	56.5	61.5	51.0	49.5	57.5	65.5
	Part built but no joint moves it (A.3)	24.5	27.5	17.5	30.5	29.0	34.5
	Joint present but wrong type/axis/range (A.3)	36.0	38.0	44.0	39.5	33.5	37.0
Design	Part count off by > 2× vs. reference (D.1)	23.0	31.0	31.0	28.5	31.0	37.5
	Low fidelity to the photo (D.3 < 0.4)	1.0	8.5	5.0	4.5	13.0	19.5
Realization	Assembly sequence not fully executable (R.1)	60.0	51.0	67.5	50.5	48.0	57.5
	No functional capability works (R.3)	35.5	34.0	30.0	32.5	40.0	51.0
Tool use	At least one failed <code>create_part</code> call	15.5	16.5	27.5	8.5	11.5	10.5

Fable 5.1 following closely. The strongest open model, Qwen3.5-72B, is competitive on Soundness and Realization (82.9 and 57.2) but remains substantially lower on Affordance (28.3) and Design (52.7). Domain-specific generators show the widest variation across dimensions, performing well when their output representation directly enforces particular structural properties but less consistently on tasks that require richer part decomposition and functional behavior.

Interpreting the Scores Where a Closed-source API Does Not Lead. For a small number of metrics, we observe that another system matches or exceeds GPT-6 Astra. These isolated results, however, should not be interpreted as evidence that the corresponding system is a stronger general-purpose builder. Each metric captures only one aspect of a generated structure, and high performance on a particular dimension may arise from properties enforced by the system’s representation, from producing simpler structures, or from optimizing visual appearance at the expense of functionality. We therefore examine these cases individually and explain why such scores can be misleading when interpreted in isolation.

Soundness of Brick Generators. BrickGPT and LegoACE achieve the strongest results on the connectivity- and collision-related Soundness metrics, outperforming all API-based systems on these dimensions. This advantage is largely explained by their representations and decoding procedures rather than by broader design capability. Both systems generate structures on a discrete stud lattice, and BrickGPT further rejects placements that would collide with existing bricks and

rolls back unstable construction steps. As a result, connectivity and collision avoidance are strongly enforced by construction. However, the same systems perform substantially worse on metrics that assess whether the generated structure corresponds to the requested object and contains the appropriate functional components. Their Design and Affordance scores remain comparatively low, indicating that a structure can be physically coherent without faithfully realizing the intended object or its functionality. These results therefore illustrate that high Soundness is necessary, but not sufficient, for successful structure generation, and it is not surprising that several systems could ‘outperform’ the best LLM-based solutions.

Soundness of Part-based Mesh Generators. A related effect appears among part-based mesh generators. Particulate and Cube3D+CubePart achieve strong scores on selected Soundness metrics because their generation procedures naturally encourage geometric compatibility between parts. Particulate first generates a complete mesh and subsequently segments it into components, meaning that the resulting parts inherit a shared surface and are unlikely to interpenetrate. CubePart similarly generates parts within a common object volume, which encourages contact between adjacent components. This geometric consistency, however, does not necessarily imply structural robustness. For example, PartCrafter performs substantially worse on load-bearing stability, suggesting that its components are not always connected in mechanically meaningful ways. These results distinguish geometric compatibility from structural reasoning: parts may fit together spatially without being designed to support one another under physical constraints.

Soundness and realization of simpler designs. Several open and smaller general-purpose models also match or outperform GPT-6 Astra on selected Soundness and Realization metrics, particularly those related to supported connections, assembly executability, and material plausibility. A key factor is that these systems tend to produce considerably smaller assemblies. For example, Qwen3.5-27B and GPT-OSS-120B generate designs with substantially fewer parts than GPT-6 Astra and Claude Opus 5. This difference directly affects metrics such as S.1, R.1, and R.2, all of which become easier to satisfy as assembly complexity decreases. Fewer parts imply fewer interfaces at which unsupported or incorrect connections can occur, shorter assembly sequences reduce the number of ordering dependencies, and a smaller set of generic components simplifies material assignment. The corresponding tradeoff appears on metrics that evaluate completeness, decomposition quality, and functional structure. Systems that produce simpler assemblies tend to omit required components and mechanisms more frequently, resulting in lower Affordance and Design-fidelity scores. This pattern suggests that strong performance on local structural or realization criteria can partly reflect reduced design complexity rather than stronger end-to-end generation capability. Moreover, many failures captured by these metrics are locally repairable: a geometric gap can often be closed, a fastener added, or a material reassigned without changing the overall design. In contrast, a missing mechanism, incomplete decomposition, or incorrect functional structure usually requires more fundamental redesign. We therefore interpret Soundness and low-level Realization metrics as important necessary conditions, while Affordance, Design fidelity, and Operability provide stronger evidence of whether a model can generate the object as a complete functional system.

Effective-part generation of Cube3D+CubePart. Cube3D+CubePart performs notably better than other mesh-based generators on effective-part generation and also surpasses several open general-purpose models. This result should be interpreted in the context of its input representation. CubePart receives an explicit part schema, and in our evaluation protocol this schema corresponds to the set of components required by the task. The system therefore begins with substantial information about which parts should be present, rather than having to infer the decomposition entirely from the conditioning input. Our metric still requires those components to occupy appropriate spatial locations, so the system does not receive full credit merely for reproducing the supplied labels. Nevertheless, part of its advantage reflects access to the expected component inventory. Its simulation results further show that the presence of correctly named parts does not necessarily translate into functional articulation, as the generated components rarely produce meaningful motion.

Aesthetic quality of mesh generators. Mesh-based generators show a different form of specialization on the aesthetic metric. Cube3D and related systems produce smooth, coherent, and visually refined surfaces and consequently perform very well on reference-free aesthetic evaluation. This behavior is consistent with their training on large collections of 3D assets, where generating a polished and watertight surface is a central objective. However, aesthetic quality alone does not establish that the generated object matches the requested instance. A text-conditioned system may generate a visually convincing generic member of a category while missing the specific structural configura-

tion required by the task. This distinction is reflected in the gap between reference-free aesthetics and reference-conditioned Design fidelity. Conversely, image-to-3D reconstruction methods such as PartPacker achieve stronger visual fidelity to the target object but still lack the articulation and functional decomposition required for high Affordance and Operability. Aesthetic quality should therefore be interpreted together with fidelity and functionality rather than as an independent indicator of overall design capability.

Material generation of PhysX-Anything. PhysX-Anything is distinctive among the specialized generators because it explicitly predicts a material for each generated component. As a result, it achieves meaningful performance on material plausibility, unlike most domain-specific baselines. This result demonstrates a useful capability, but it should not be conflated with end-to-end product realizability. The system still produces relatively coarse part decompositions, and its generated objects exhibit almost no successful functional operation in simulation. Material plausibility therefore captures one aspect of realization quality, but does not by itself indicate that the generated structure is complete, mechanically functional, or manufacturable as a coherent product.

Overall interpretation. Taken together, these cases reveal three recurring reasons why a specialized or open system may match or exceed the strongest closed-source API on an individual metric. First, some properties are guaranteed or strongly encouraged by the output representation itself, as in lattice-based brick generation or mesh segmentation. Second, some metrics become easier to satisfy when a system generates smaller and less complex assemblies. Third, systems optimized for visual quality can perform well on appearance-based criteria without necessarily reproducing the correct object structure or functionality. These observations motivate interpreting LMBuild as a multidimensional benchmark rather than comparing systems through isolated metric maxima. Soundness, material plausibility, and aesthetic quality capture important properties of a generated structure, but none alone establishes that the system has identified the correct components, organized them into the intended design, and produced the required functionality. Across the metrics that more directly require these capabilities, including effective-part generation, functional decomposition, reference fidelity, and operability, the closed-source APIs remain strongest overall. The consistency of the level-level results across object sub-samples further suggests that these differences reflect persistent variation in end to end model capability rather than advantages tied to a small set of objects or individual metrics.

Errors of state-of-the-art models. Table 8 summarizes the dominant failure modes of the six closed-source APIs. All systems produce a design for every task, indicating that their primary limitations are functional rather than procedural. Physical defects do occur, but they are generally secondary: between 6.5% and 21% of generated designs contain at least one disconnected part, while 3%–16.5% contain interpenetrating parts, with Claude Haiku 4.5 representing a notable outlier at 38% and 38.5%, respectively. Most of these defects are small gaps or overlaps, particularly on LEGO and product-CAD objects, and could plausibly be corrected through local geometric adjustments. The more consequential failures arise from incomplete functional decomposition. A.3 is the weakest Affordance metric across these systems, with an average score of 37.9. Among failed motion targets, 57% fail because the required moving component, such as a lid, steering link, or handle, is never generated; a further 17% include the relevant part but lack a joint capable of moving it. The remaining failures are associated with incorrect joint type, axis, range, or offset. A similar pattern appears in A.2: in 54.5%–68.5% of designs, at least half of the required components are missing or incorrectly positioned, with many omissions corresponding to internal parts that are not directly visible from the exterior. These decomposition failures propagate directly to functional execution. R.3, which evaluates each declared capability according to its weakest required element, averages only 18.3 across the six systems. Nearly every design omits at least one component referenced by its functional description, such as a timer, thermal sensor, or lever. As a result, no evaluated capability succeeds at all on roughly 30%–40% of tasks, increasing to 51% for Claude Haiku 4.5. Assembly planning introduces an additional source of error: between 48% and 68% of generated assembly sequences are not fully executable. The first failure is often a floating component, such as a main body placed before the legs or supports required to hold it, even when the final geometry could in principle be assembled under a different ordering. In contrast, low-level tool interaction is rarely the limiting factor. Only 2%–7% of `create-part` calls fail, and the models typically recover from these failures within the same episode. Performance also varies systematically across object sources. Product-level CAD and Fusion 360 objects are the most difficult, with 81% and 73% of designs, respectively, missing or misplacing required components, compared with 48% for Artiverse furniture

and appliances. Qualitative examples reinforce the same pattern: Claude Opus 5 can generate a toaster with a recognizable bread slot while omitting the lever, timer, thermal sensor, and ejection mechanism; it can produce a toilet with a plausible seat hinge but no lid or flush handle; and GPT-6 Astra can generate an off-road vehicle with rotating wheels while failing to provide articulated steering links. Taken together, these results suggest that current frontier models are comparatively effective at producing recognizable external geometry, but remain substantially weaker at recovering the internal component structure, articulation, and mechanisms that make an object function. This gap is precisely what the Affordance and Realization levels of LMBuild are intended to capture.

A.6 LIMITATIONS AND FUTURE WORK

Evaluation scope and interpretation. LMBuild is designed as a standardized framework for comparing generated structures, rather than as a certification procedure for real-world engineering readiness. Its metrics operationalize buildability and functionality through reproducible geometric, semantic, kinematic, and simulation-based tests. These tests provide substantially stronger evidence than visual plausibility alone, but they do not establish that a generated artifact can be fabricated, assembled, or operated safely in every physical setting. In particular, the current evaluation does not model manufacturing tolerances, fastener strength, elastic deformation, fracture, fatigue, tool access, wiring or fluid routing, actuator limits, or sustained loading. LMBuild scores should therefore be interpreted as measures of performance under fixed benchmark criteria and as comparative evidence of physical and functional plausibility, not as proof of deployability or safety.

Approximate physical models. Several evaluation components necessarily use simplified physical assumptions. Stability focuses on tipping; assembly feasibility uses discretized insertion paths and intermediate-support checks; and operability uses rigid-body simulation with standardized density, friction, collision geometry, and actuation parameters. These choices make evaluation deterministic and comparable across heterogeneous structures, while isolating geometric and kinematic feasibility from uncertainty in inferred physical properties. Nevertheless, different parameterizations or higher-fidelity models may change individual scores, especially for structures whose operation depends on deformable components, material-specific behavior, or complex contact. Future work should study metric sensitivity and ranking stability under alternative thresholds and simulation parameters, and validate a representative subset through physical fabrication, robotic assembly, or expert mechanical assessment. Benchmark coverage. The experiments in this paper use the 50-object LMBuild-Core split. This split is deliberately coverage-oriented: it spans multiple structural families, articulation regimes, construction paradigms, and difficulty levels rather than reproducing the empirical frequency distribution of any one source dataset. Consequently, the reported results characterize performance across the selected construction regimes but should not be interpreted as population-level estimates over all real-world objects. The 2,399-object LMBuild-Full split is intended to support broader evaluation, including category-level analysis, rare mechanisms, and more difficult product-scale assemblies. Extending evaluation to the full split and to additional domains such as compliant mechanisms, electronics, and fluid-powered systems is an important direction.

Automated evaluation and human calibration. Design Quality combines deterministic measurements with two VLM judges calibrated against ratings from four human annotators. On held-out designs, the resulting metrics preserve system-level human rankings well, while item-level correlations and human-human agreement show that individual designs remain inherently more ambiguous. The present calibration covers a limited set of objects and systems and may not capture every valid design style or decomposition strategy. In addition, Wikipedia-derived capability sheets provide an auditable source of real-world knowledge but may omit uncommon components or alternative realizations. Future versions should broaden the annotator and system pool, include domain experts where appropriate, report uncertainty for agreement statistics, and continue expanding accepted component synonyms, material alternatives, and functional realizations.

System comparability and statistical uncertainty. General-purpose agents receive the same task specification, environment semantics, tool definitions, and budgets, but provider-specific execution interfaces and modality support necessarily differ. Domain-specific systems additionally retain their native inputs and outputs and are therefore comparable only on shared applicable metrics. Unless otherwise stated, the current results use one final generation trajectory per model-task pair. Confidence intervals obtained by resampling benchmark objects quantify variation across tasks, but not within-model stochasticity across repeated generations. Future evaluation should include repeated

runs where computationally feasible, standardized native tool-calling interfaces, and reporting of both task-level and run-level uncertainty.

Outlook. Beyond scaling models and tool budgets, future work should improve the knowledge, skills, and creative physical reasoning that models bring to structure generation. First, models could internalize reusable construction knowledge, including geometric constraints, assembly rules, component affordances, and functional properties, rather than rediscovering them at inference time. Recent work on textual and multimodal policy internalization suggests that such structured knowledge can be incorporated more directly into model behavior (Liu et al., 2026b; Wang et al., 2026). Similar methods could provide stronger priors for joints, interfaces, load-bearing structures, clearances, and other recurring physical constraints. Second, building agents could acquire reusable procedural skills through exploration and interaction. Work on environment exploration, recursive self-improvement, and reasoning skills shows that agents can discover and retain useful strategies from experience (Liu et al., 2026c; Zhu et al., 2026a; Zhao et al., 2026). In LMBuild, agents could similarly explore alternative construction strategies, learn from failed checks or simulations, and accumulate reusable skills for creating parts, joints, and mechanisms. Third, future systems should support more creative physical construction, including inventing functional parts, repurposing components, and refining designs through multi-turn interaction. Recent work highlights the difficulty of affordance-based creative reasoning in multimodal models (Qian et al., 2026b;a). Combining such reasoning with iterative tool use, execution feedback, and code-based construction interfaces could enable agents to propose, test, and revise increasingly functional designs (Wang et al., 2024; Yang et al., 2024). A complementary direction is to close the loop between digital generation and physical fabrication. Future extensions could incorporate additive-manufacturing constraints such as minimum feature size, wall thickness, overhangs, tolerances, material choice, and build volume, and evaluate whether generated structures can be successfully printed, assembled, and operated. Physical validation would enable direct measurement of print success, dimensional accuracy, assembly feasibility, structural integrity, and functional performance, while supporting sim-to-real calibration of LMBuild’s evaluation metrics. Beyond 3D printing, future work could consider additional manufacturing processes, richer material and contact mechanics, fasteners, compliant components, and electrical or fluid routing. Expanding evaluation to LMBuild-Full, repeated generations, and evaluator-sensitivity analysis would further improve coverage and reliability. Ultimately, LMBuild could serve not only as a benchmark for buildable generation, but also as an environment for studying how agents acquire the knowledge, skills, and creative physical intelligence required to construct functional real-world artifacts.